

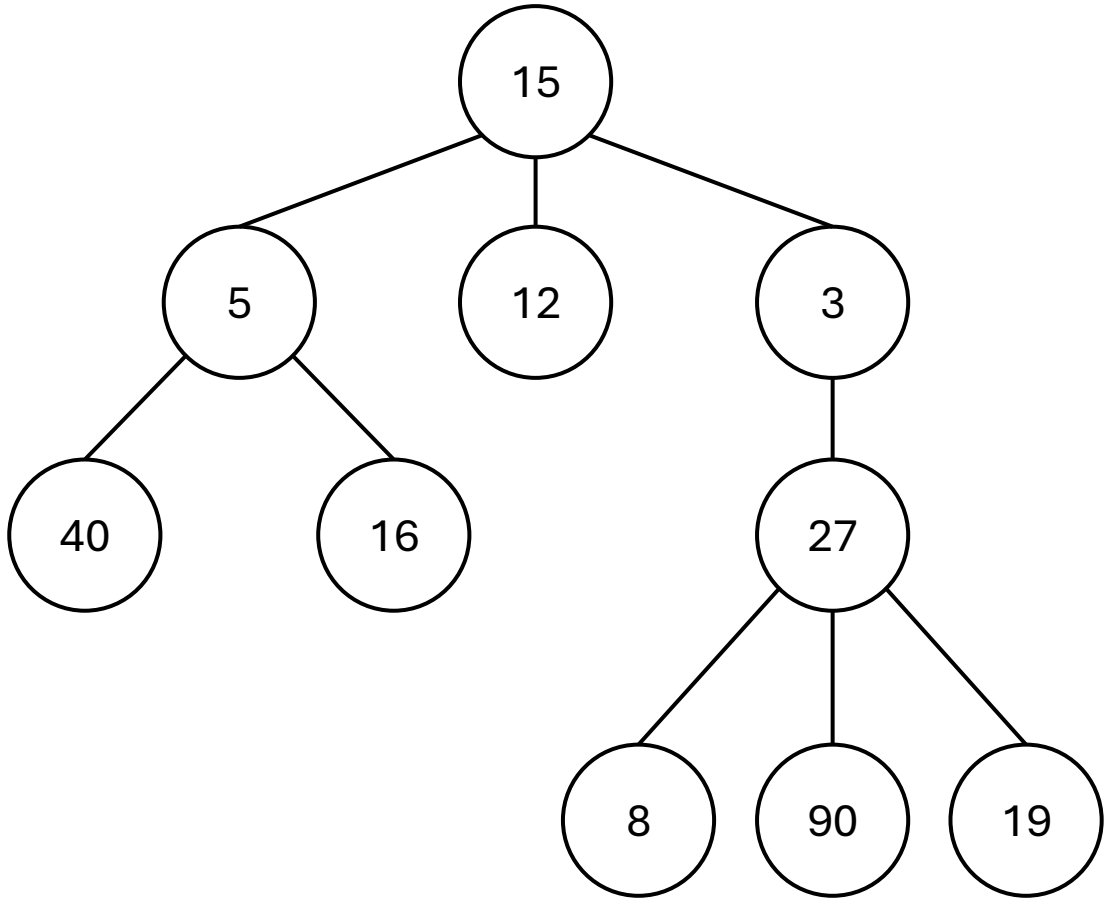
Bilgisayar Mühendisliği Bölümü
BIMU2057 Veri Yapıları

Dr. Öğr. Üyesi Nihal ALTUNTAŞ

nihal.altuntas@iuc.edu.tr

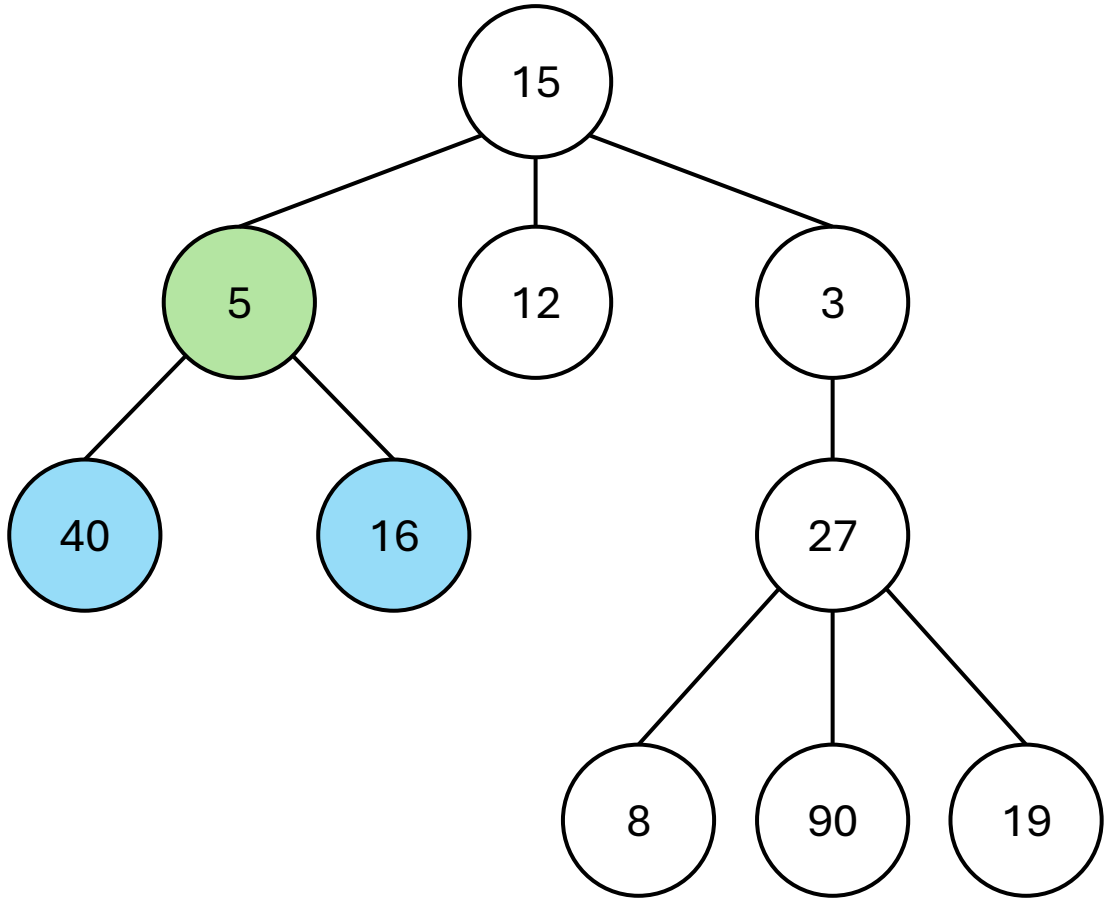
Ağaç (Tree)

- Düğümlerin hiyerarşik olarak bağlandığı lineer olmayan bir veri yapısıdır.
  - Düğüm bağlantıları arasında ebeveyn-çocuk (parent-child) ilişkisi vardır.
  - N düğüm bulunan bir ağaçta N-1 kenar (edge) vardır.
  - Ağaçtaki iki düğüm arasında tam olarak bir bağlantı yolu (path) bulunur.



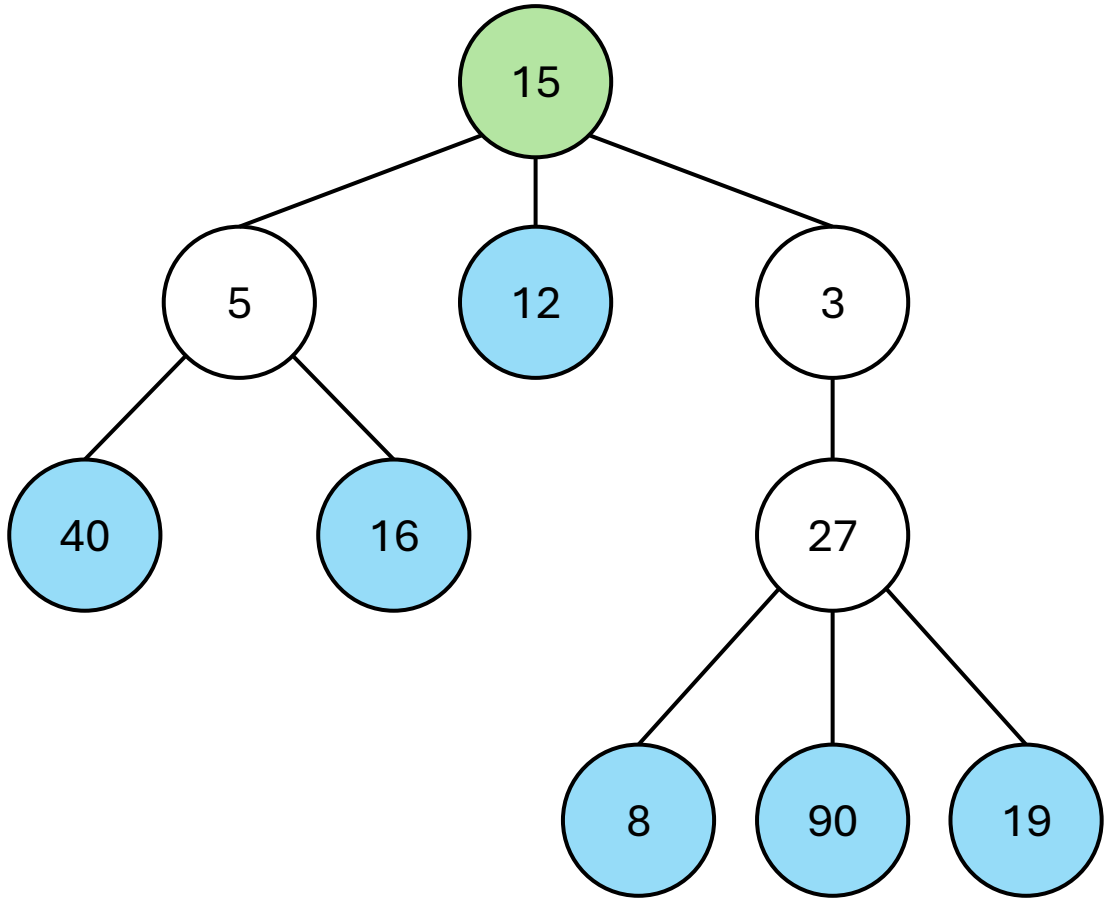
Ağaç (Tree) - Terminoloji

- Ebeveyn Düğüm (Parent Node)
  - Hiyerarşide bir düğümün hemen üstündeki düğümdür.
    - 5 nolu düğüm, 40 ve 16 nolu düğümlerin ebeveynidir.
- Çocuk Düğüm (Child Node)
  - Hiyerarşide bir düğümün hemen altindaki düğümdür.
    - 40 ve 16 nolu düğümler, 5 nolu düğümün çocuğudur.
- Ağaç Boyutu (Tree Size)
  - Ağaçtaki düğüm sayısıdır.
    - Yandaki ağaç için 10'dur.



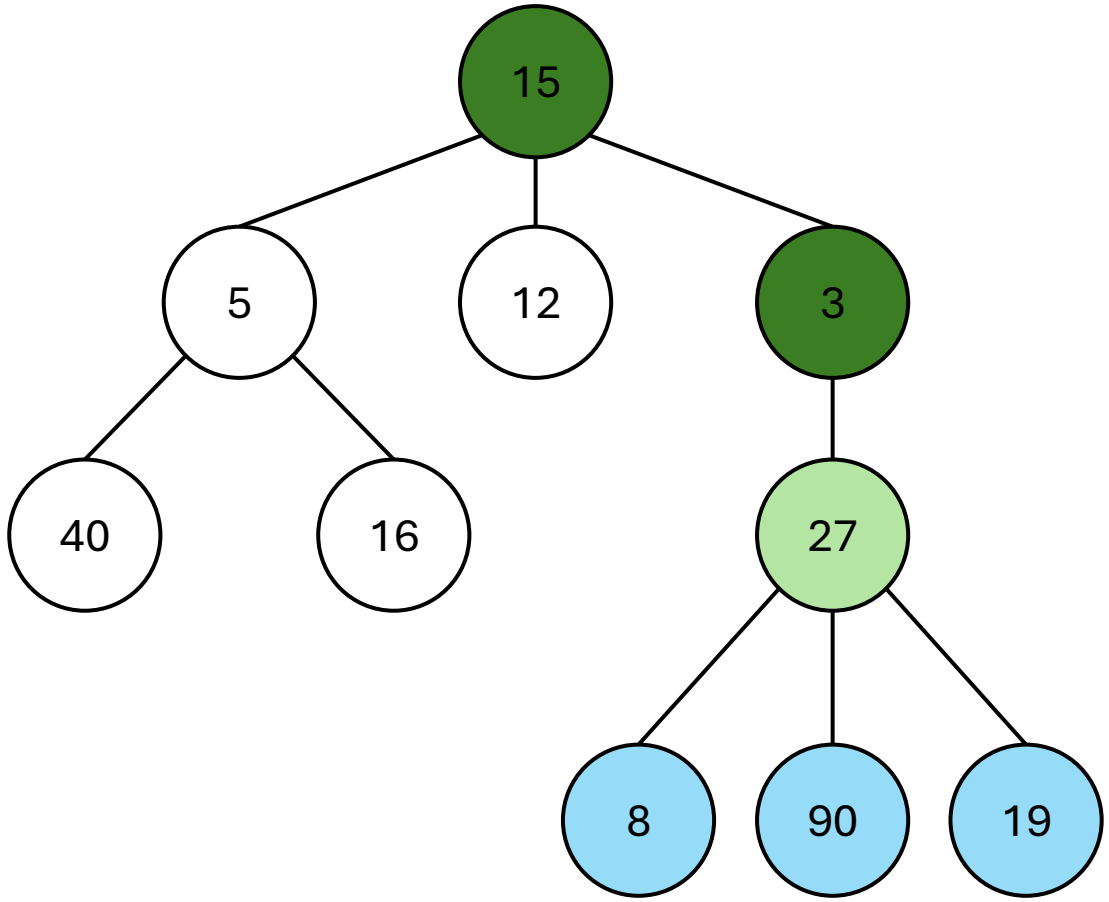
Ağaç (Tree) - Terminoloji

- Kök Düğüm (Root Node)
  - Hiyerarşide en tepedeki düğümdür.
  - Bir ağaçta tek bir kök bulunur.
    - 15 nolu düğüm
- Yaprak Düğüm (Leaf Node)
  - Çocuğu olmayan düğümlerdir.
    - 12, 40, 16, 8, 90 ve 19 nolu düğümler
- Ara Düğüm (Internal Node)
  - Bir ebeveyni ve en az bir çocuğu olan düğümlerdir.
    - 5, 3 ve 27 nolu düğümler



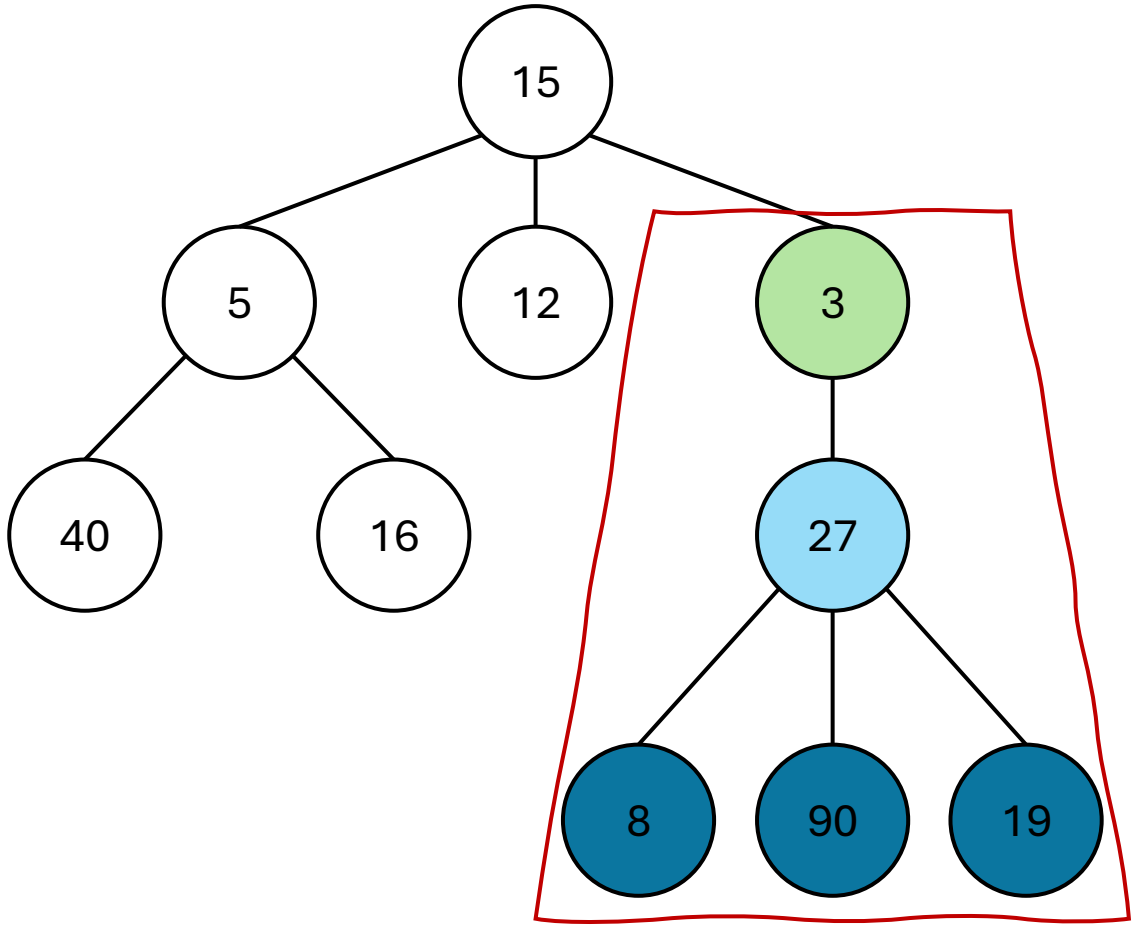
Ağaç (Tree) - Terminoloji

- Ata/Üst Düğüm (Ancestor Node)
  - Verilen düğümden köke kadar olan tüm düğümlerdir.
    - 8 nolu düğümün üst düğümleri: 27, 3 ve 15
- Torun/Alt Düğüm (Descendant Node)
  - X düğümü Y düğümünün üst düğümü ise Y düğümü X düğümün alt düğümüdür.
    - 3 nolu düğümün alt düğümleri: 27, 8, 90, 19
- Kardeş Düğüm (Sibling Node)
  - Aynı ebeveyne sahip düğümlerdir.
    - 8, 90 ve 19 nolu düğümler kardeştir.



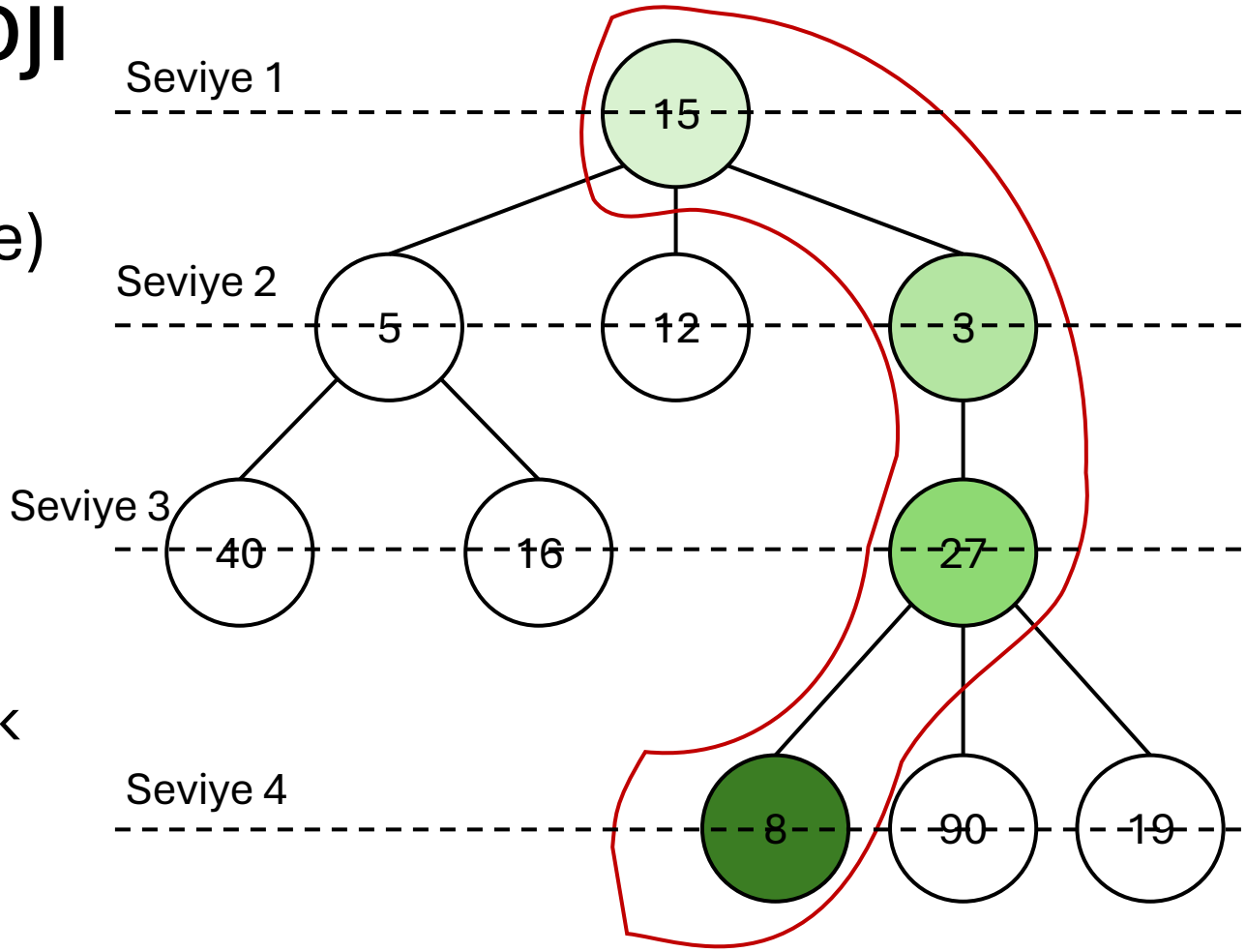
Ağaç (Tree) - Terminoloji

- Altağaç (Subtree)
  - Ağaç içerisindeki bir aradüğüm ve bu düğümün tüm alt düğümlerinden oluşan ağaçtır.
- Düğüm Derecesi (Degree of a Node)
  - Bir düğümün sahip olduğu alt ağaç (çocuk) sayısı, o düğümün derecesini verir.
    - 3 nolu düğümün derecesi: 1
    - 27 nolu düğümün derecesi: 3
  - Yaprak düğümlerin derecesi sıfırdır.



Ağaç (Tree) - Terminoloji

- Düğüm Seviyesi (Level of a Node)
  - Bir düğümden kök düğüme giden yoldaki düğüm sayısı
    - Kök düğüm seviyesi 1 kabul edilir.
- Ağacın yüksekliği (Height of the Tree)
  - Kökten en alt seviyedeki bir yaprak düğüme kadar olan düğüm sayısı.
    - Yandaki ağacın yüksekliği: 4



Ağaç Yapısı Kullanım Alanları

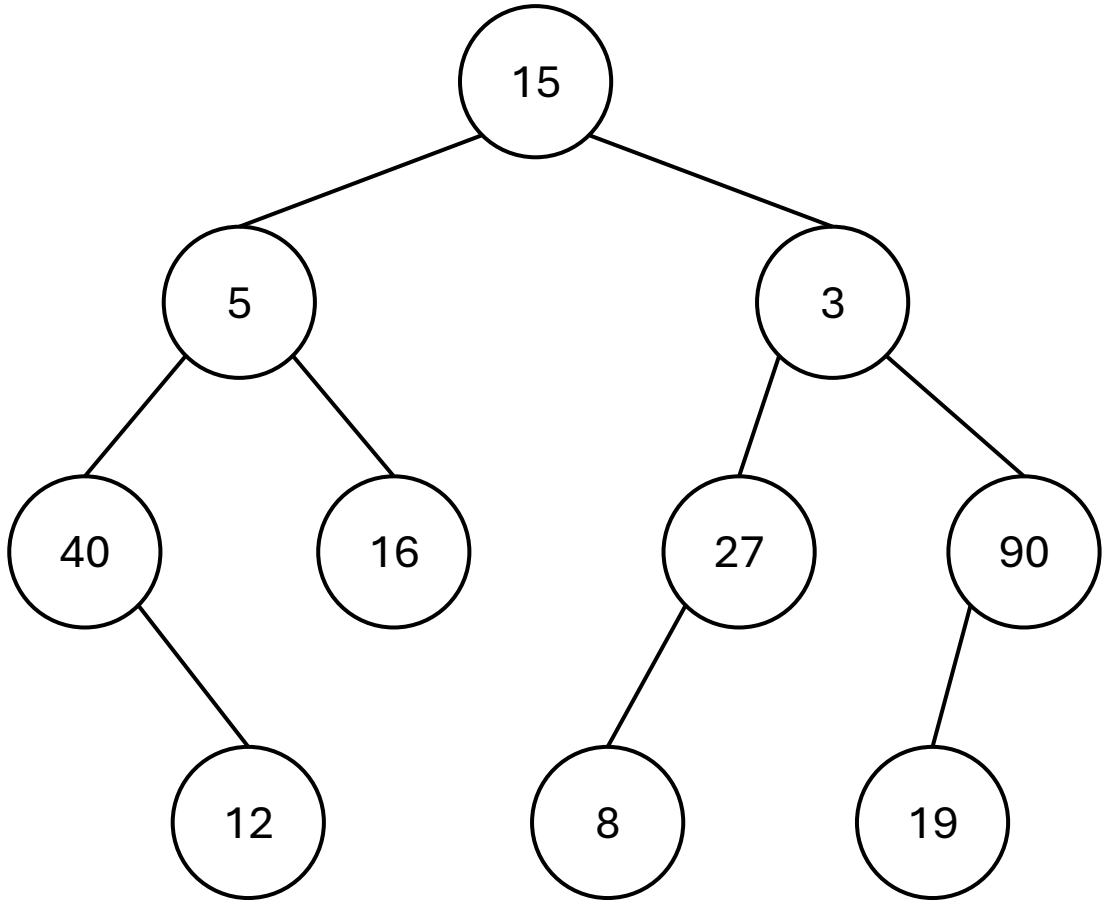
- Hiyerarşik düzen gerektiren uygulama alanlarında ağaç yapısı kullanılır.
  - Bilgisayarlardaki dosyalama sistemi: İçi içe klasörler ve bu klasörlerin içerisinde bulunan dosyalar ağaç yapısında saklanır.
  - HTML/XML dosyalarının içeriklerinin modellenmesi: <html>, <head>, <body> gibi tag'lerin her biri ağaç yapısında bir düğüm olacak şekilde tanımlanır.
- Arama ve sıralama gibi işlemlerin hızlandırılması için tercih edilir.
  - Array ve linked list gibi lineer sistemler arama kullanıldığında N veri içerisinde arama işlem karmaşıklığı O(N)'dir.
  - Ağaç yapısı kullanıldığında bu karmaşıklık O(logN) olarak iyileşir.
  - Bu nedenle veri tabanları ve indeksleme gereken durumlarda tercih edilir.

Ağaç Çeşitleri

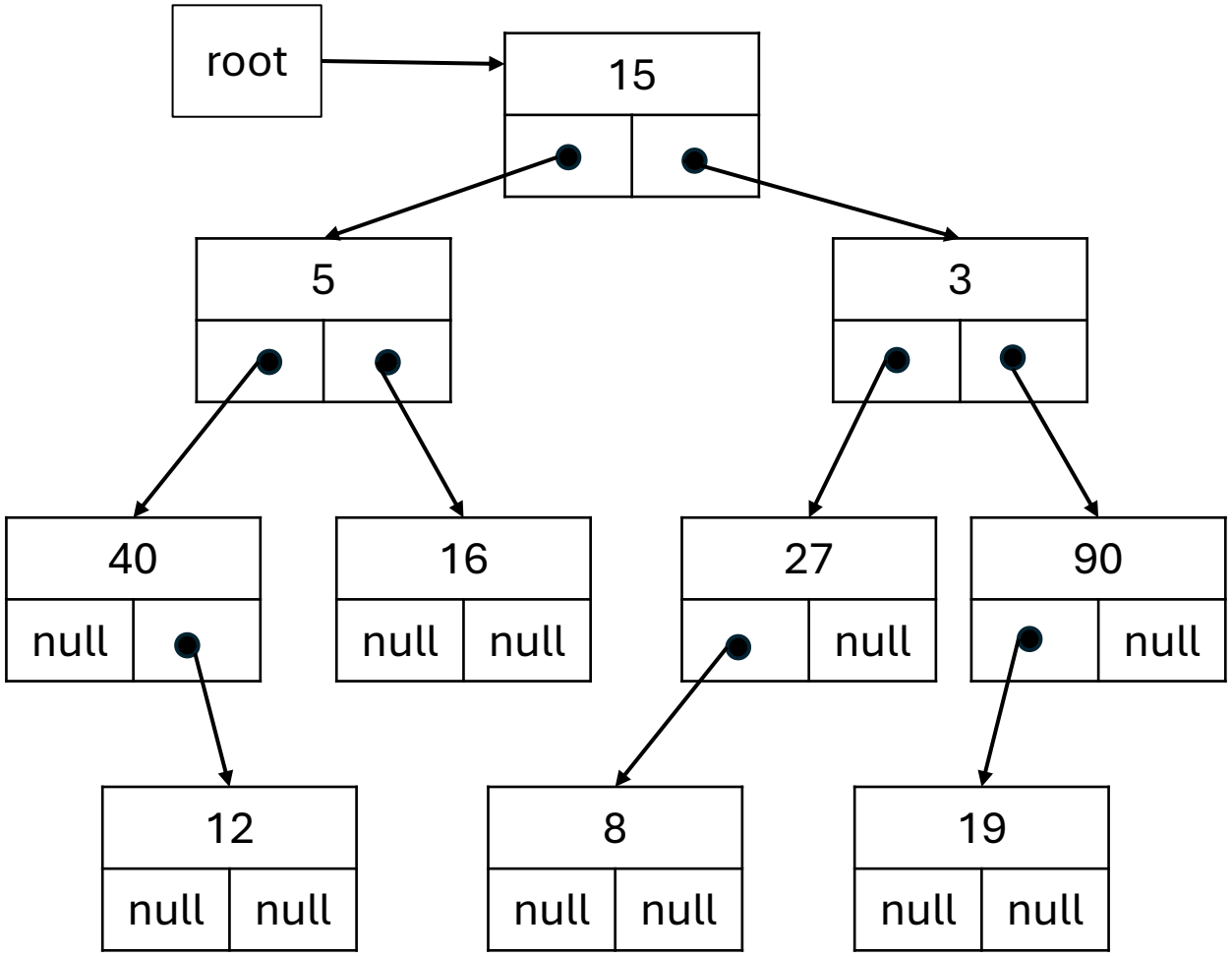
- İkili Ağaç (Binary Tree)
  - Binary Search Tree
  - Heap Tree
  - AVL Tree
  - Red Black Tree
- Üçlü Ağaç (Ternary Tree)
- Çoklu Ağaç (N-ary Tree)

İkili Ağaç (Binary Tree)

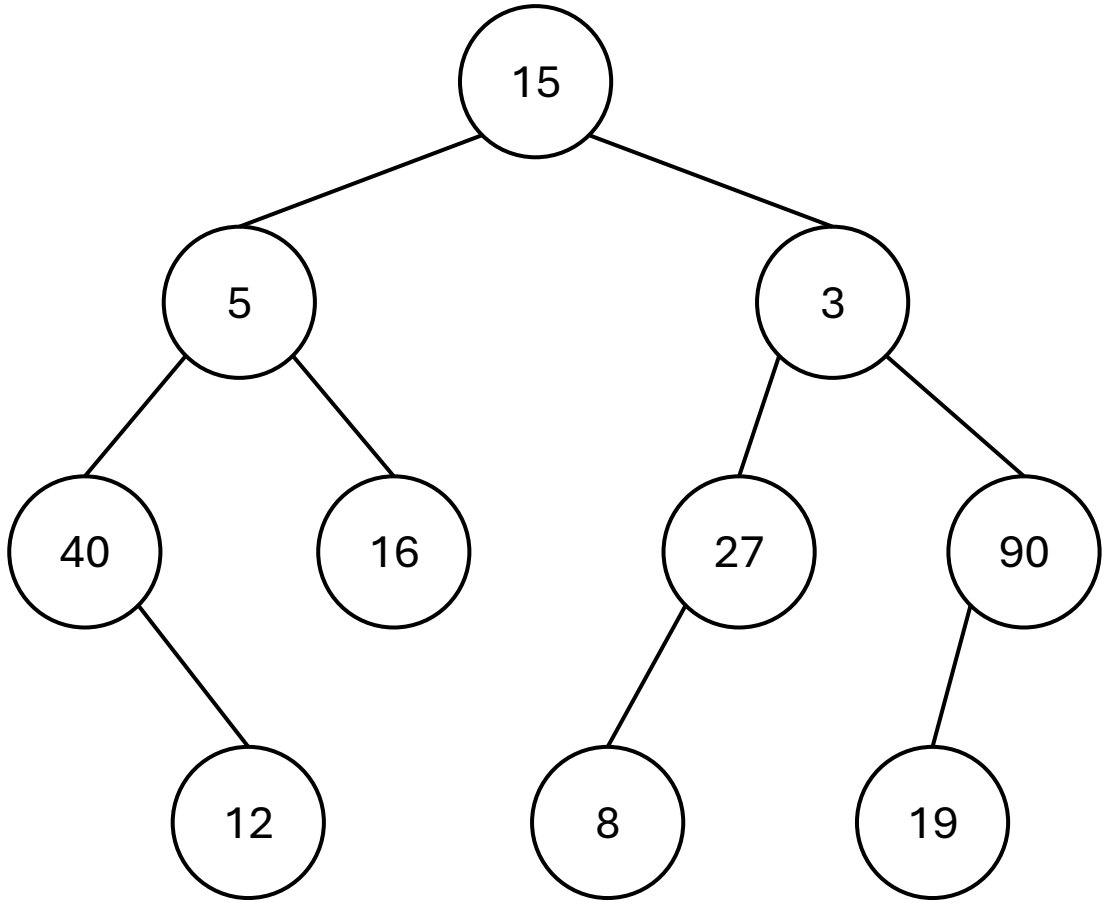
- Her düğümün en fazla iki çocuğu vardır.
  - Sol çocuk (left child) ve sağ çocuk (right child)
- Sol ve sağ çocuklar birbirinden farklı yorumlanır.
  - Tek bir çocuk olsa bile sol/sağ belirtilmelidir.



İkili Ağaç (Binary Tree)



Fiziksel gösterim

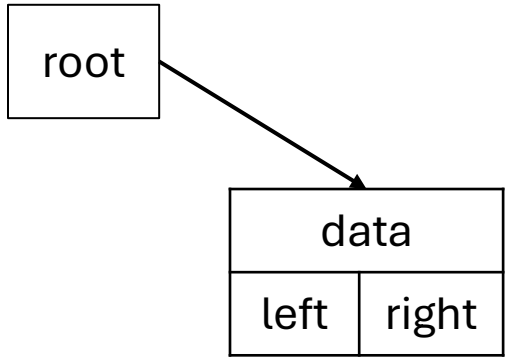


Kavramsal gösterim

İkili Ağaç (Binary Tree) Uygulama C++

```
struct Node {
    int data;
    Node* left = nullptr;
    Node* right = nullptr;
};

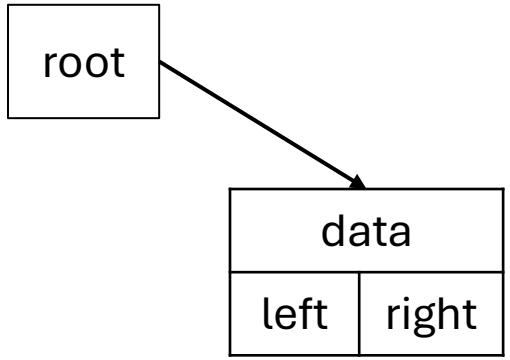
struct BinaryTree {
    Node* root = nullptr;
};
```



İkili Ağaç (Binary Tree) Uygulama Python

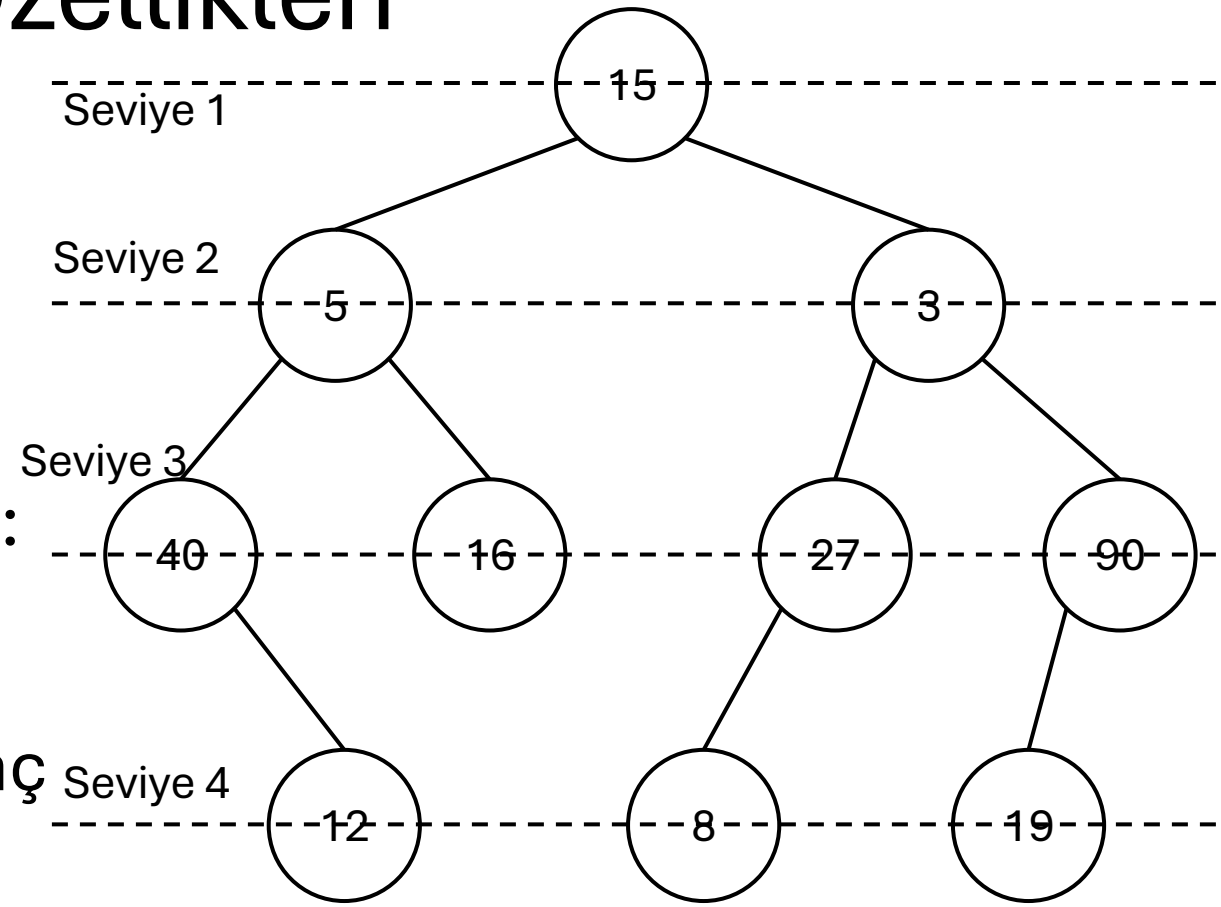
```
class Node:
    def __init__(self, d):
        self.data = d
        self.left = None
        self.right = None

class Binary_Tree:
    def __init__(self):
        self.root = None
```



İkili Ağaç (Binary Tree) Özellikleri

- Her bir s seviyesinde olabilecek maksimum düğüm sayısı: 2^{s-1}
  - $2^{1-1} = 1$, $2^{2-1} = 2$, $2^{3-1} = 4$, $2^{4-1} = 1$
- Yüksekliği y olan bir ikili ağaçta bulunan maksimum düğüm sayısı: $2^y - 1$
  - $2^4 - 1 = 15$
- N adet düğüm bulunan bir ikili ağaç için gereken minimum yükseklik: $\lfloor log_2 N \rfloor + 1$
  - $\lfloor log_2 10 \rfloor + 1 = 4$

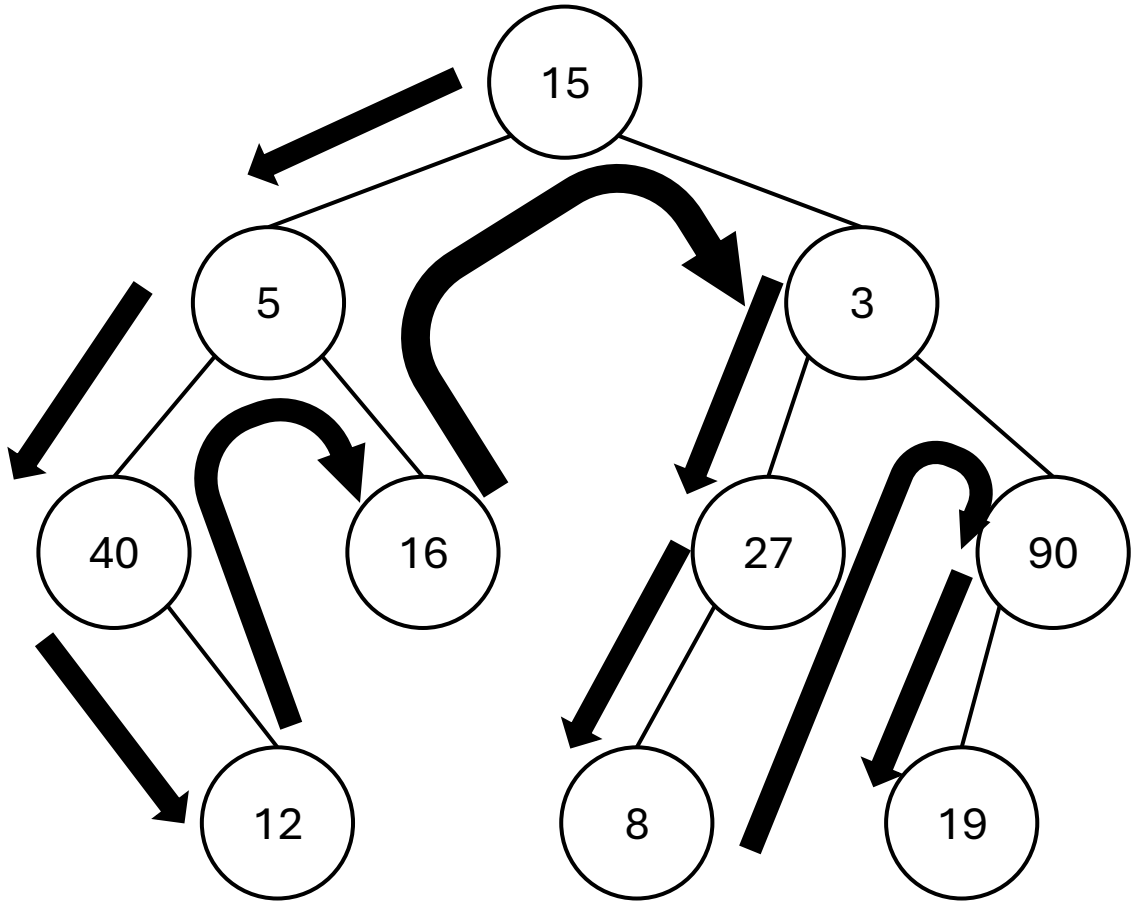


İkili Ağaç (Binary Tree) İşlemler

- Dolaşma (Traversal)
  - Derinlik Öncelikli Dolaşma (Depth-First Search (DFS) Traversal)
  - Genişlik Öncelikli Dolaşma (Breadth-First Search (BFS) Traversal)
- Arama
- Düğüm Ekleme ve Silme

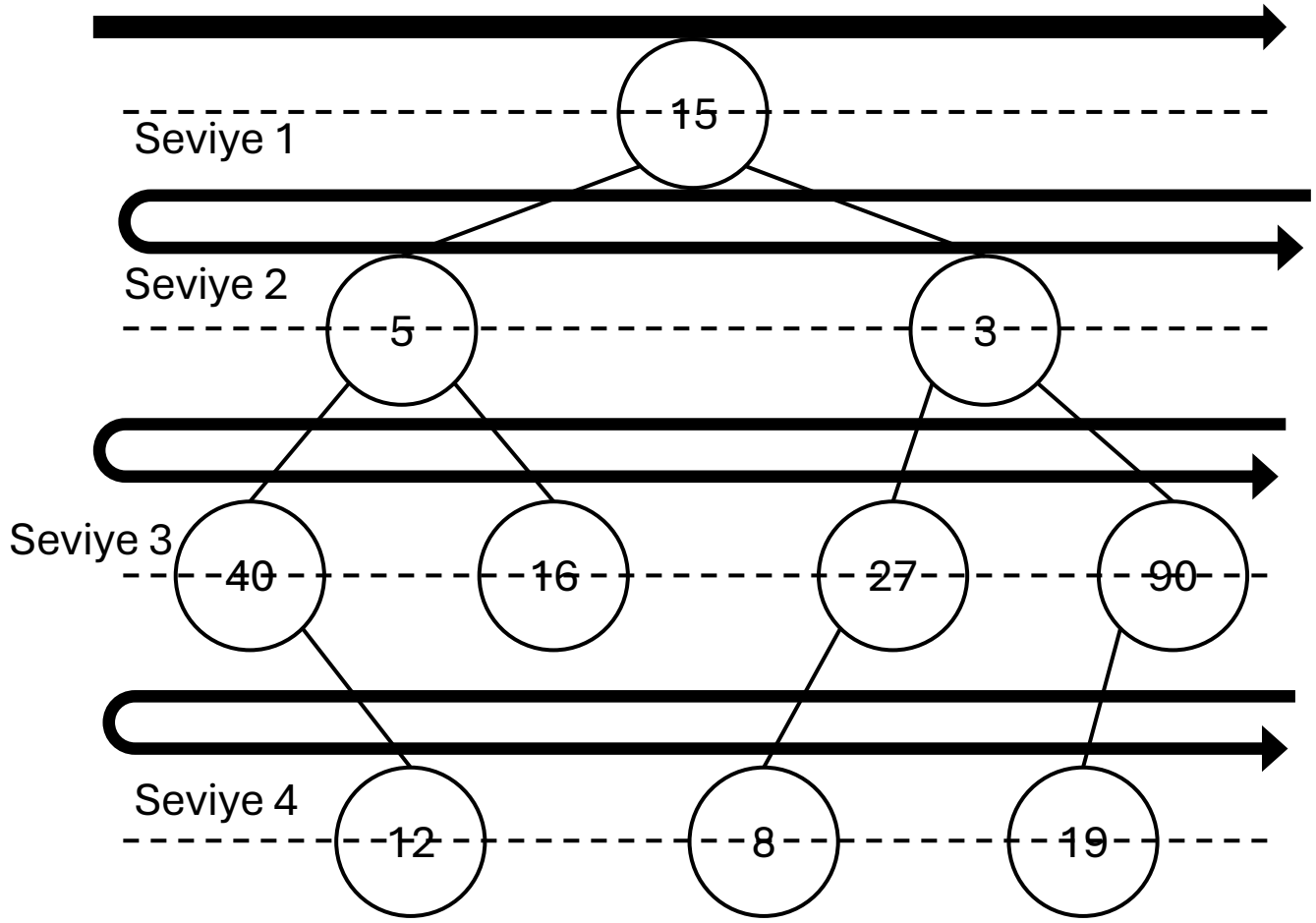
Derinlik Öncelikli (Depth-First) Dolaşma

- Ağaç tarama algoritmalarının çoğu recursive olarak derinlik öncelikli yöntemi kullanır.
- Önce sol koldan tüm düğümler mümkün olan en uzak düğüme (yaprak) kadar ziyaret edilir.
- Bir yaprağa ulaşıldığında, tarama algoritması geri gelir (backtrack) ve sağ koldan tüm diğer ziyaret edilmemiş düğümler ziyaret edilir.



Genişlik Öncelikli (Breadth-First) Dolaşma

- Bazı uygulamalarda genişlik tarama gerekebilir.
- Recursive olmayan bir tarama olup, Seviye Sırası Tarama olarak da isimlendirilir.
- Yardımcı veri yapısı olarak **Kuyruk** kullanılır.
- Kökten itibaren, ağaçtaki düğümler seviye-seviye dolaşılır.



Derinlik Öncelikli (Depth-First) Dolaşma

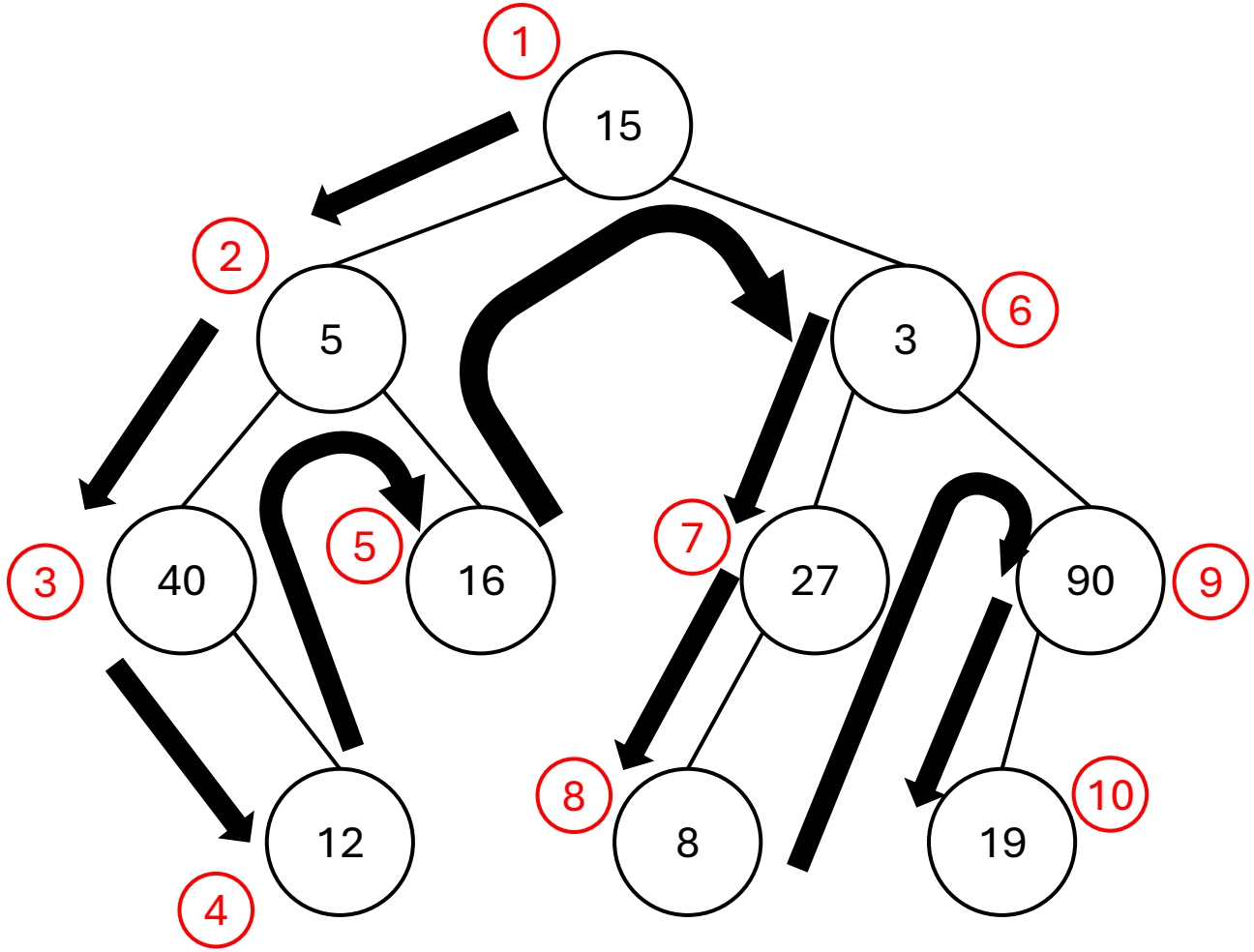
- Depth-first ile ikili ağacın tüm düğümlerine uğrayarak ağacın taranması 3 şekilde yapılabilir.
  - Preorder
  - Inorder
  - Postorder

- Üç tarama türü de tanımları gereği özyinelemeli işlemler şeklindedirler.

Derinlik Öncelikli Preorder

- Tarama sırası
  - Önce düğümün kendisi
  - Sonra sol altağacı
  - Ardından sağ altağacı
- Her düğüm için bu işlem tekrarlanır.

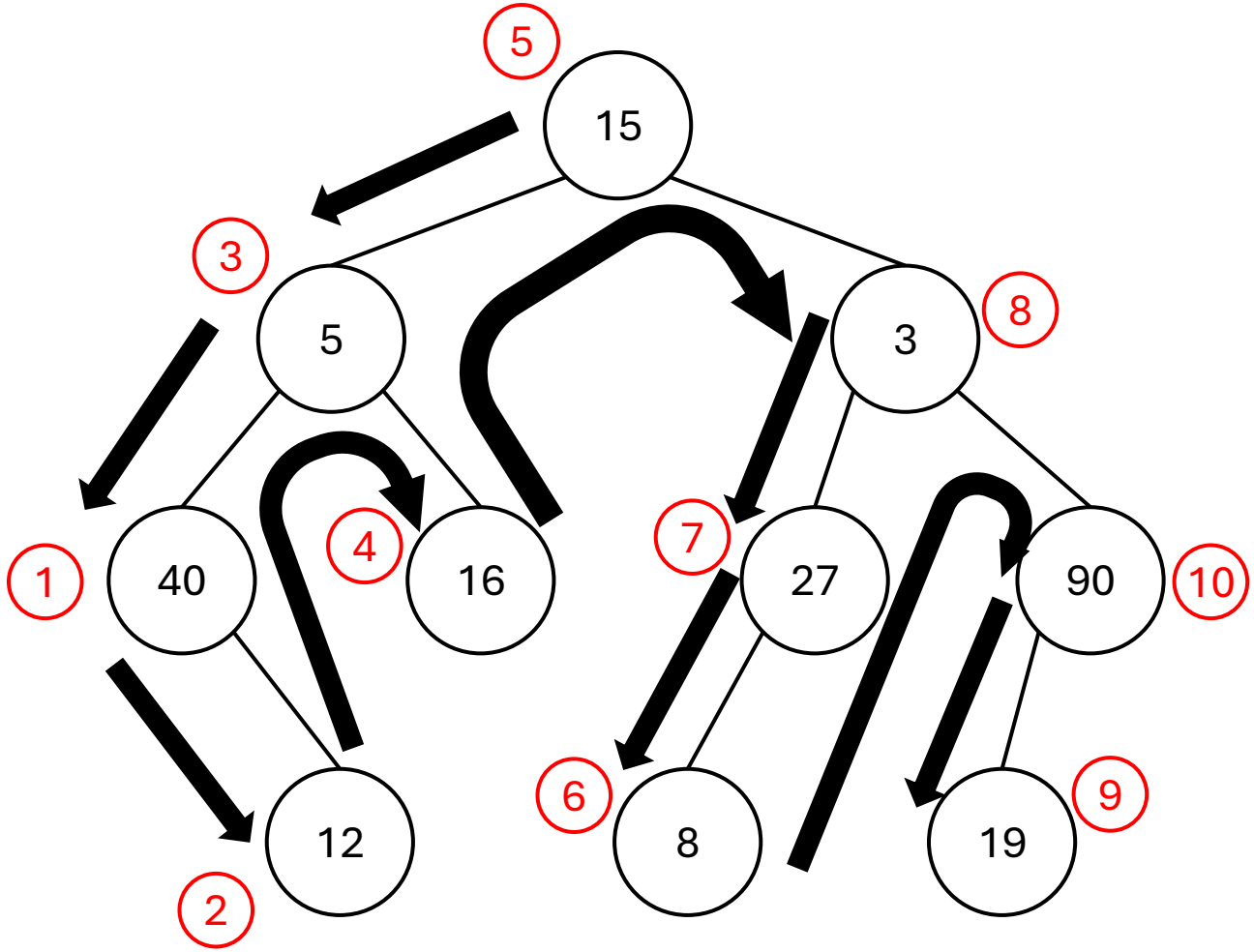
- Ekran çıktısı:
  - 15 5 40 12 16 3 27 8 90 19



Derinlik Öncelikli Inorder

- Tarama sırası
  - Önce sol altağacı
  - Sonra düğümün kendisi
  - Ardından sağ altağacı
- Her düğüm için bu işlem tekrarlanır.

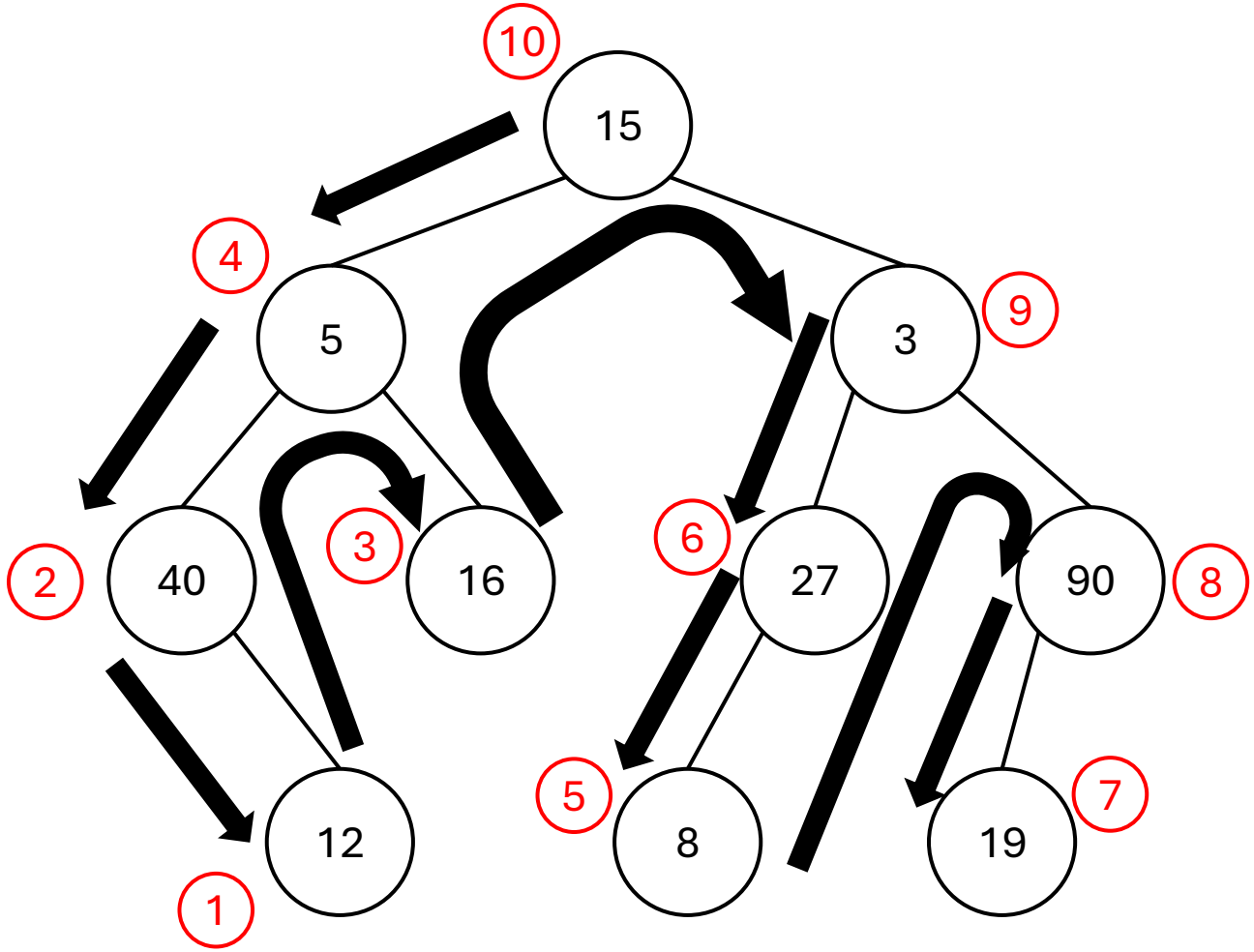
- Ekran çıktısı:
  - 40 12 5 16 15 8 27 3 19 90



Derinlik Öncelikli Postorder

- Tarama sırası
  - Önce sol altağacı
  - Sonra sağ altağacı
  - Ardından düğümün kendisi
- Her düğüm için bu işlem tekrarlanır.

- Ekran çıktısı:
  - 12 40 16 5 8 27 19 90 3 15



Derinlik Öncelikli (Depth-First) C++

- İlgili fonksiyonları ekleyelim

```
struct BinaryTree {
    Node* root = nullptr;
    void dfsPreorder();
    void dfsInorder();
    void dfsPostorder();
};
```

Derinlik Öncelikli (Depth-First) Dolaşma Python

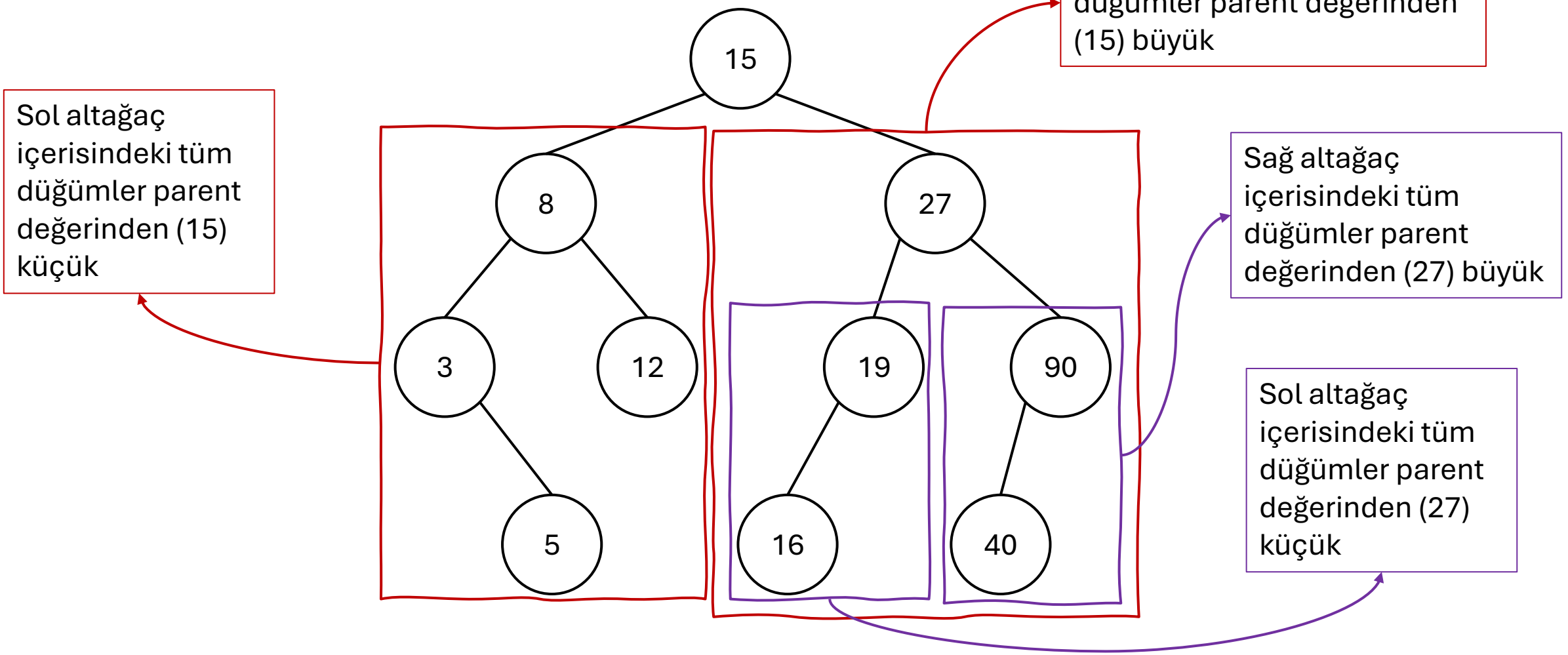
- İlgili fonksiyonları ekleyelim

```
class Binary_Tree:
    def __init__(self):
        self.root = None
    def dfs_preorder_travers(self):
    def dfs_inorder_travers(self):
    def dfs_postorder_travers(self):
```

İkili Sıralama Ağacı (Binary Search Tree)

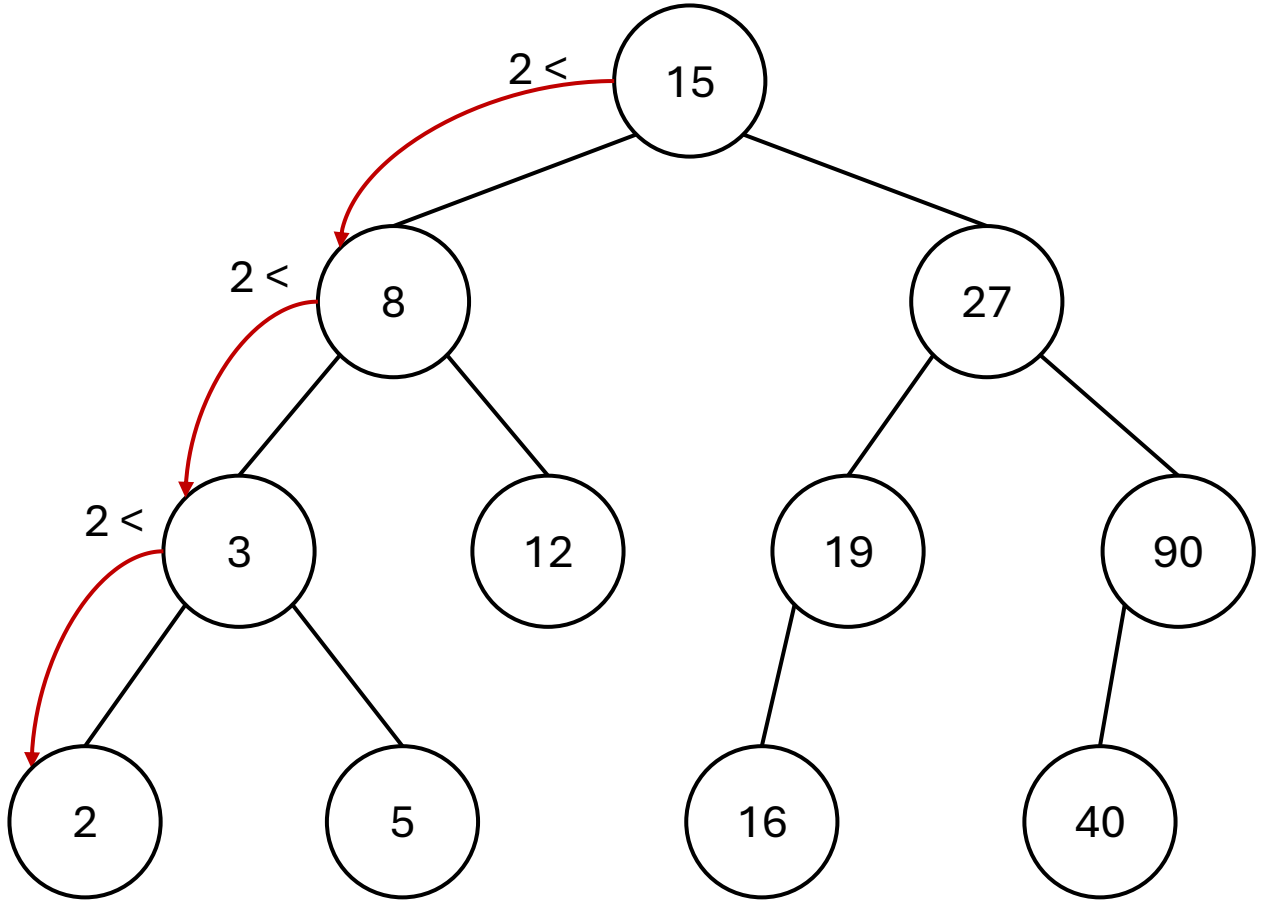
- Binary Searh Tree (BST) düğümlerdeki verileri aşağıdaki kurala göre tutan bir ikili ağaçtır.
  - Herhangi bir düğüm için, sol altağacında bulunan tüm düğümlerin değerleri o düğümün değerinden küçüktür.
  - Herhangi bir düğüm için, sağ altağacında bulunan tüm düğümlerin değerleri o düğümün değerinden büyüktür.
- Bu yapı sıralama, arama, eleman ekleyip çıkarma gibi işlemlerde avantaj oluşturmaktadır.

Binary Search Tree



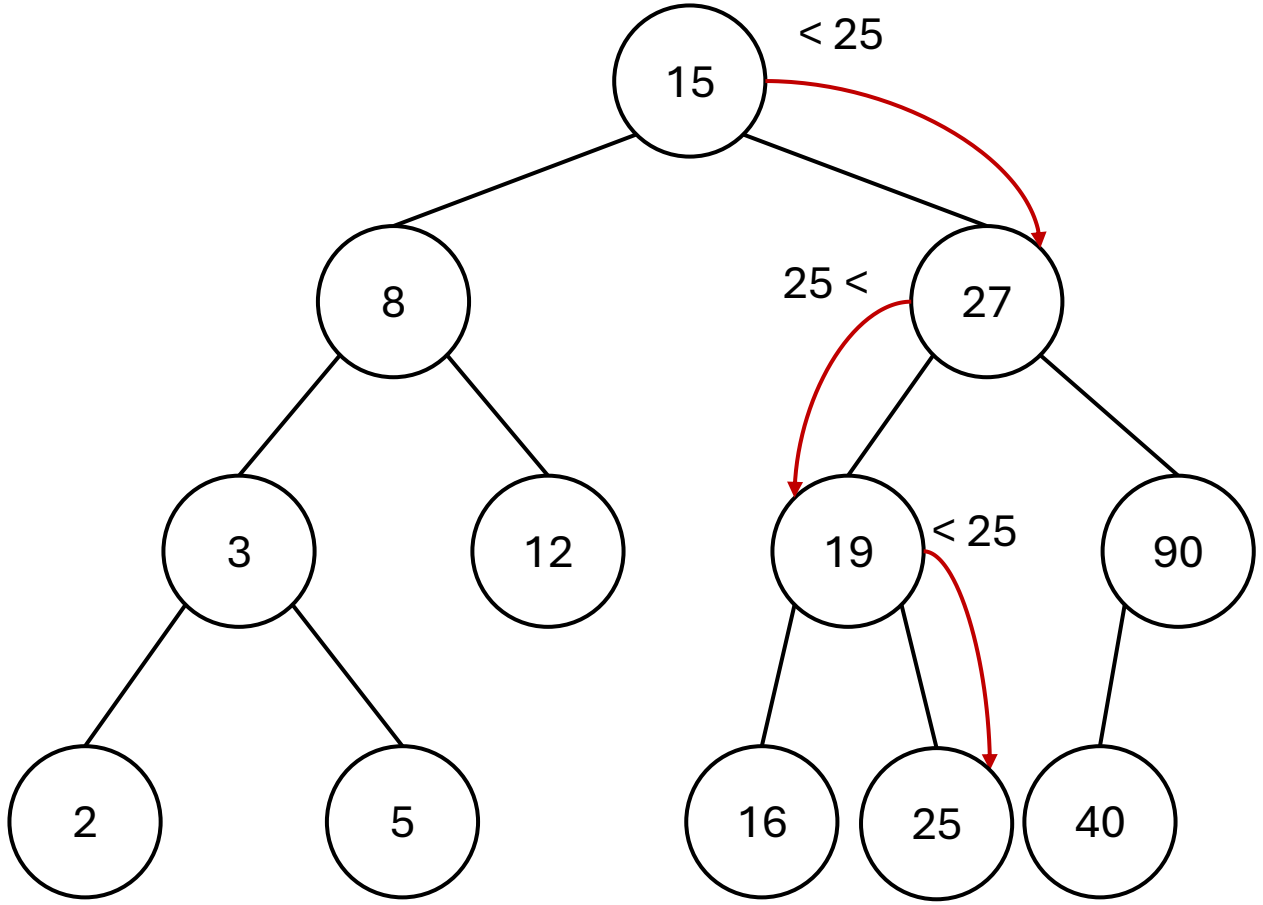
Binary Search Tree Düğüm Ekleme

- Ekle: 2



Binary Search Tree Düğüm Ekleme

- Ekle: 25

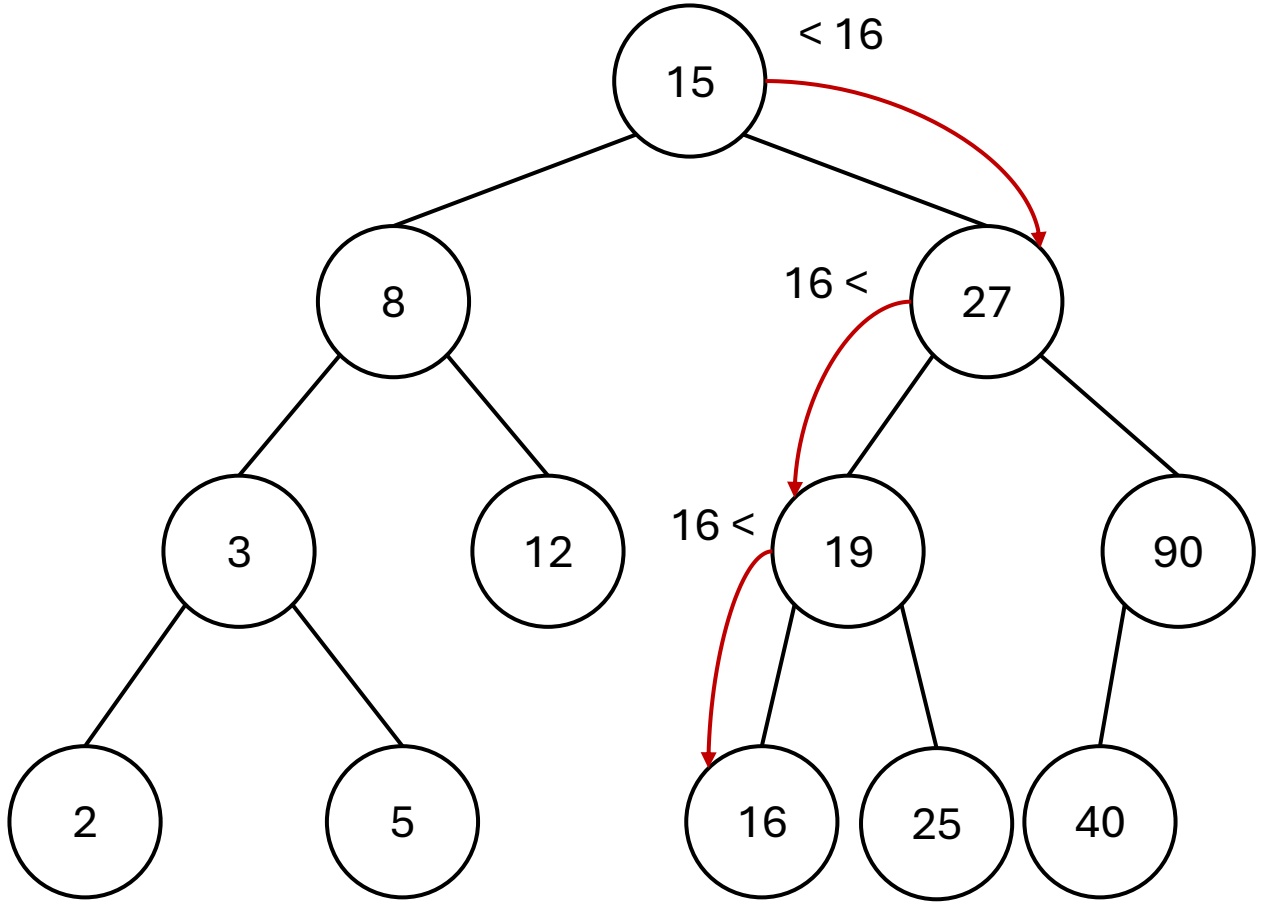


Binary Search Tree Düğüm Silme

- BST ağacında düğüm silme işlemi iki aşamadan oluşur
  - Silinecek düğümü bul
  - Silme işlemini gerçekleştir
    - Silinecek düğüm yaprak (leaf) olabilir.
      - Hiç çocuk yok direkt sil
    - Silinecek düğümde tek bir çocuğu (sol ya da sağ) olabilir.
      - Silinecek düğümün ebeveyni (parent) ile tek çocuğunu bağla
    - Silinecek düğümün iki çocuğu olabilir
      - Farklı yaklaşımlar var
      - Sağ altağaçtaki en küçük (kendisinden bir büyük) düğüm ile yer değiştir ve ilgili düğümü sil
      - Sol altağaçtaki en büyük (kendisinden bir küçük) düğüm ile yer değiştir ve ilgili düğümü sil
      - Sağ altağacı kendi yerine kaydır, sol altağacı ise sağ altağaca ekle
      - Sol altağacı kendi yerine kaydır, sağ altağacı ise sol altağaca ekle

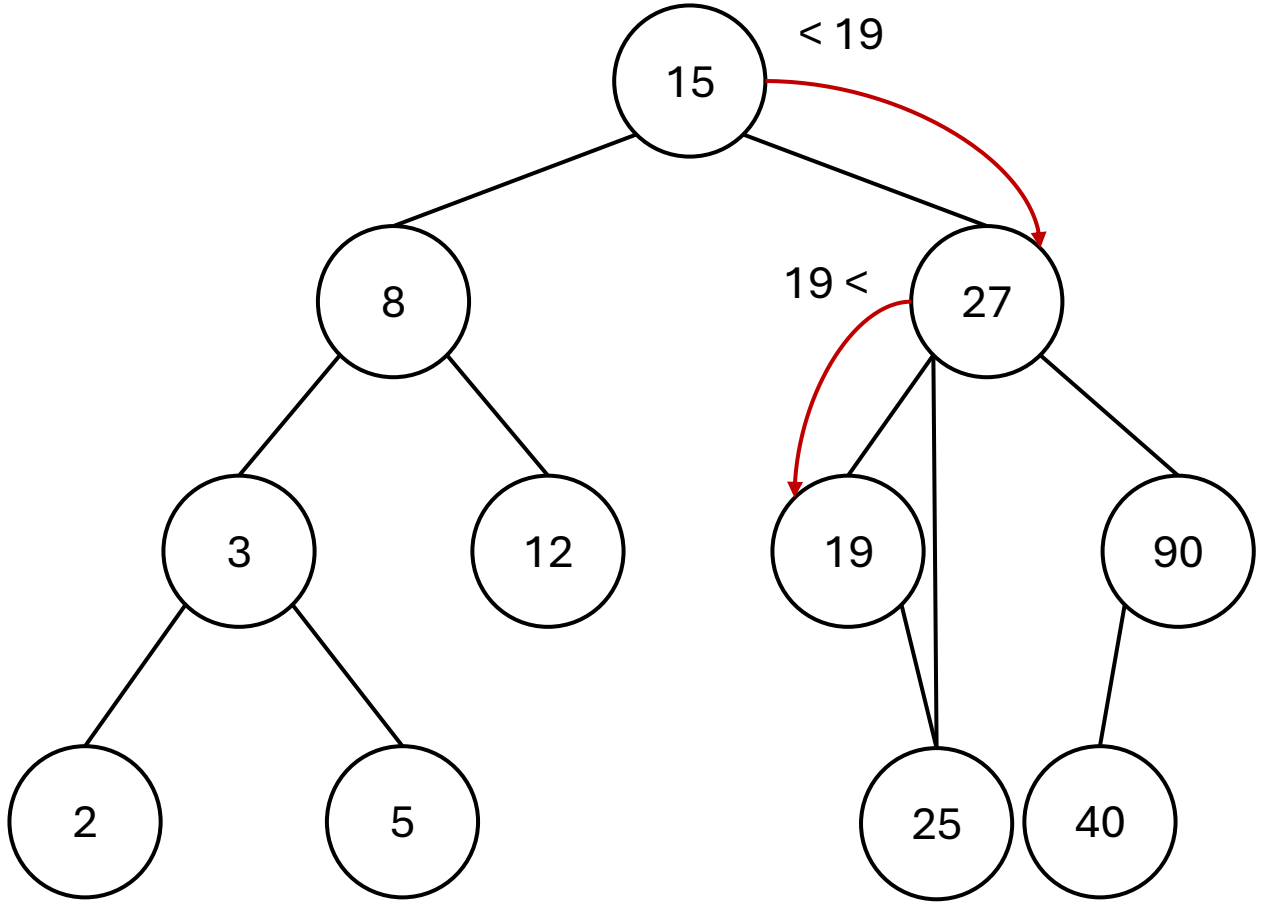
Binary Search Tree Düğüm Silme

- Sil: 16
- Yaprak düğüm



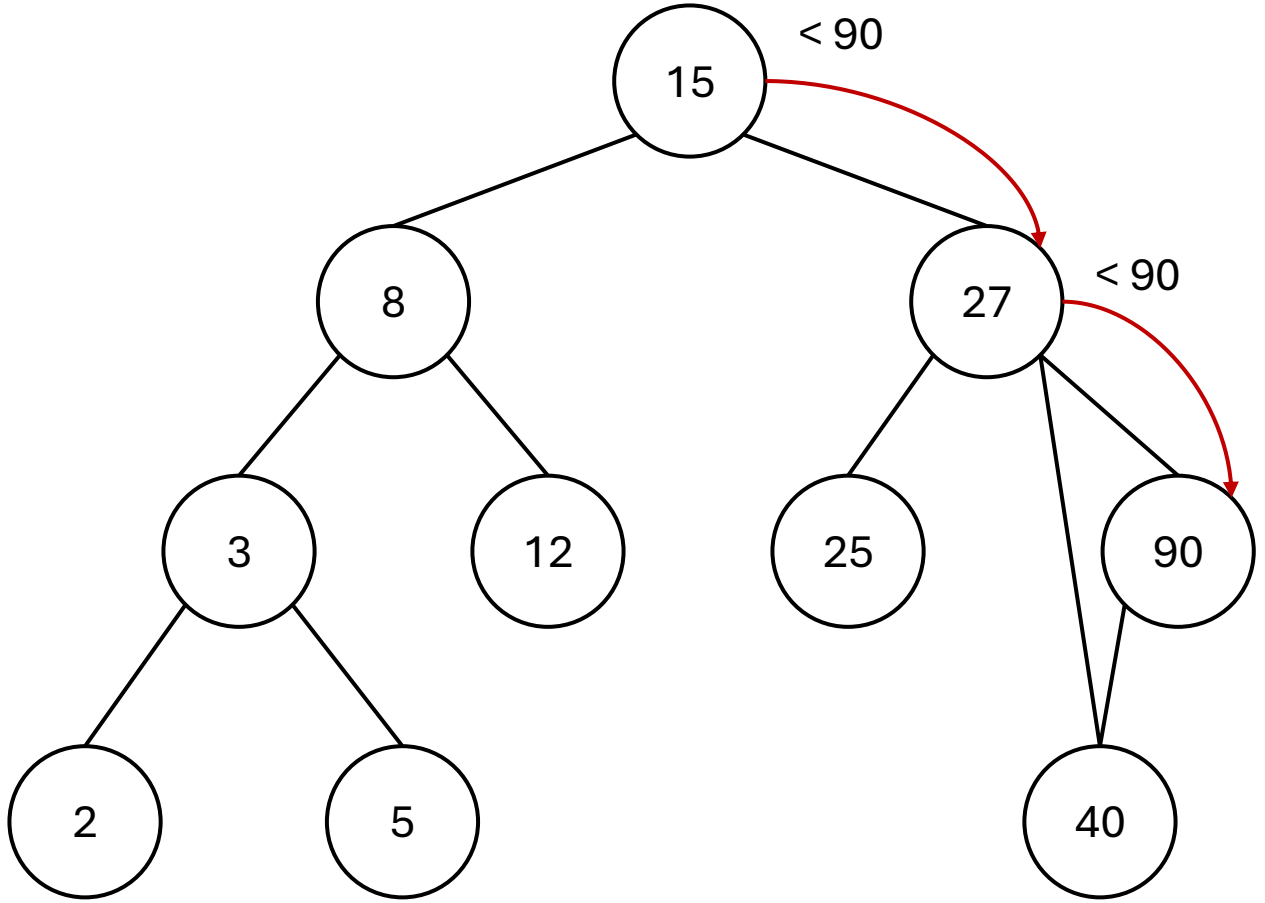
Binary Search Tree Düğüm Silme

- Sil: 19
- Sadece sağ çocuk var



Binary Search Tree Düğüm Silme

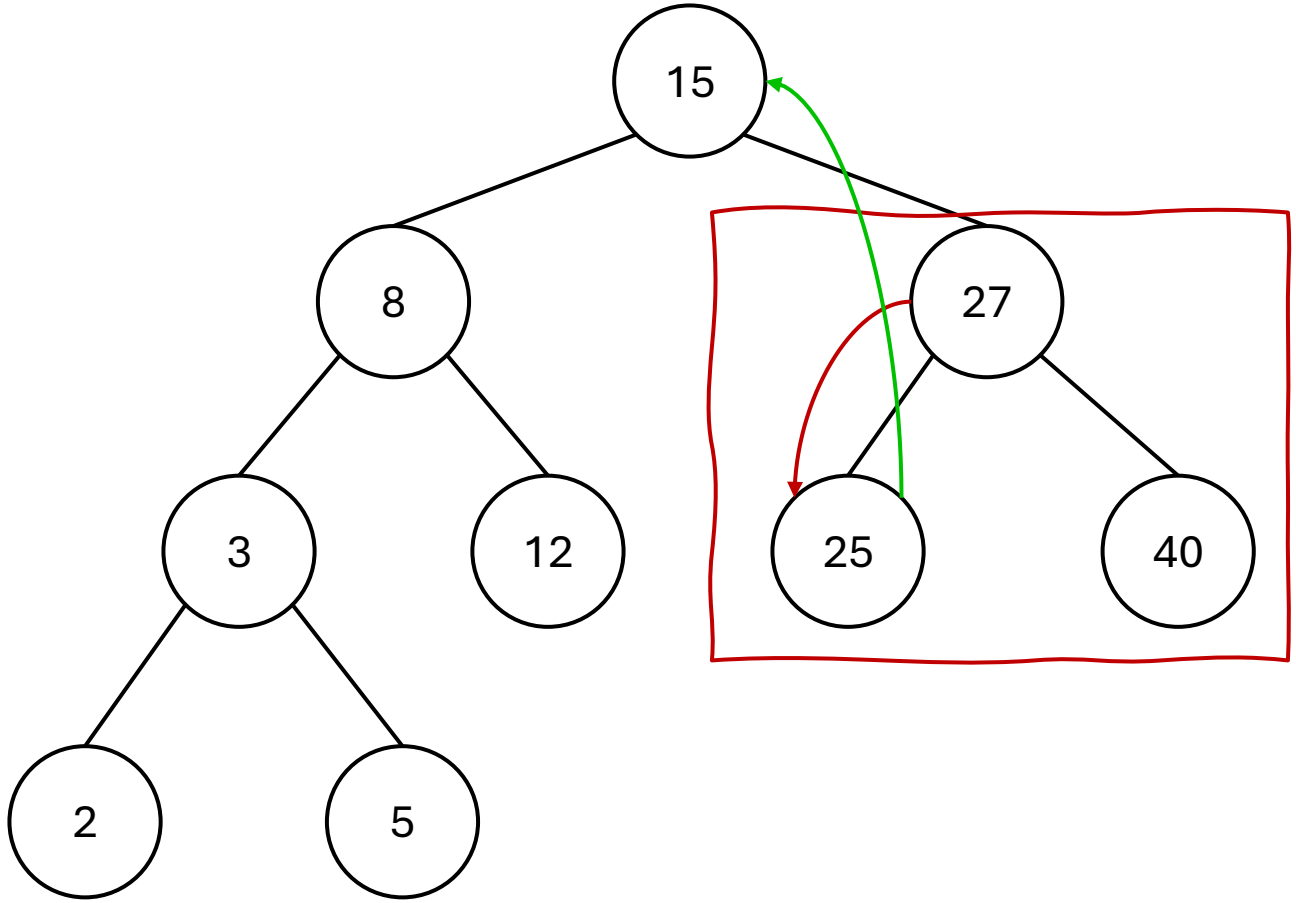
- Sil: 90
- Sadece sol çocuk var



Binary Search Tree Düğüm Silme

- Sil: 15
- İki çocuk da var (sol ve sağ)
  - **Sağ altağaçtaki en küçük (kendisinden bir büyük) düğüm ile yer değiştir**
  - İlgili düğümü sil

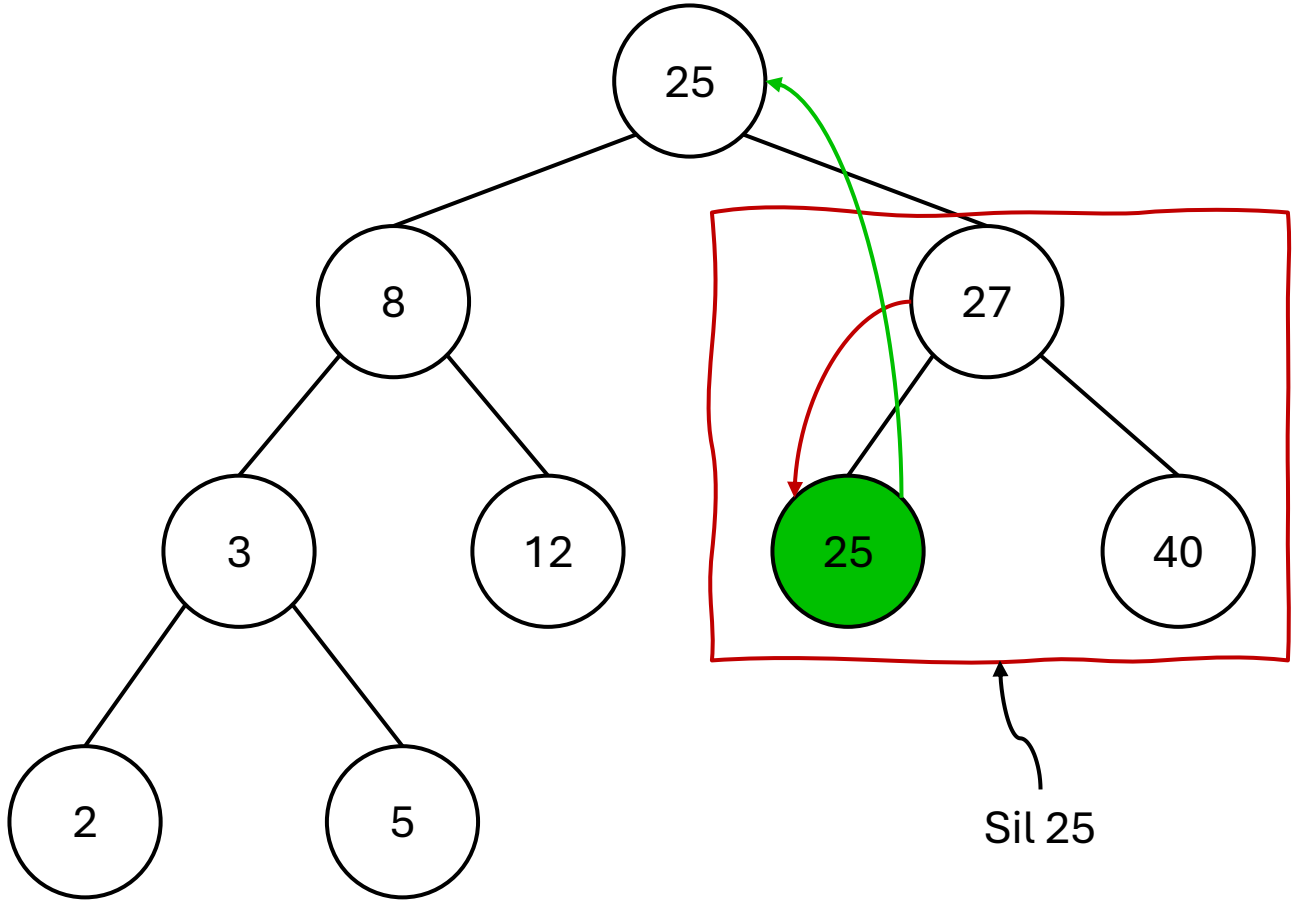
Yaklaşım 1



Binary Search Tree Düğüm Silme

- Sil: 15
- İki çocuk da var (sol ve sağ)
  - Sağ altağaçtaki en küçük (kendisinden bir büyük) düğüm ile yer değiştir
  - **İlgili düğümü sil**

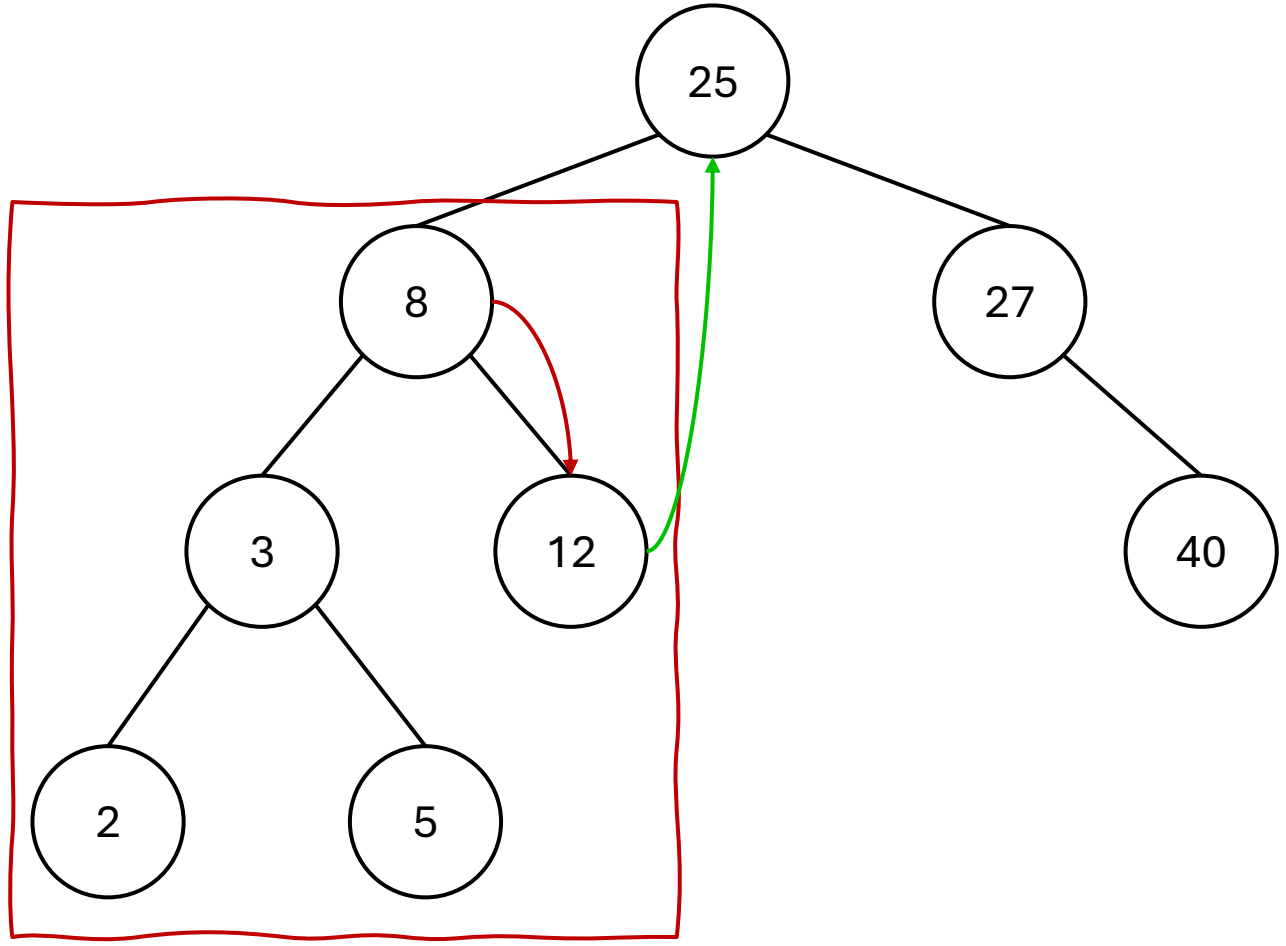
Yaklaşım 1



Binary Search Tree Düğüm Silme

- Sil: 25
- İki çocuk da var (sol ve sağ)
  - **Sol altağaçtaki en büyük (kendisinden bir küçük) düğüm ile yer değiştir**
  - İlgili düğümü sil

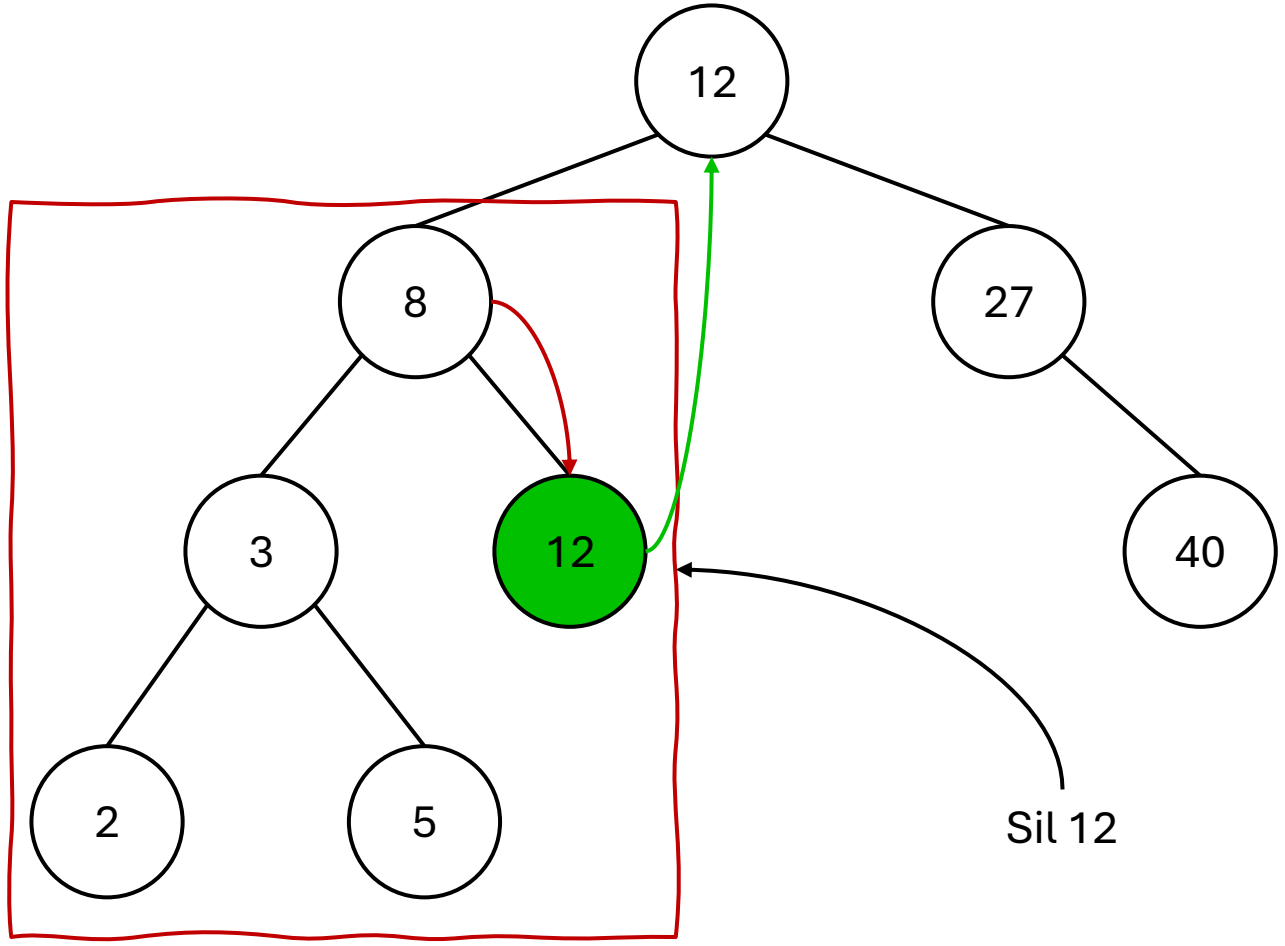
Yaklaşım 2



Binary Search Tree Düğüm Silme

- Sil: 25
- İki çocuk da var (sol ve sağ)
  - Sol altağaçtaki en büyük (kendisinden bir küçük) düğüm ile yer değiştir
  - **İlgili düğümü sil**

Yaklaşım 2



Binary Search Tree Düğüm Ekle/Sil C++

- İlgili fonksiyonları ekleyelim

```
struct BinarySearchTree {
    Node* root = nullptr;
    Node* insert(int value);
    Node* delete(int value);
};
```

Binary Search Tree Düğüm Ekle/Sil Python

- İlgili fonksiyonları ekleyelim

```
class Binary_Search_Tree:
    def __init__(self):
        self.root = None
    def insert(self, value):
    def delete(self, value):
```

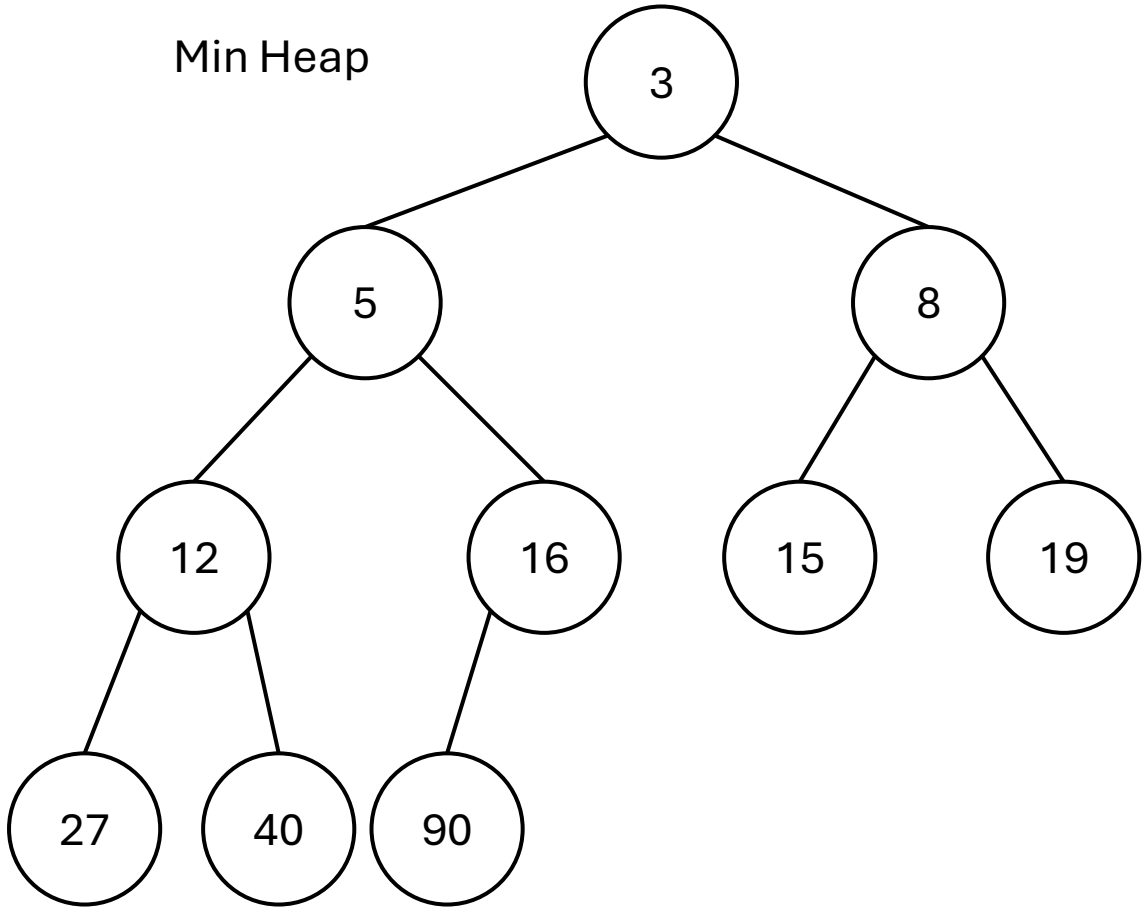
Binary Tree Durumları

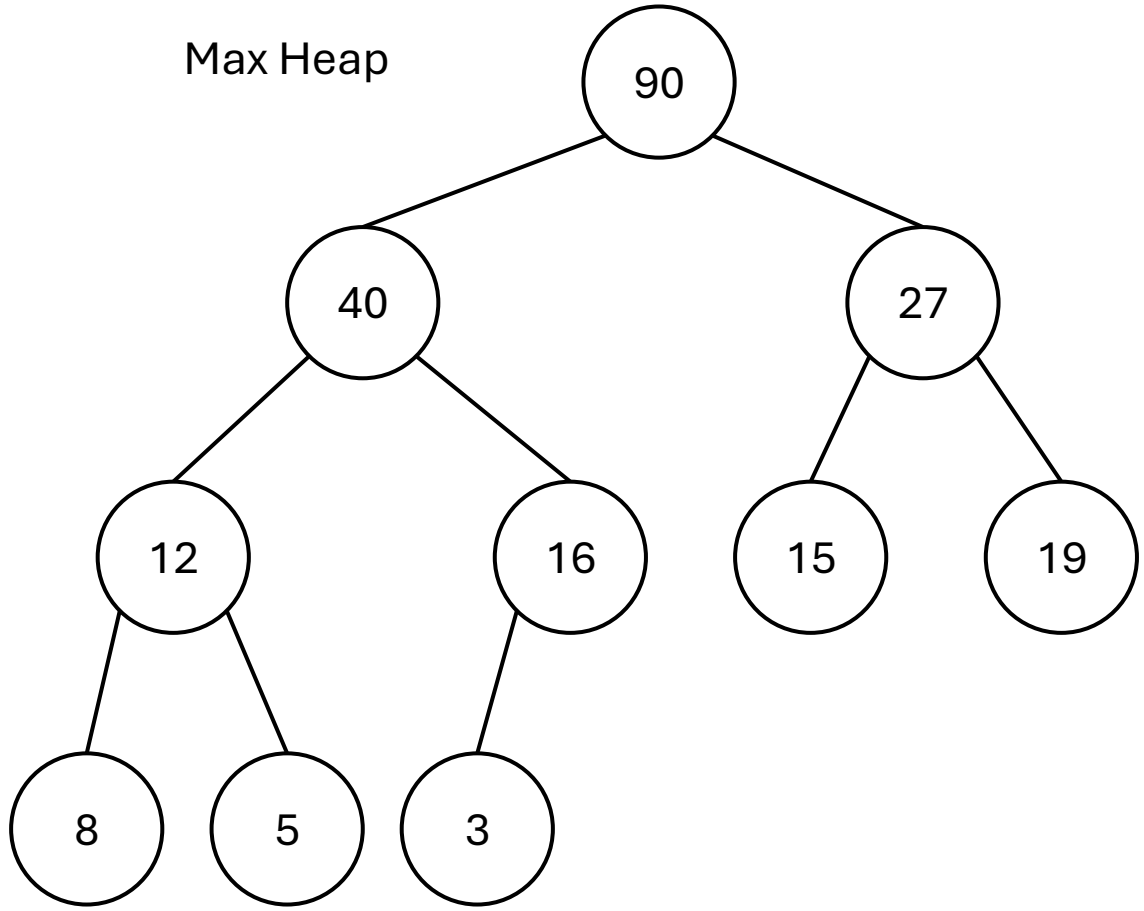
- Full Binary Tree
  - Yaprak olmayan tüm düğümlerin tam olarak 2 çocuğu vardır
    - Tek çocuklu düğüm yok
- Complete Binary Tree
  - En alt seviye dışındaki tüm seviyeler tam olarak doludur.
  - En alt seviyedeki düğümler ise soldan sağa şekilde doludur.
- Perfect Binary Tree
  - Tüm seviyeler tam olarak doludur.
  - Bütün yapraklar aynı seviyedir.
- Balanced Binary Tree
  - Tüm düğümler için sol ve sağ altağaç yükseklikleri farkı en fazla 1'dir.

Heap Tree

- Complete binary tree yapısına sahiptir
- Heap Çeşitleri
  - Min Heap: Her düğümün çocuklarının değeri kendisinden büyük ya da eşittir
  - Max Heap: Her düğümün çocuklarının değeri kendisinden küçük ya da eşittir
- Heap genellikle öncelikli kuyruk (priority queue) uygulamalarında kullanılır
  - En küçük ya da en büyük öncelik her zaman kökte

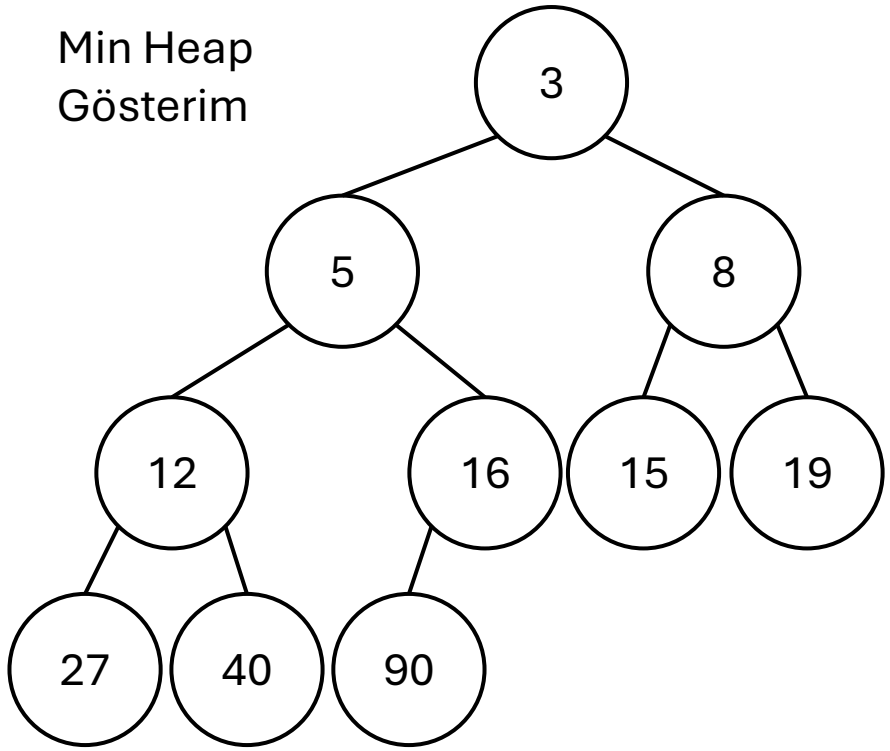
Heap Tree

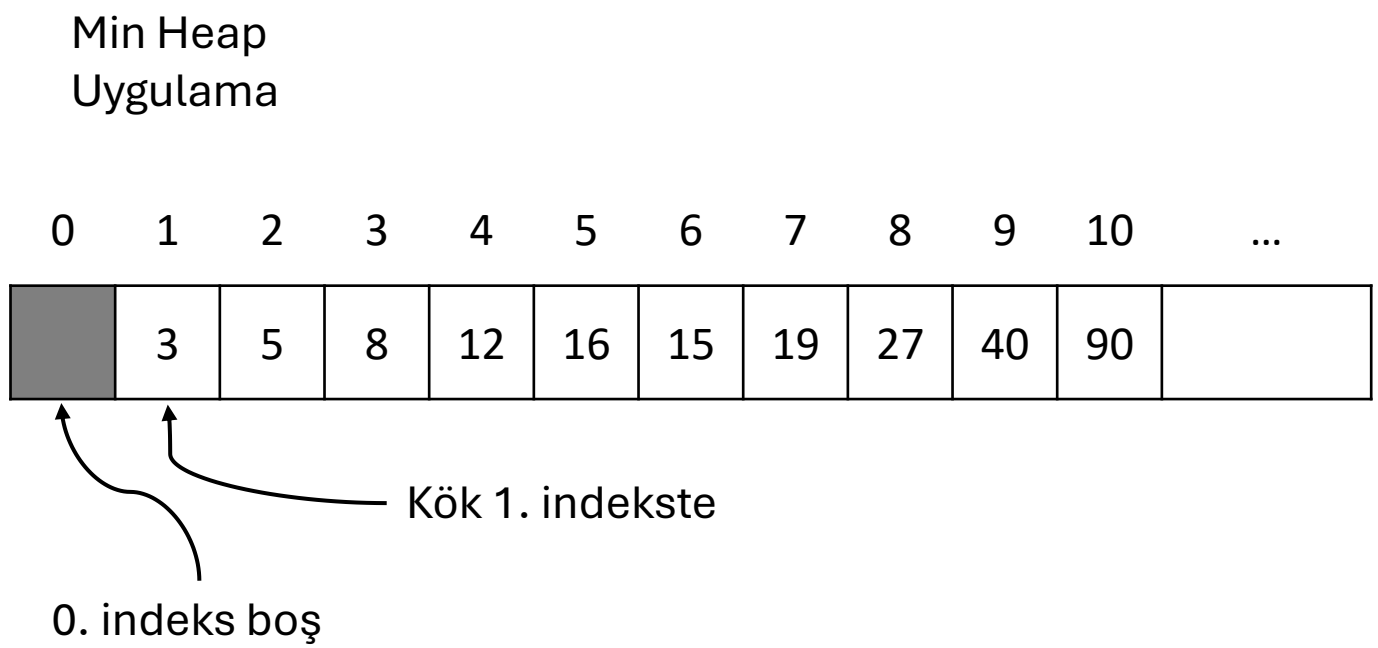




Heap Tree Uygulama

- Binary heap uygulaması genellikle array olarak gerçekleştirilir.



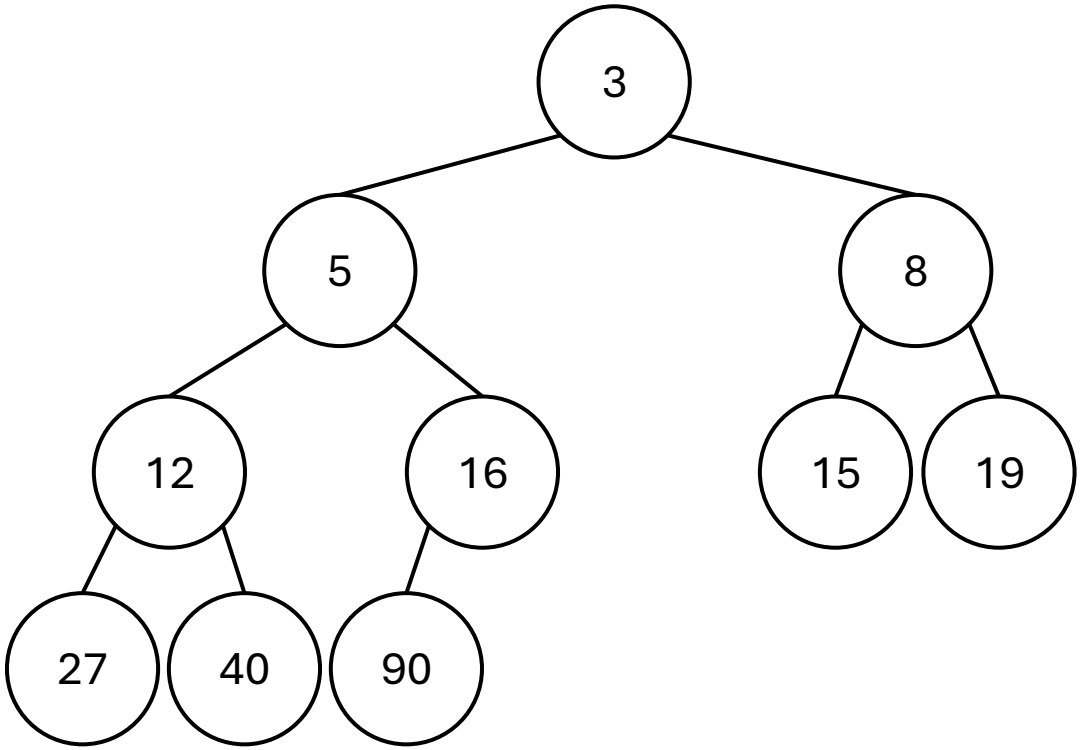


Heap Tree

- Array içerisinde i indeksinde bulunan bir elemanın
  - Ebeveyn (parent) indeksi: $\lfloor i/2 \rfloor$
  - Sol çocuk (left child) indeksi: $2i$
  - Sağ çocuk (right child) indeksi: $2i + 1$
- Heap işlemler
  - heapify
    - Aşağıdan yukarı (bottom-up)
    - Yukarıdan aşağı (top-down)
  - Ekle (insert)
    - En sona ekleme yap
    - Bottom-up heapify gerçekleştir
  - Min/Maks çıkar (extract min/max)
    - Kök değer ile en sondaki değeri yer değiştir
    - En son düğümü ağaçtan çıkar
    - Top-down heapify gerçekleştir

Heap Tree

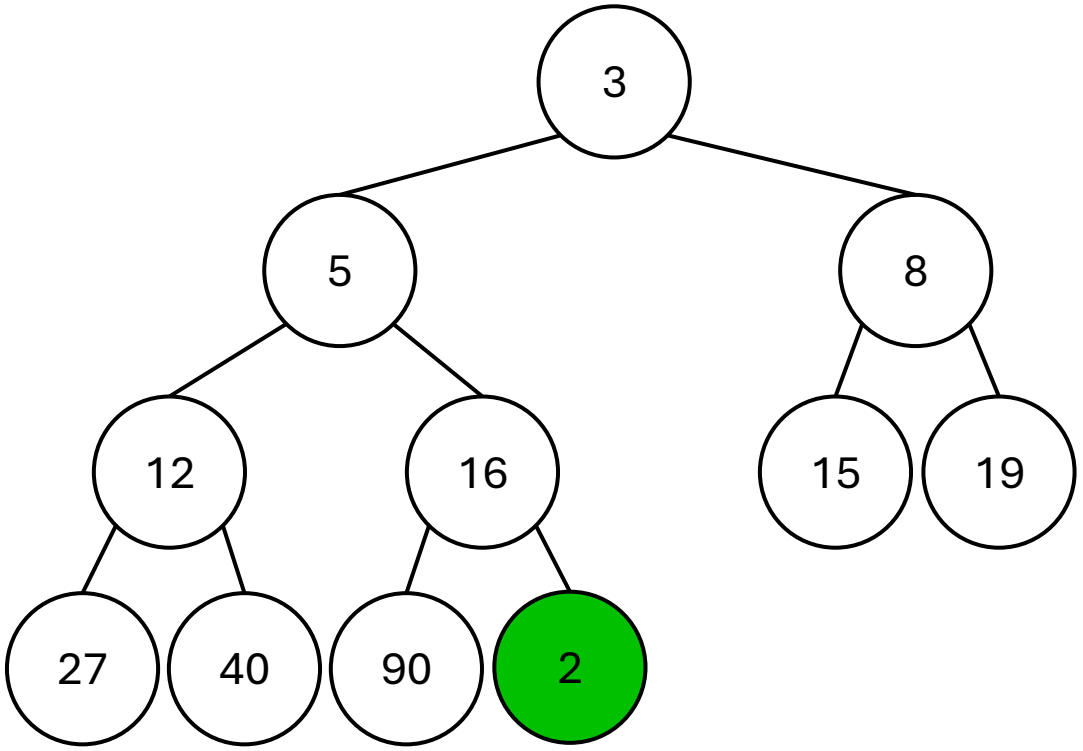
- Ekle: 2
  - En sona ekleme yap
  - Bottom-up heapify gerçekleştir
    - Parent daha büyükse yer değiştir
      - Aynı kontrolü yeni konum için yap



0	1	2	3	4	5	6	7	8	9	10	11	12	...
	3	5	8	12	16	15	19	27	40	90			

Heap Tree

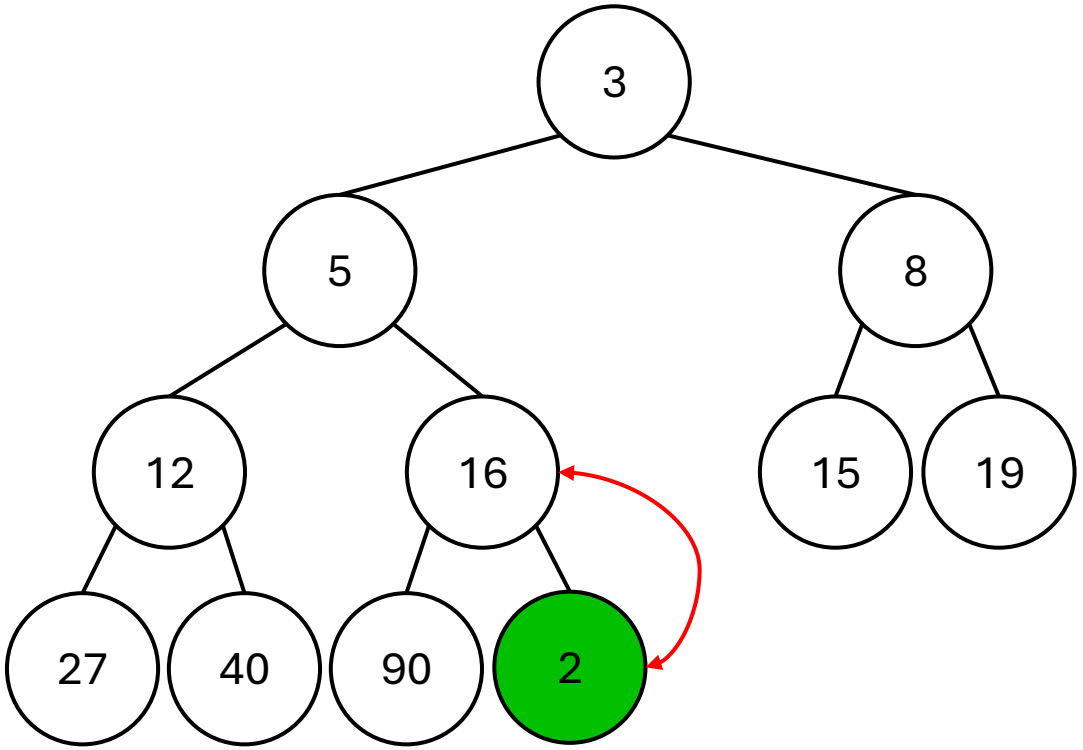
- Ekle: 2
  - **En sona ekleme yap**
  - Bottom-up heapify gerçekleştir
    - Parent daha büyükse yer değiştir
      - Aynı kontrolü yeni konum için yap



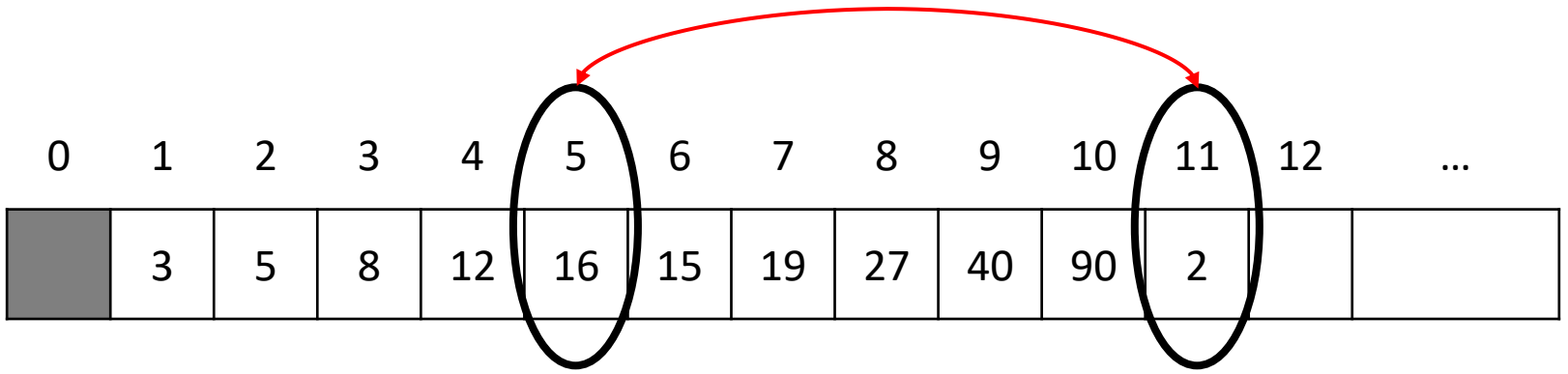
0	1	2	3	4	5	6	7	8	9	10	11	12	...
	3	5	8	12	16	15	19	27	40	90	2		

Heap Tree

- Ekle: 2
  - En sona ekleme yap
  - **Bottom-up heapify gerçekleştir**
    - **Parent daha büyükse yer değiştir**
      - Aynı kontrolü yeni konum için yap

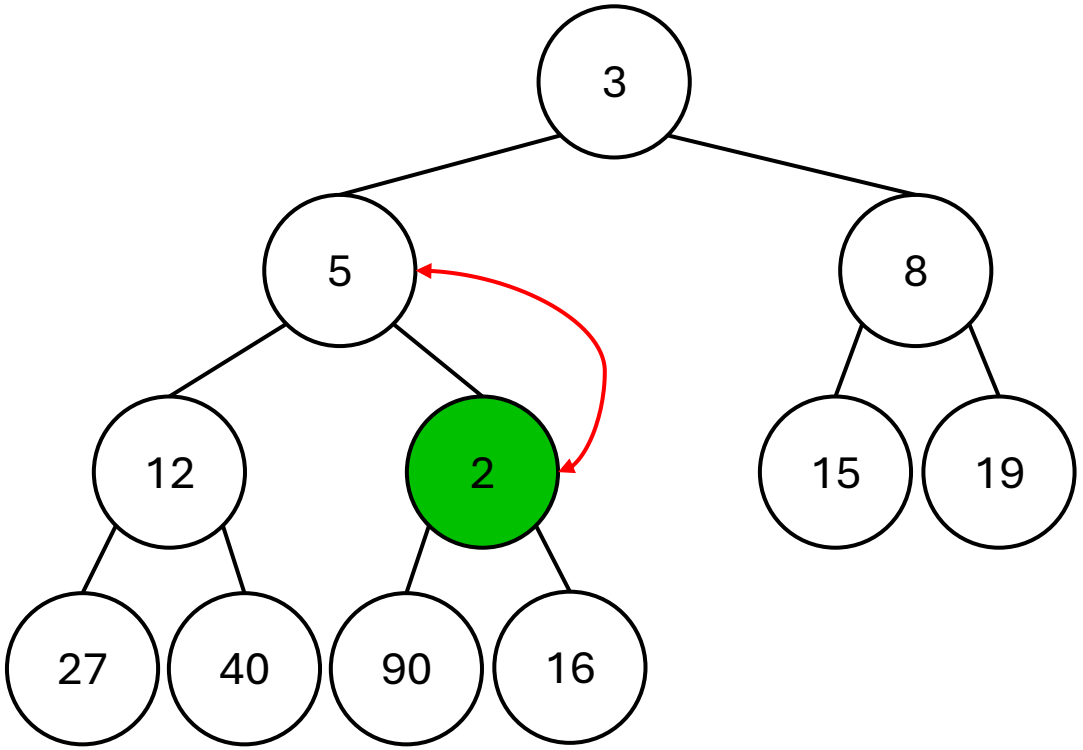


16 > 2

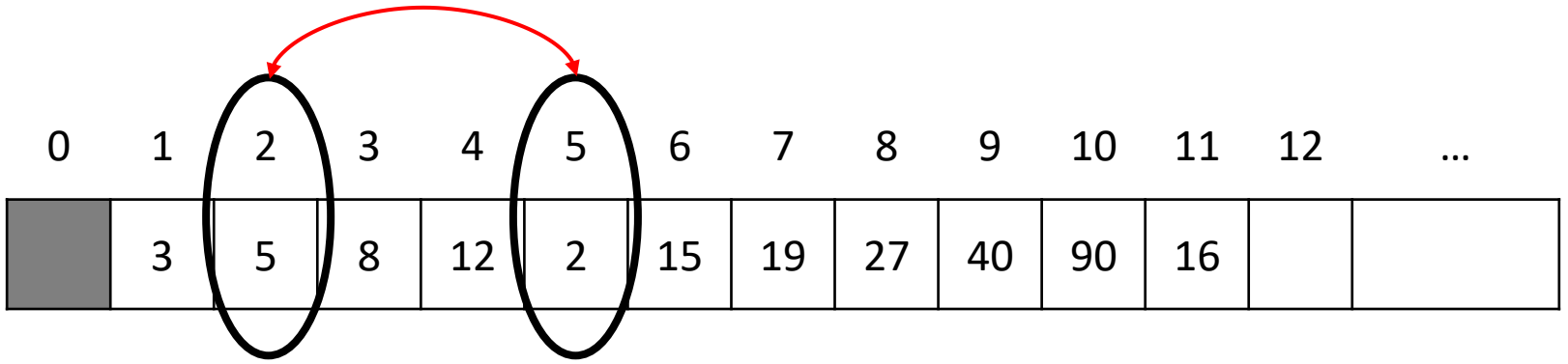


Heap Tree

- Ekle: 2
  - En sona ekleme yap
  - **Bottom-up heapify gerçekleştir**
    - **Parent daha büyükse yer değiştir**
      - **Aynı kontrolü yeni konum için yap**

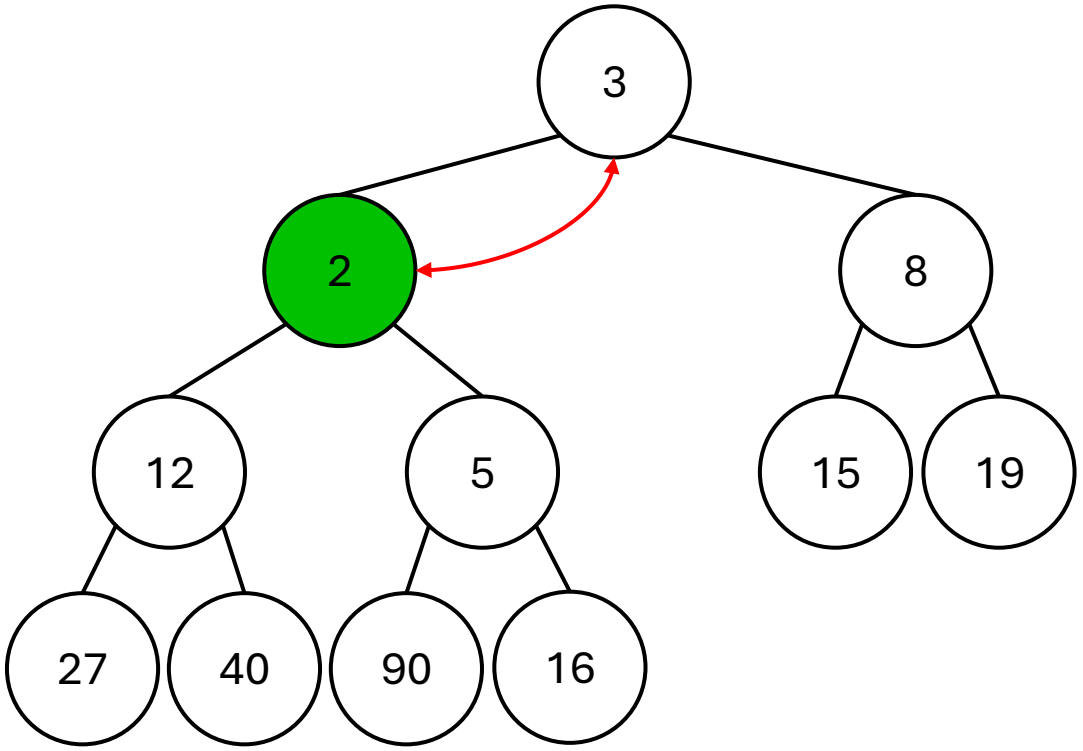


5 > 2

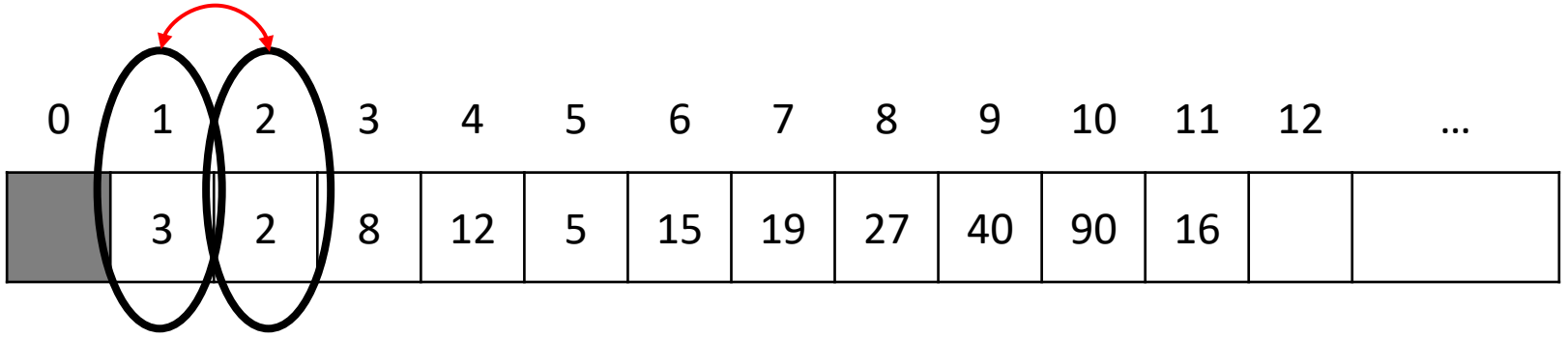


Heap Tree

- Ekle: 2
  - En sona ekleme yap
  - **Bottom-up heapify gerçekleştir**
    - **Parent daha büyükse yer değiştir**
      - **Aynı kontrolü yeni konum için yap**

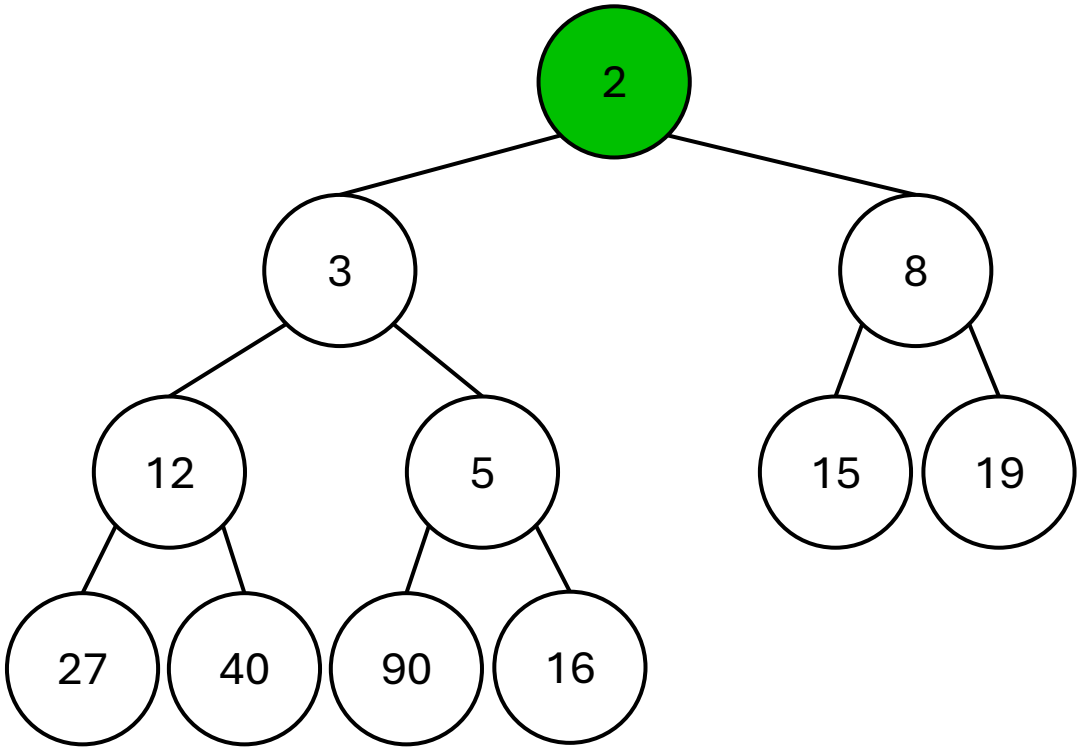


3 > 2



Heap Tree

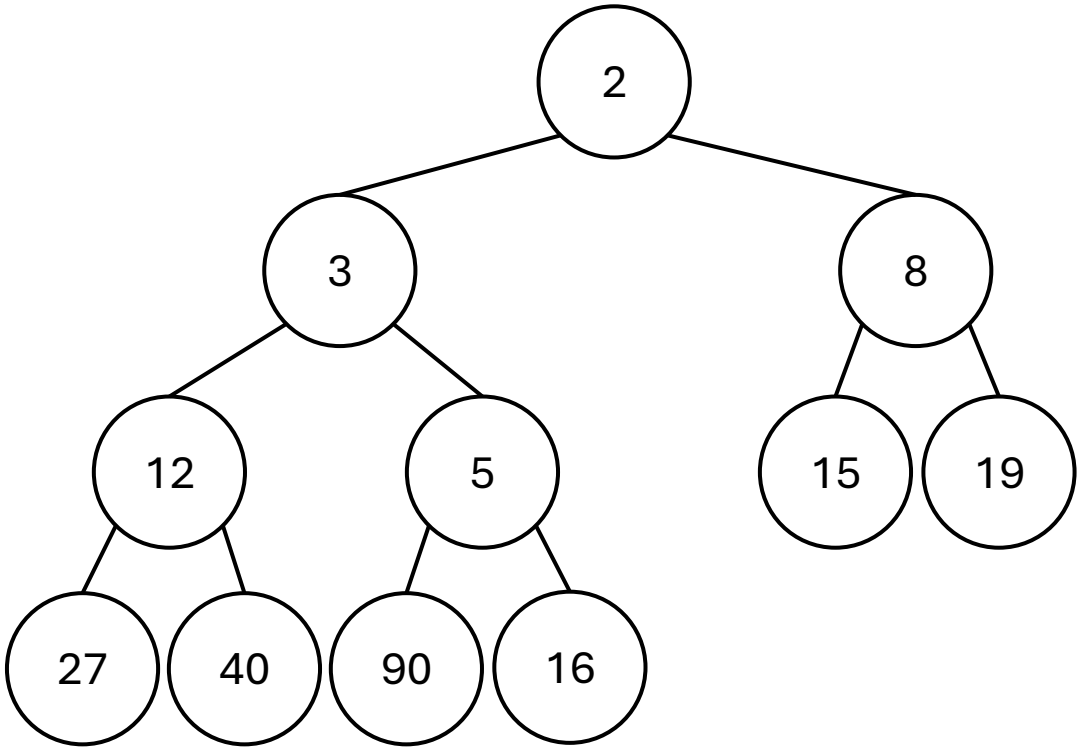
- Ekle: 2
  - En sona ekleme yap
  - Bottom-up heapify gerçekleştir
    - Parent daha büyükse yer değiştir
      - Aynı kontrolü yeni konum için yap



0	1	2	3	4	5	6	7	8	9	10	11	12	...
	2	3	8	12	5	15	19	27	40	90	16		

Heap Tree

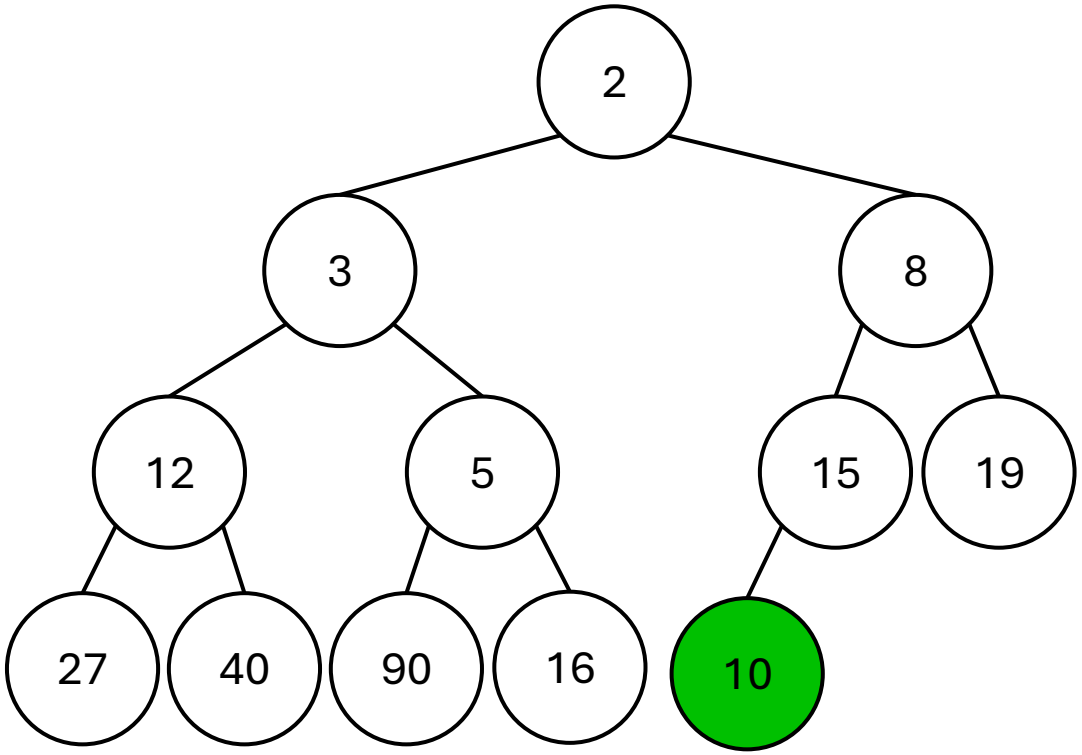
- Ekle: 10
  - En sona ekleme yap
  - Bottom-up heapify gerçekleştir
    - Parent daha büyükse yer değiştir
      - Aynı kontrolü yeni konum için yap



0	1	2	3	4	5	6	7	8	9	10	11	12	...
	2	3	8	12	5	15	19	27	40	90	16		

Heap Tree

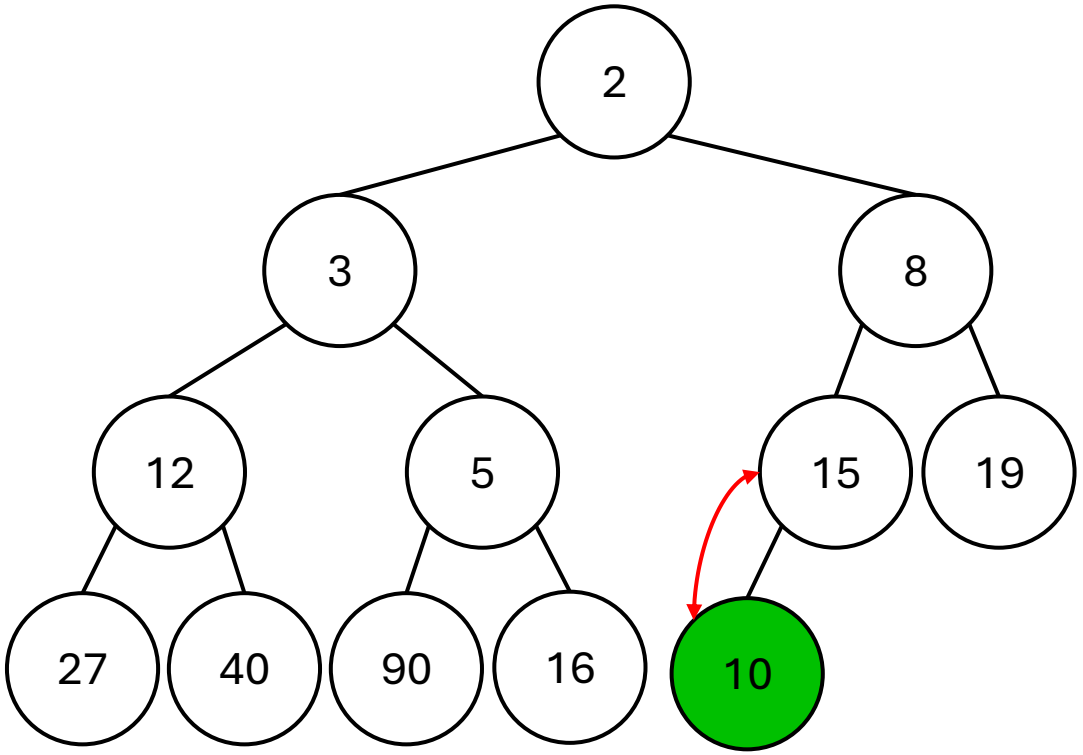
- Ekle: 10
  - **En sona ekleme yap**
  - Bottom-up heapify gerçekleştir
    - Parent daha büyükse yer değiştir
      - Aynı kontrolü yeni konum için yap

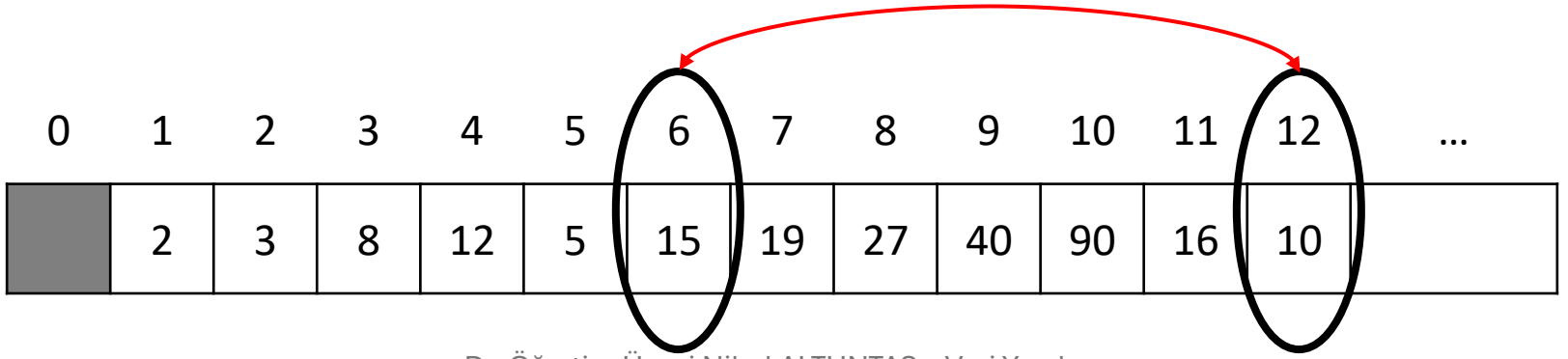


0	1	2	3	4	5	6	7	8	9	10	11	12	...
	2	3	8	12	5	15	19	27	40	90	16	10	

Heap Tree

- Ekle: 10
  - En sona ekleme yap
  - **Bottom-up heapify gerçekleştir**
    - **Parent daha büyükse yer değiştir**
      - Aynı kontrolü yeni konum için yap

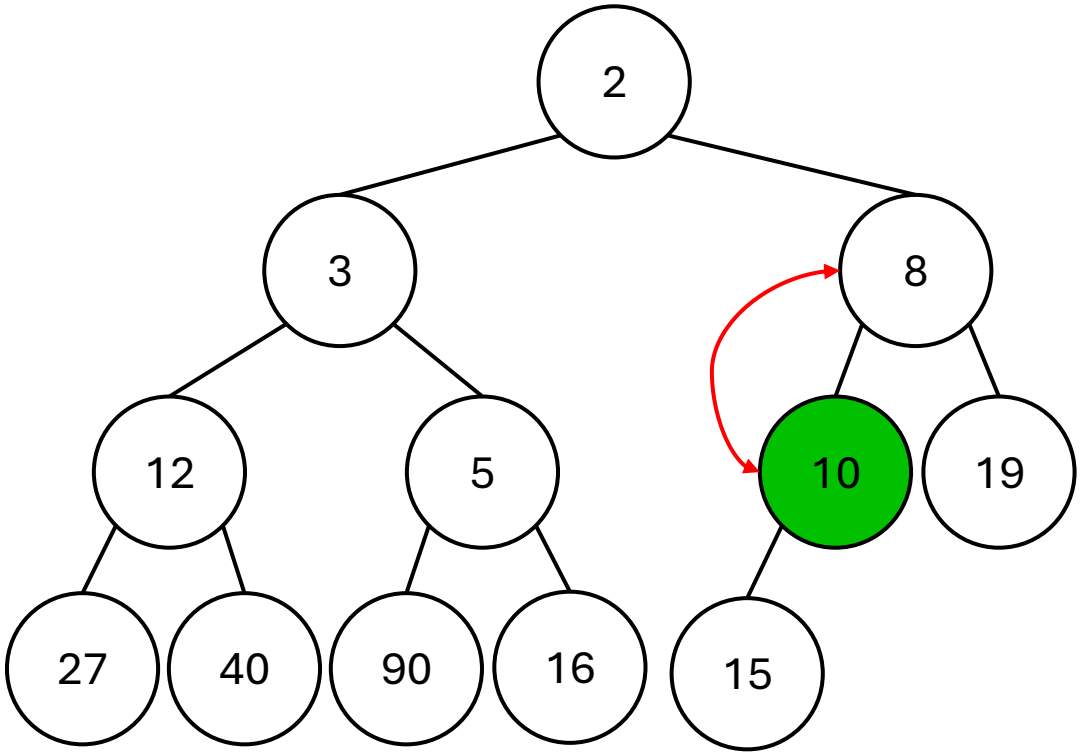




15 > 10

Heap Tree

- Ekle: 10
  - En sona ekleme yap
  - **Bottom-up heapify gerçekleştir**
    - **Parent daha büyükse yer değiştir**
      - Aynı kontrolü yeni konum için yap

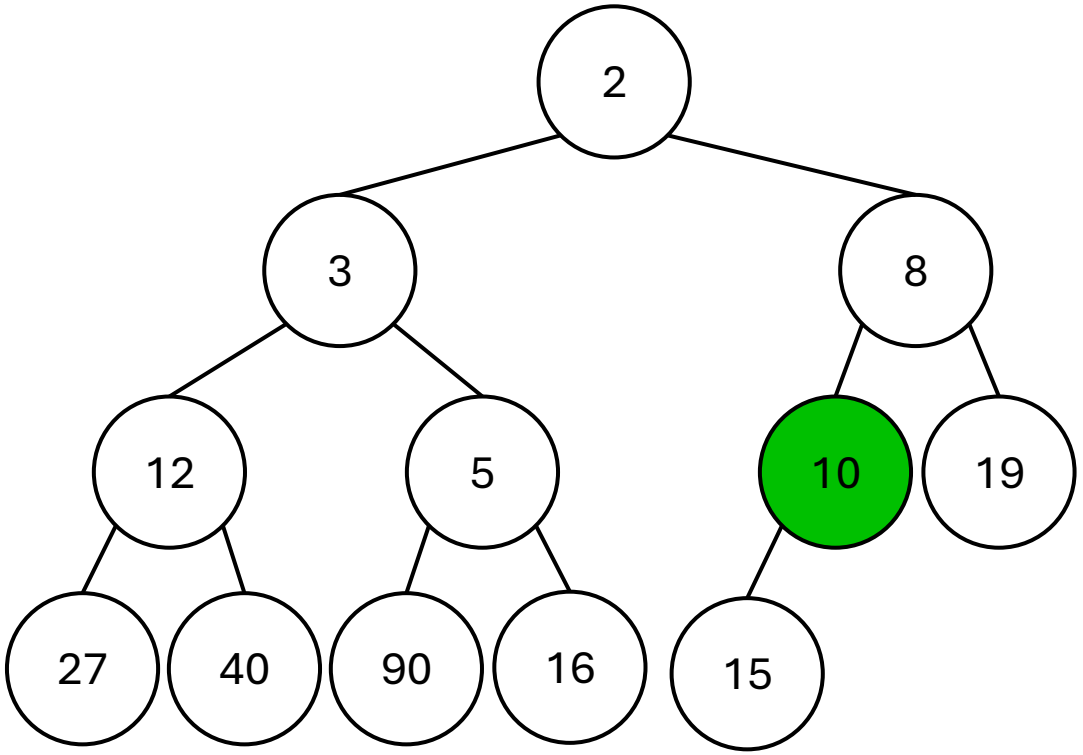




$8 \not> 10$

Heap Tree

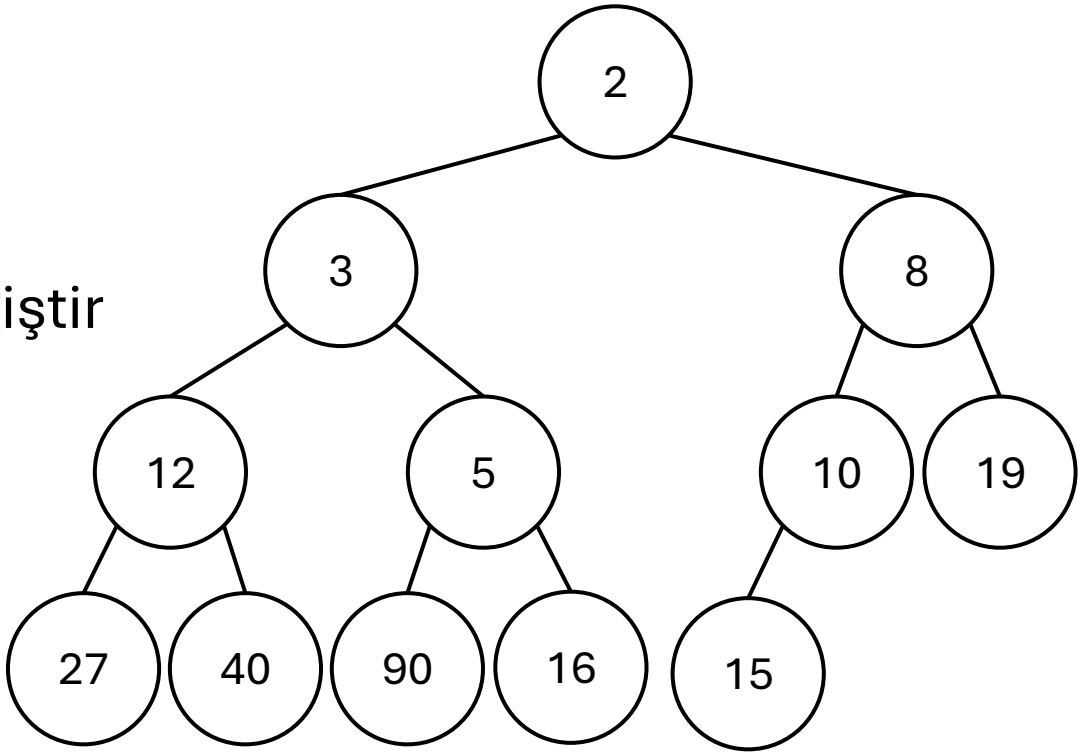
- Ekle: 10
  - En sona ekleme yap
  - Bottom-up heapify gerçekleştir
    - Parent daha büyükse yer değiştir
    - Aynı kontrolü yeni konum için yap



0	1	2	3	4	5	6	7	8	9	10	11	12	...
	2	3	8	12	5	10	19	27	40	90	16	15	

Heap Tree

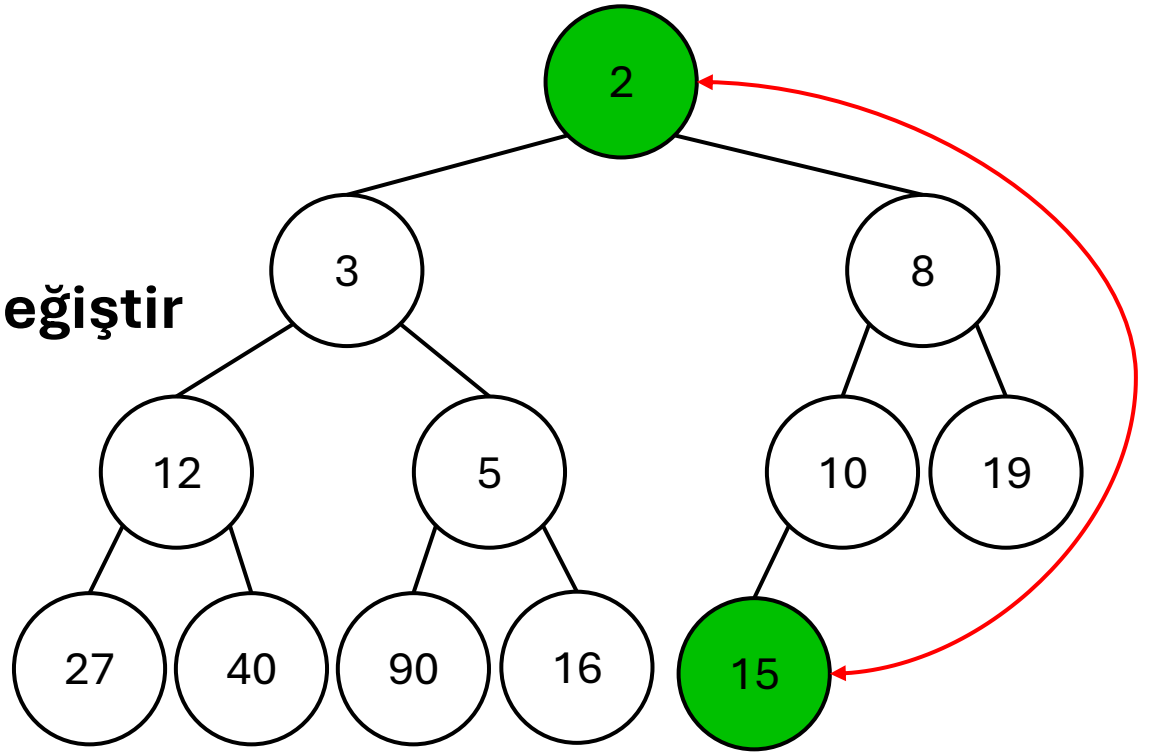
- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - Top-down heapify gerçekleştir
    - Küçük olan çocuğu bul
    - Kendisinden küçükse yer değiştir
      - Aynı kontrolü yeni konum için yap

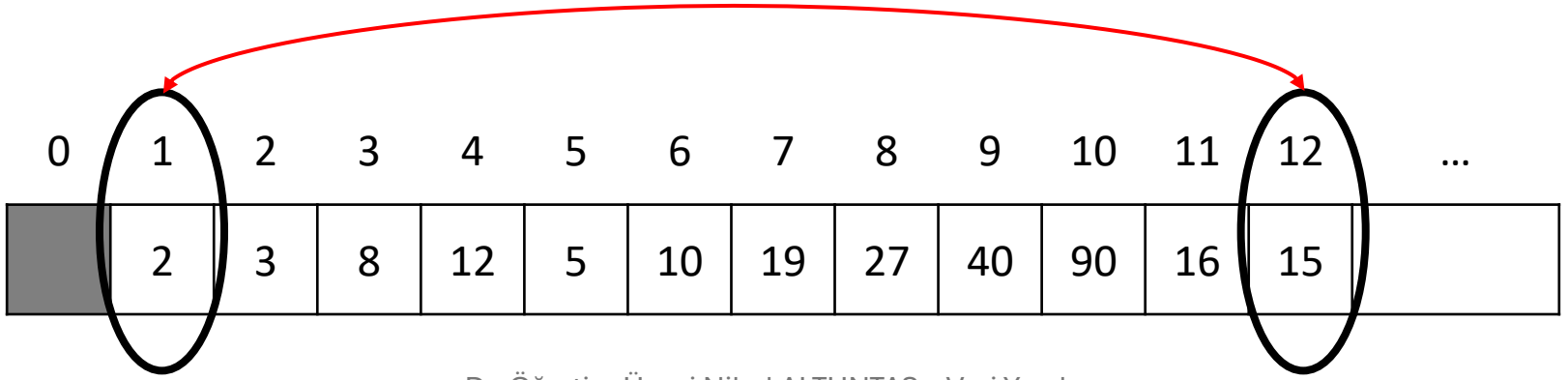


0	1	2	3	4	5	6	7	8	9	10	11	12	...
	2	3	8	12	5	10	19	27	40	90	16	15	

Heap Tree

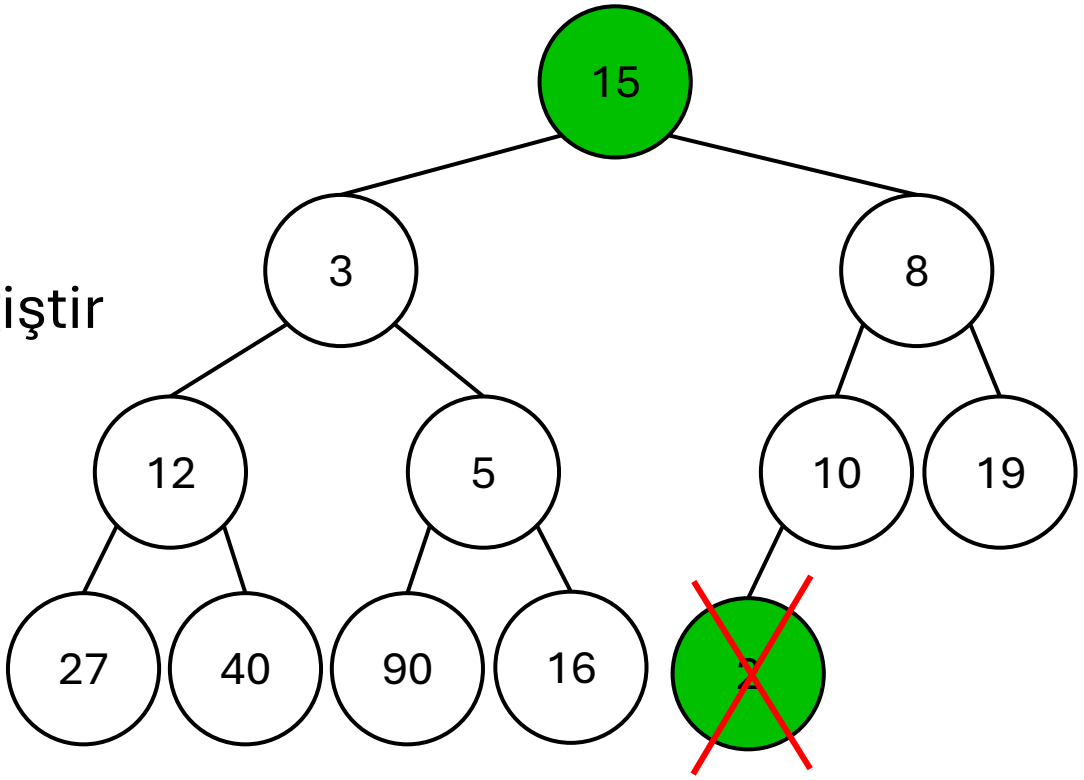
- extractMin
  - **Kök değer ile en sondaki değeri yer değiştir**
  - En son düğümü ağaçtan çıkar
  - Top-down heapify gerçekleştir
    - Küçük olan çocuğu bul
    - Kendisinden küçükse yer değiştir
      - Aynı kontrolü yeni konum için yap

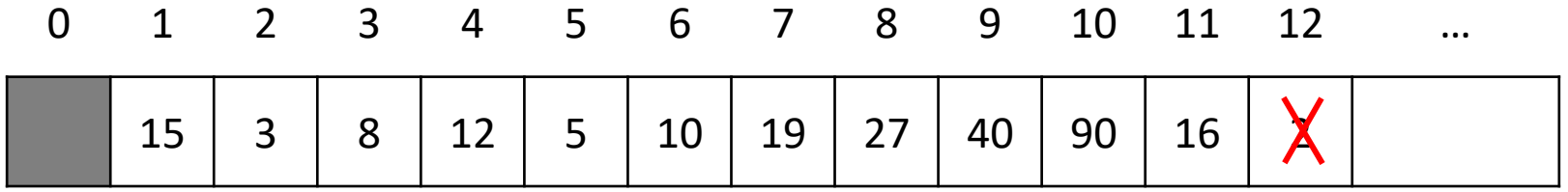




Heap Tree

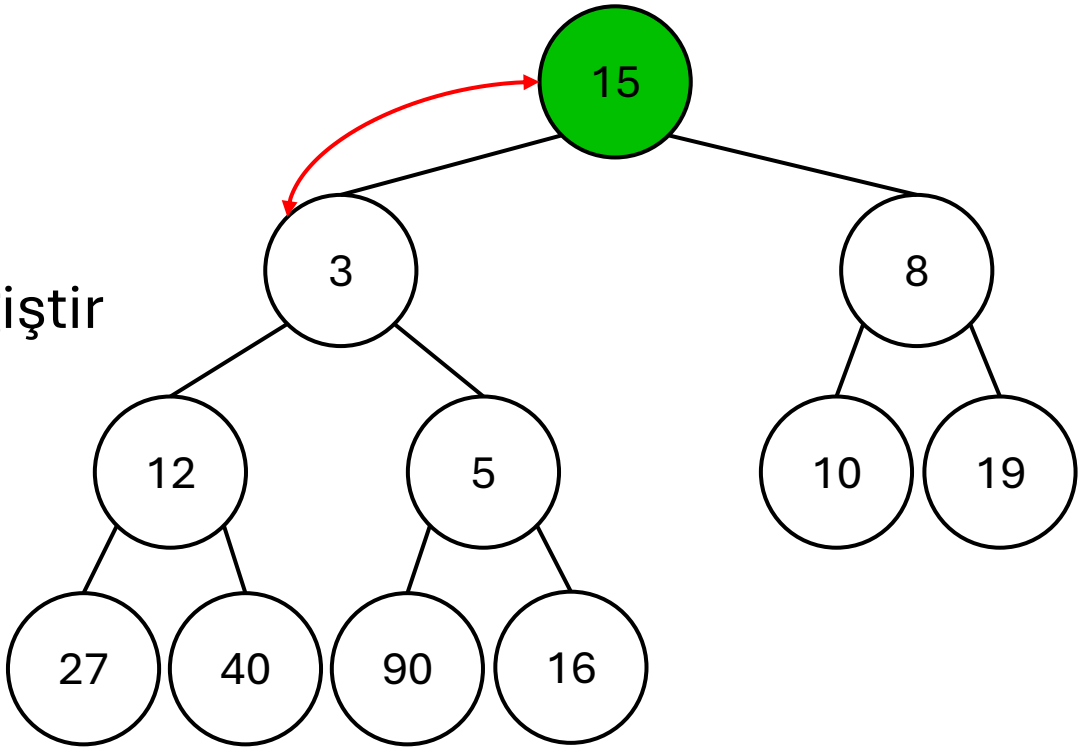
- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - **En son düğümü ağaçtan çıkar**
  - Top-down heapify gerçekleştir
    - Küçük olan çocuğu bul
    - Kendisinden küçükse yer değiştir
      - Aynı kontrolü yeni konum için yap





Heap Tree

- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - **Top-down heapify gerçekleştir**
    - **Küçük olan çocuğu bul**
    - **Kendisinden küçükse yer değiştir**
      - Aynı kontrolü yeni konum için yap

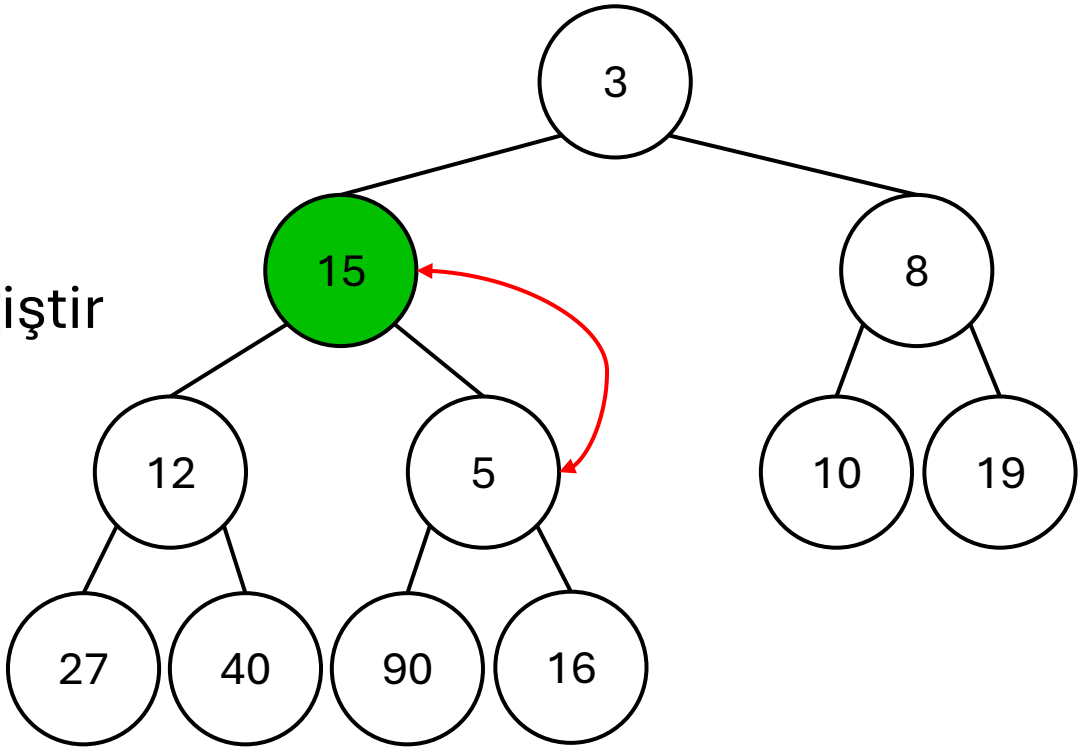


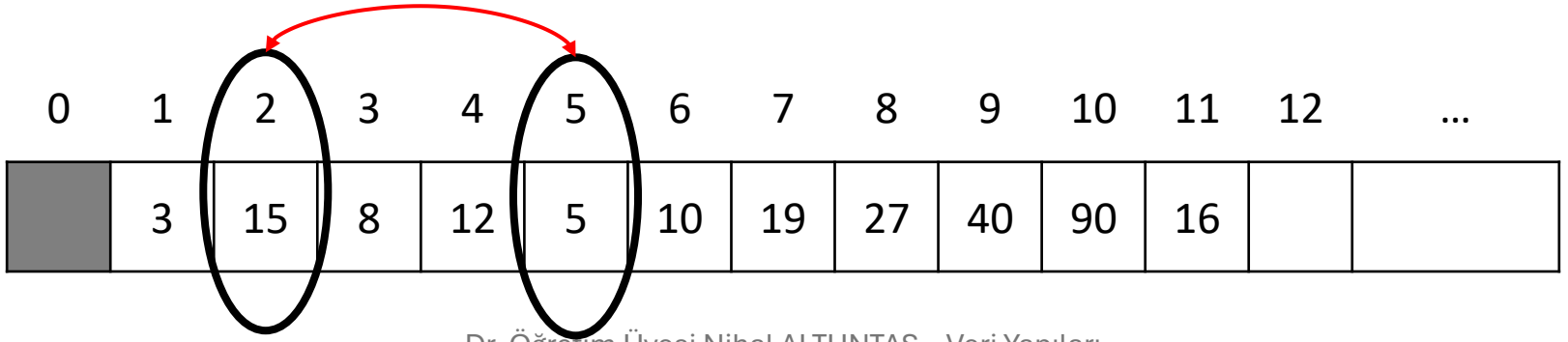


3 < 15

Heap Tree

- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - **Top-down heapify gerçekleştir**
    - **Küçük olan çocuğu bul**
    - **Kendisinden küçükse yer değiştir**
      - **Aynı kontrolü yeni konum için yap**

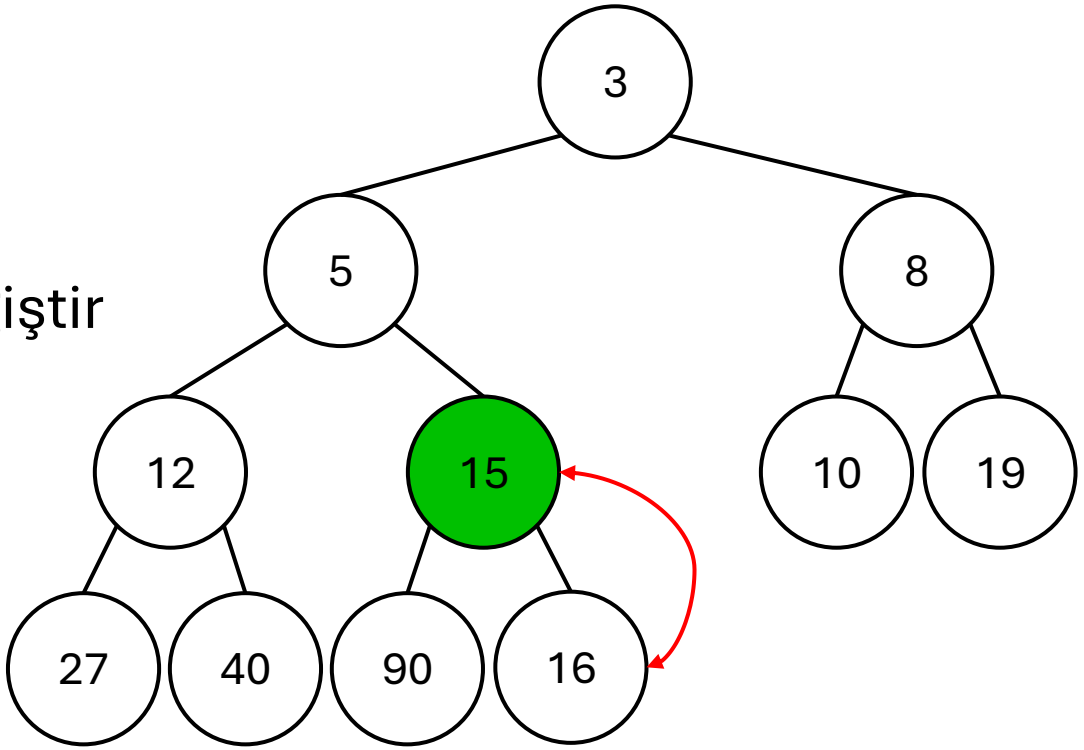




5 < 15

Heap Tree

- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - **Top-down heapify gerçekleştir**
    - **Küçük olan çocuğu bul**
    - **Kendisinden küçükse yer değiştir**
      - **Aynı kontrolü yeni konum için yap**



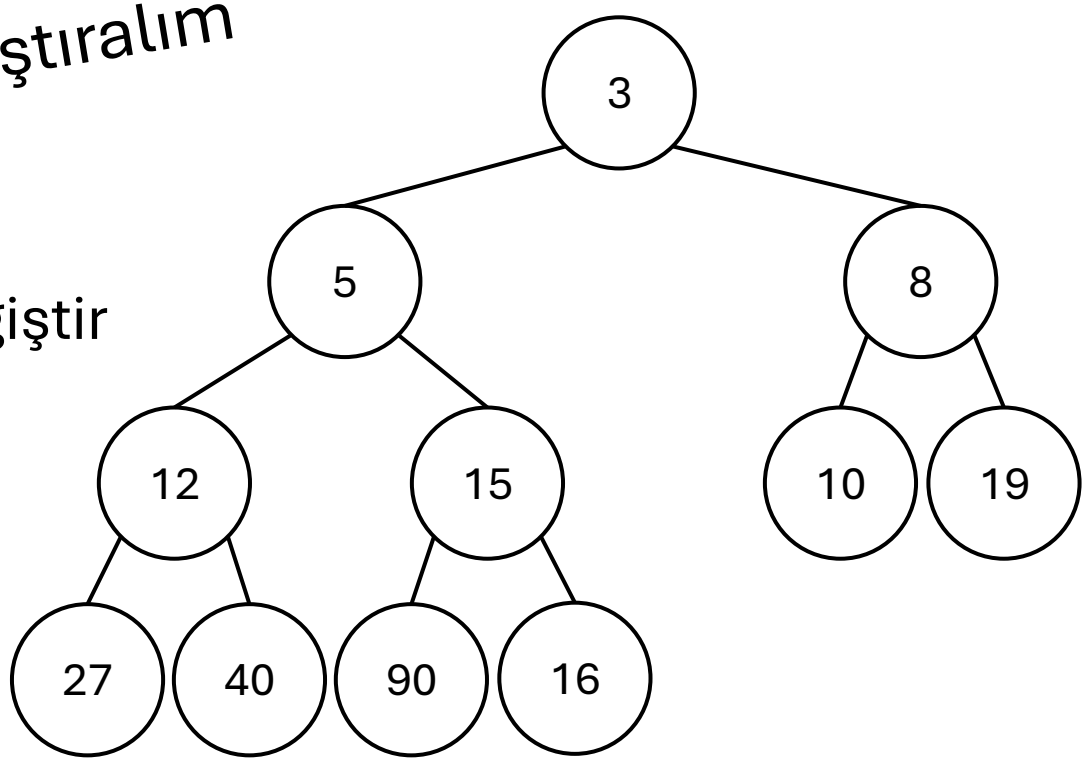


$16 \not< 15$

Heap Tree

Bir kere daha çalıştıralım

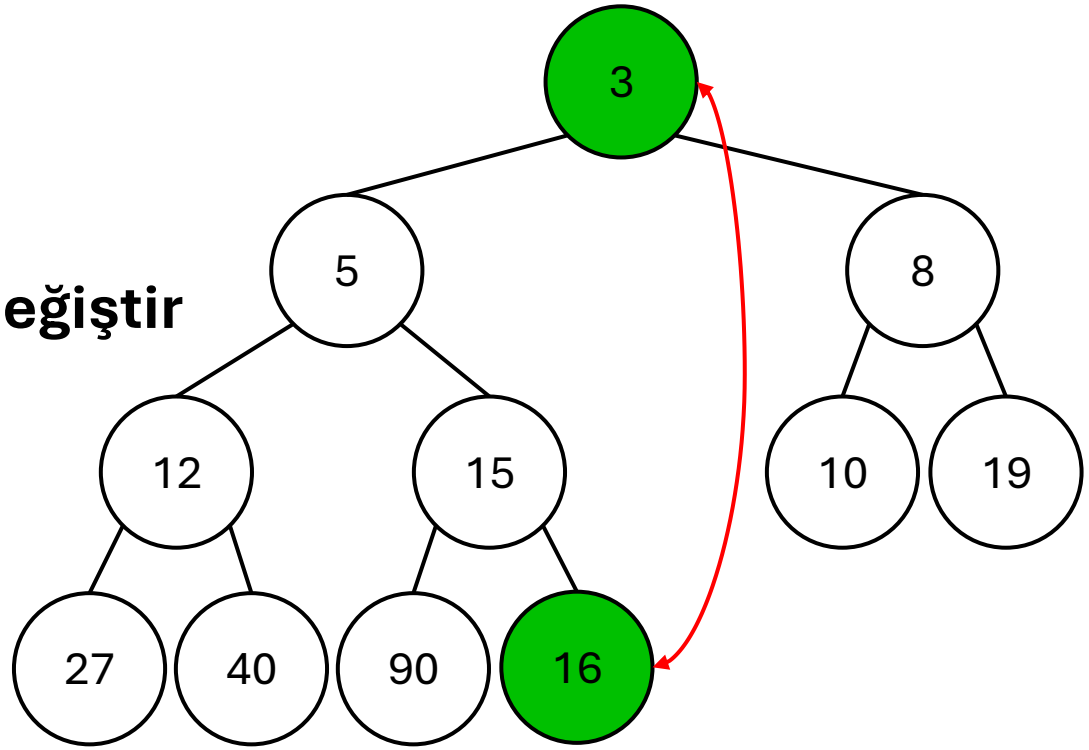
- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - Top-down heapify gerçekleştir
    - Küçük olan çocuğu bul
    - Kendisinden küçükse yer değiştir
      - Aynı kontrolü yeni konum için yap

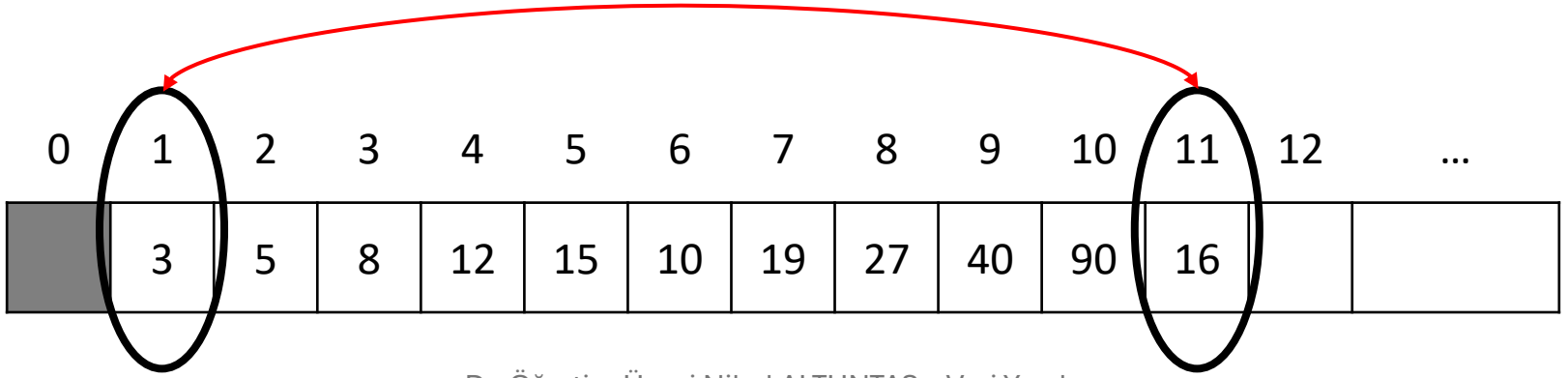


0	1	2	3	4	5	6	7	8	9	10	11	12	…
	3	5	8	12	15	10	19	27	40	90	16		

Heap Tree

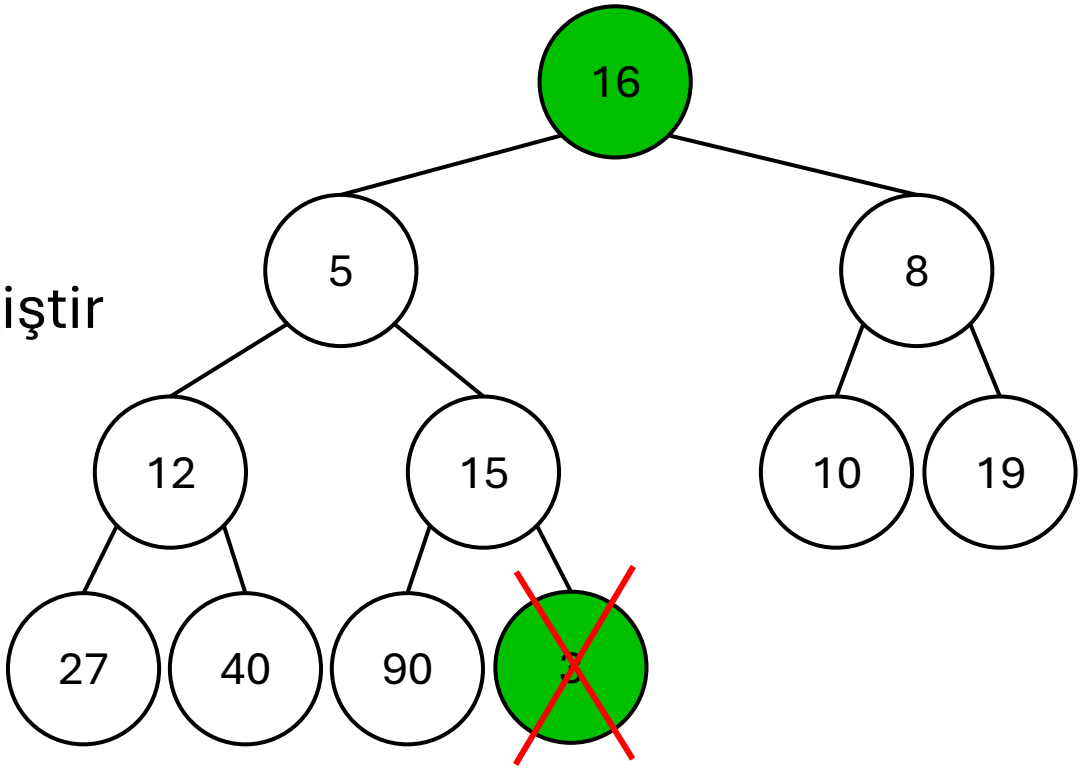
- extractMin
  - **Kök değer ile en sondaki değeri yer değiştir**
  - En son düğümü ağaçtan çıkar
  - Top-down heapify gerçekleştir
    - Küçük olan çocuğu bul
    - Kendisinden küçükse yer değiştir
      - Aynı kontrolü yeni konum için yap





Heap Tree

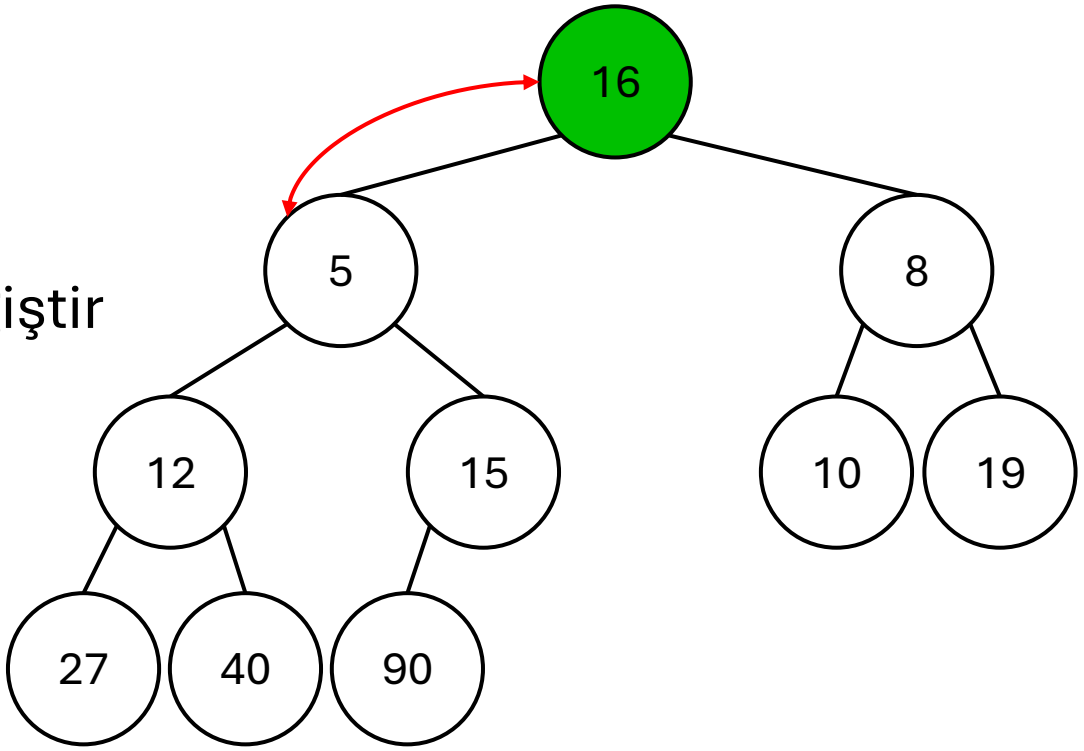
- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - **En son düğümü ağaçtan çıkar**
  - Top-down heapify gerçekleştir
    - Küçük olan çocuğu bul
    - Kendisinden küçükse yer değiştir
      - Aynı kontrolü yeni konum için yap



0	1	2	3	4	5	6	7	8	9	10	11	12	...
	16	5	8	12	15	10	19	27	40	90	~~3~~		

Heap Tree

- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - **Top-down heapify gerçekleştir**
    - **Küçük olan çocuğu bul**
    - **Kendisinden küçükse yer değiştir**
      - Aynı kontrolü yeni konum için yap

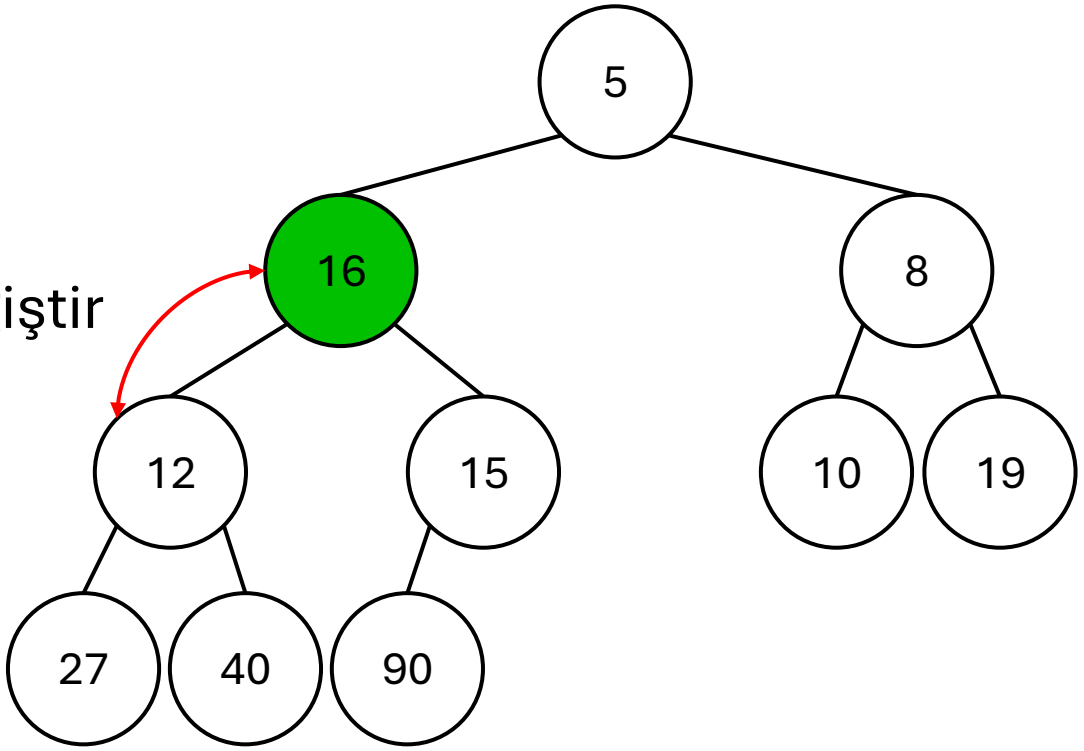




5 < 16

Heap Tree

- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - **Top-down heapify gerçekleştir**
    - **Küçük olan çocuğu bul**
    - **Kendisinden küçükse yer değiştir**
      - **Aynı kontrolü yeni konum için yap**

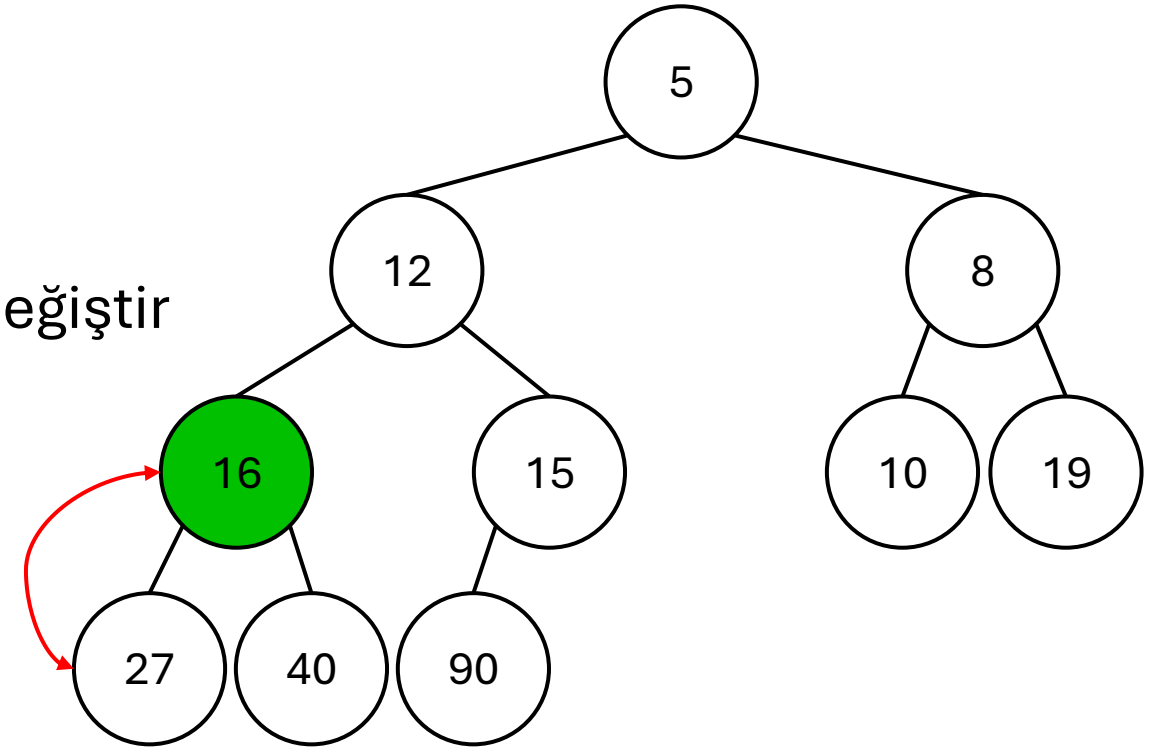


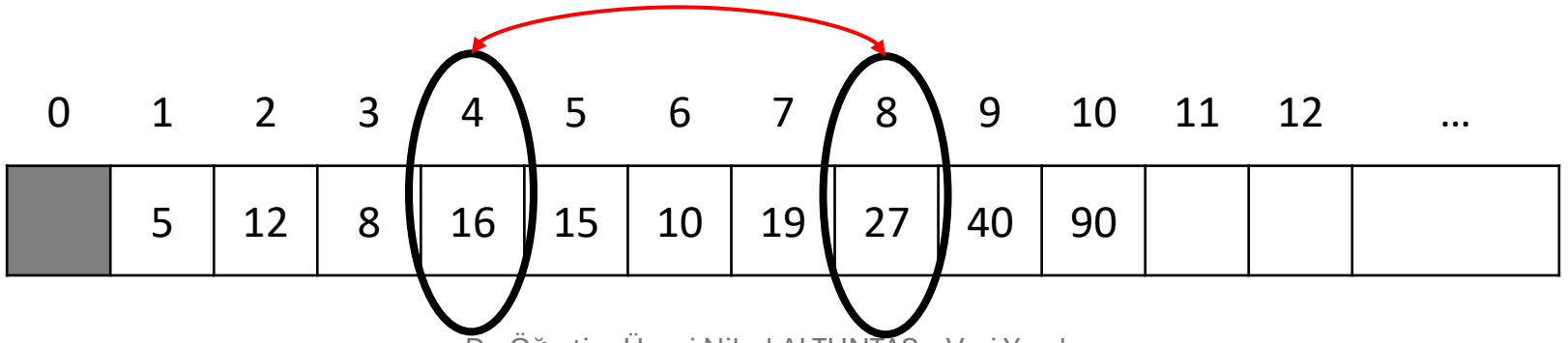


12 < 16

Heap Tree

- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - **Top-down heapify gerçekleştir**
    - **Küçük olan çocuğu bul**
    - **Kendisinden küçükse yer değiştir**
      - **Aynı kontrolü yeni konum için yap**

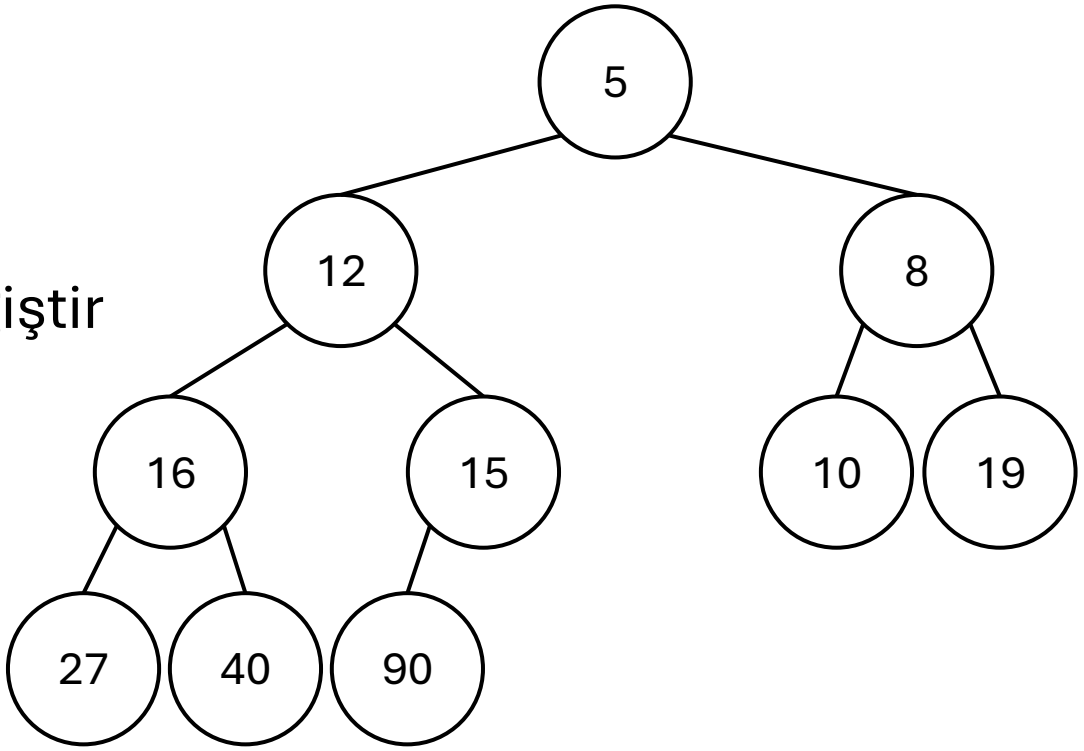




$27 \not< 16$

Heap Tree

- extractMin
  - Kök değer ile en sondaki değeri yer değiştir
  - En son düğümü ağaçtan çıkar
  - Top-down heapify gerçekleştir
    - Küçük olan çocuğu bul
    - Kendisinden küçükse yer değiştir
      - Aynı kontrolü yeni konum için yap



0	1	2	3	4	5	6	7	8	9	10	11	12	...
	5	12	8	16	15	10	19	27	40	90			

Heap Tree C++

```
struct MinHeap {
    int *heap_arr;
    int capacity;
    int heap_size;

    MinHeap(int c) {
        capacity = c;
        heap_arr = new int[capacity+1];
        heap_size = 0;
    }
    void heapifyUp(int ind);
    void heapifyDown(int ind);
    int extractMin();
    void insert(int value);
};
```

```
int parent_ind(int ind) {
    return i / 2;
}

int left_ind(int ind) {
    return 2 * ind;
}

int right_ind(int ind) {
    return 2 * ind + 1;
}
```

Heap Tree Python

```
class MinHeap:
    def __init__(self):
        self.heap_arr = [None]
        self.heap_size = 0

    def heapify_up(self, ind):
    def heapify_down(self, ind):
    def insert(self, value):
    def extract_min(self):
```

```
    def parent_ind(self, ind):
        return ind // 2

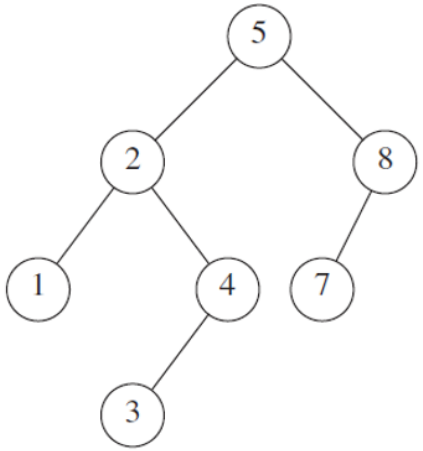
    def left_ind(self, ind):
        return 2 * ind

    def right_ind(self, ind):
        return 2 * ind + 1
```

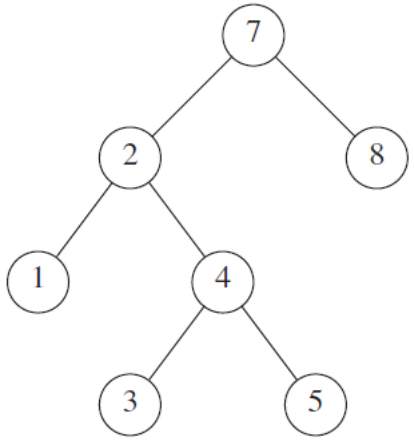
AVL (Adelson-Velskii-Landis) Tree

- Her zaman denge durumunu koruyan BST ağacıdır
  - BST (Binary Search Tree)
    - Herhangi bir düğüm için, sol altağacında bulunan tüm düğümlerin değerleri o düğümün değerinden küçüktür.
    - Herhangi bir düğüm için, sağ altağacında bulunan tüm düğümlerin değerleri o düğümün değerinden büyüktür.
  - Balanced Binary Tree
    - Tüm düğümler için sol ve sağ altağaç yükseklikleri farkı en fazla 1'dir.
- Ekleme ve çıkarma işlemleri sonrası tüm düğümler için bir denge faktörü kontrolü gerçekleştirilir
  - DengeFaktörü = solAltağaçYüksekliği - sağAltağaçYüksekliği
  - Dengeli bir ağacın tüm düğümleri için: -1 ≤ DengeFaktörü ≤ 1

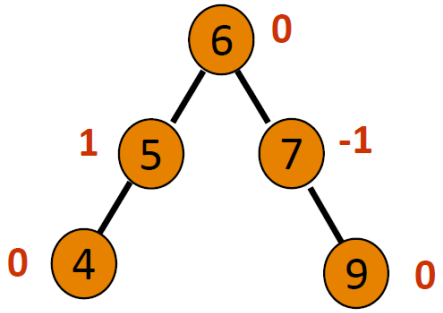
AVL Tree



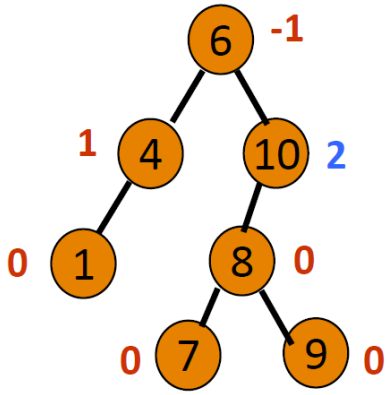
AVL



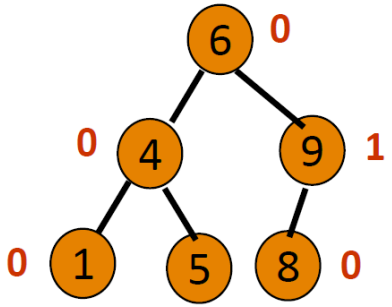
AVL Değil



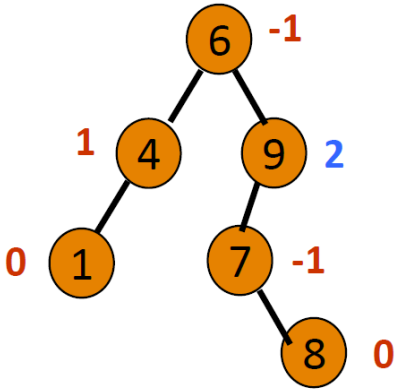
AVL Ağacı



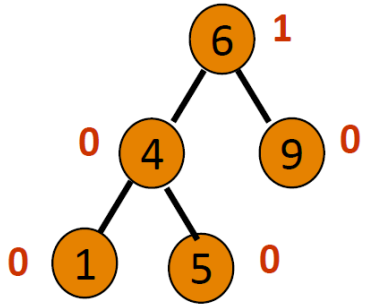
AVL Ağacı Değildir



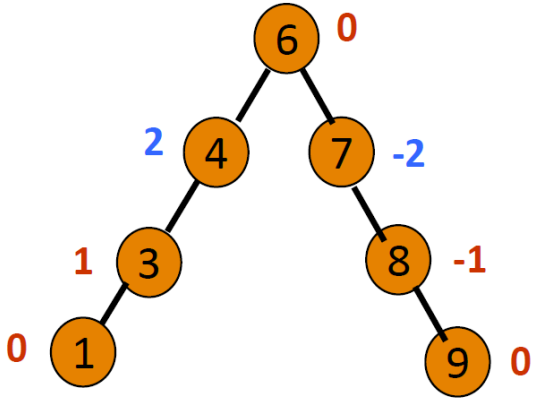
AVL Ağacı



AVL Ağacı Değildir



AVL Ağacı



AVL Ağacı Değildir

AVL Tree Yeni Düğüm Ekleme

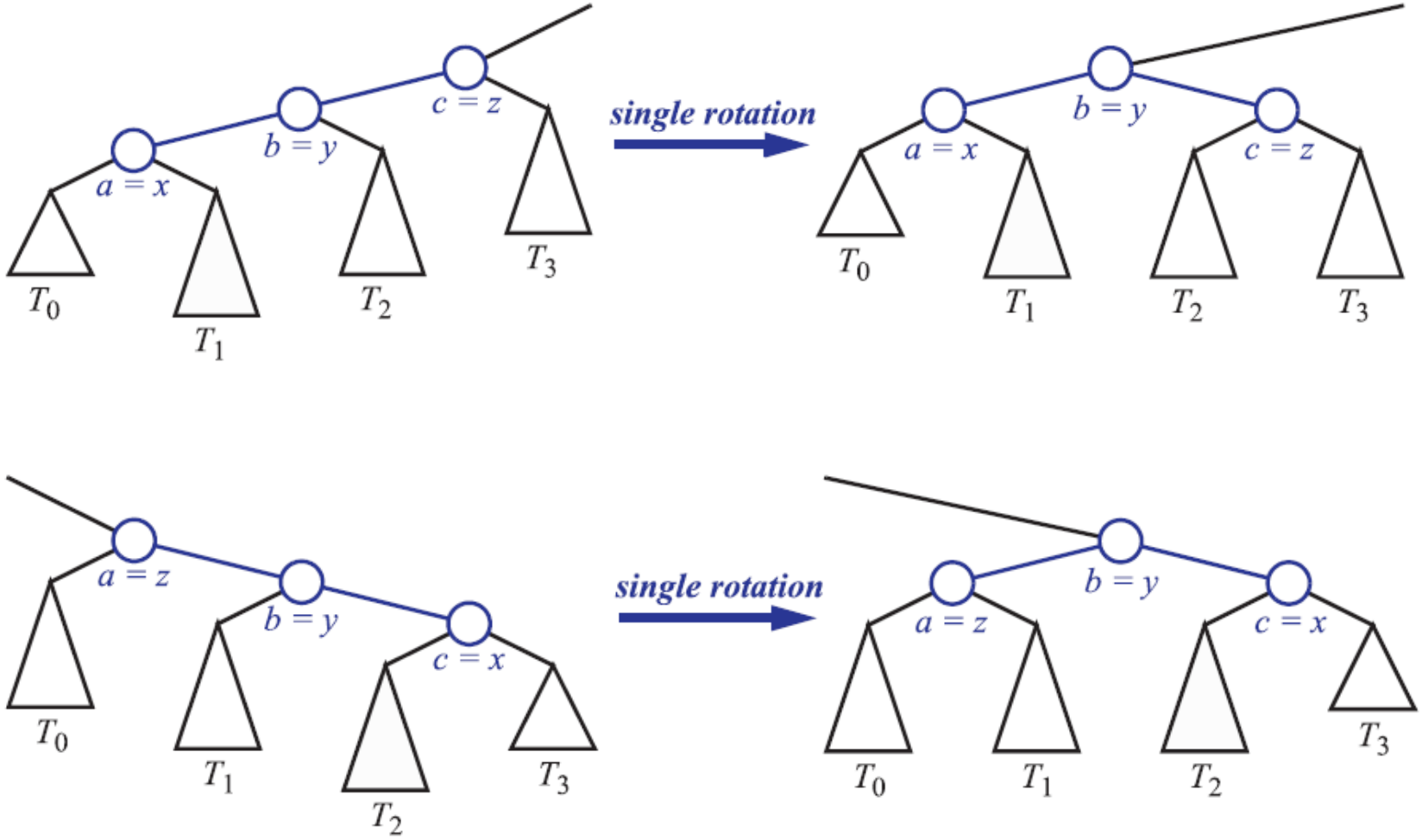
- AVL ağaçlarına ekleme yapılırken hem denge hem de BST özelliği kaybolmamalıdır.
- Ekleme işleminde bazı düğümlerin yükseklikleri değişebileceğinden ağacı tekrar denge durumuna ayarlamak gerekir.
  - Düğümü ekle (BST özelliğine göre ilgili yeri bul)
  - Köke doğru tüm üst düğümlerin yüksekliğini güncelleyerek denge faktörlerini kontrol et
    - [-1, 1] aralığı dışına çıkan düğümler için döndürme gerçekleştir

AVL Tree

- Dengenin bozulması durumu
  - Tek döndürme gerektiren:
    - LL Dengesizliği: Düğümün sol altağacının soluna eklenmesi
    - RR Dengesizliği: Düğümün sağ altağacının sağına eklenmesi
  - Çift döndürme işlemi gerektiren:
    - LR Dengesizliği: Düğümün sol altağacının sağına eklenmesi
    - RL Dengesizliği: Düğümün sağ altağacının soluna eklenmesi

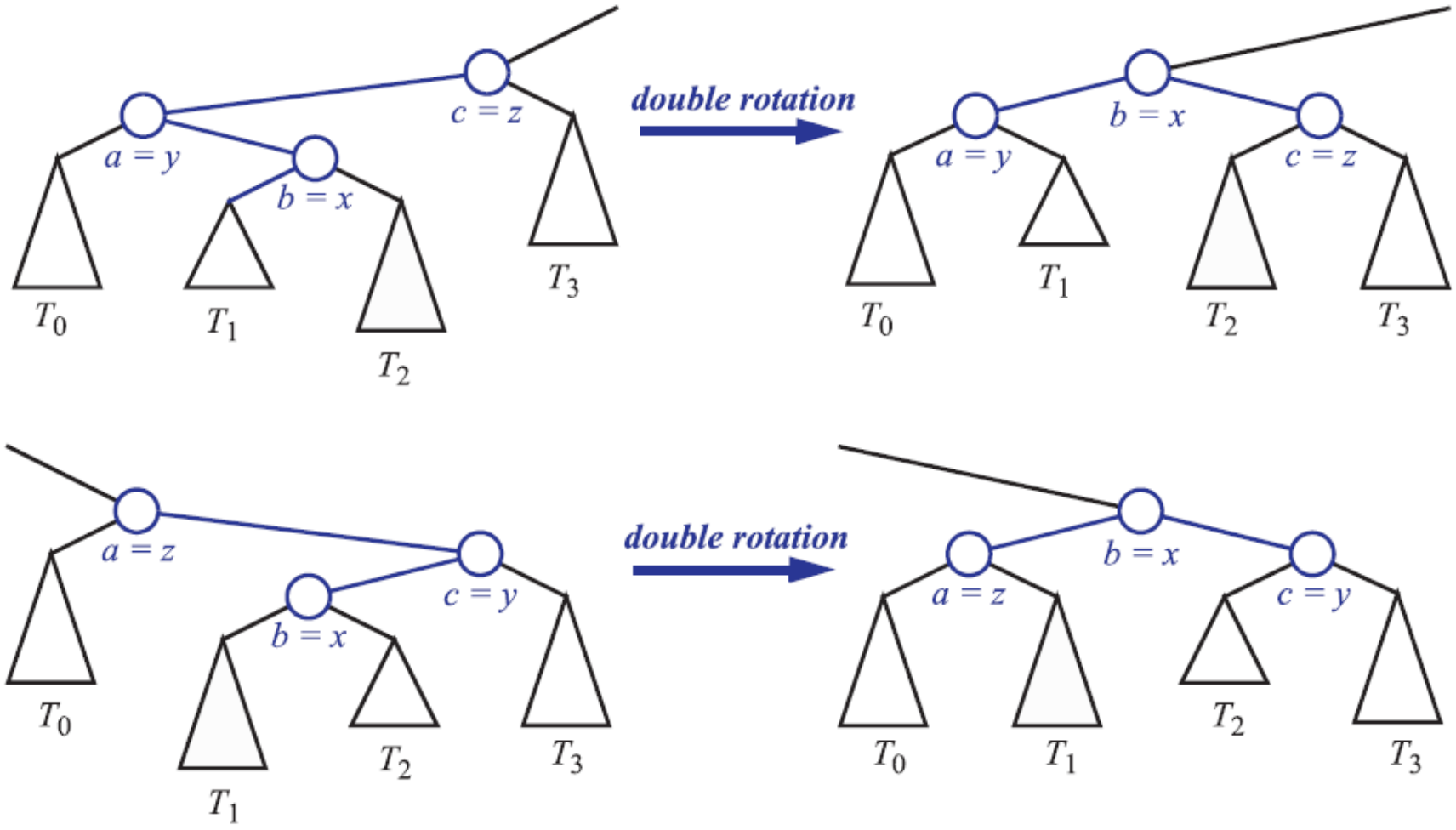
AVL Tree

- Tek döndürme



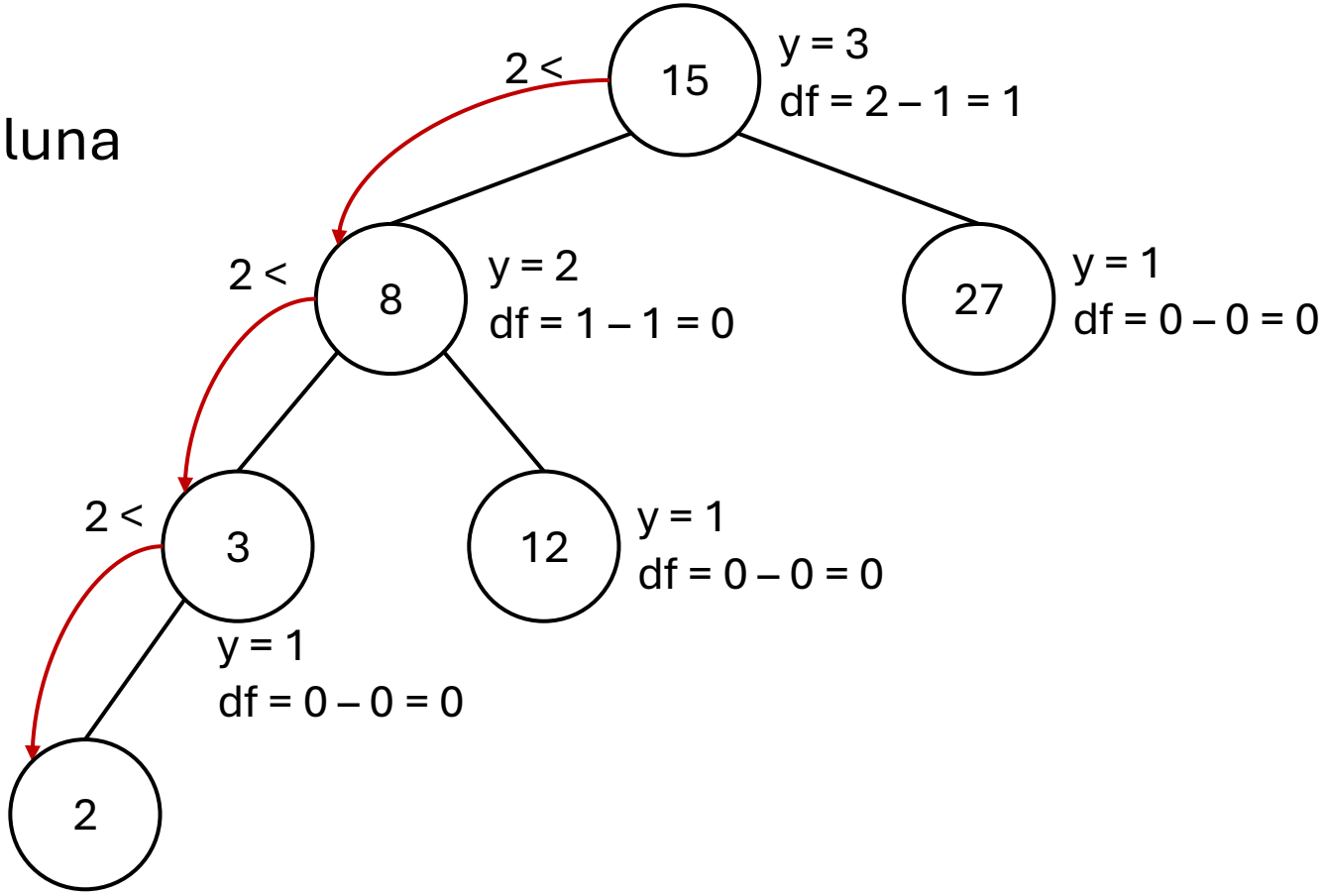
AVL Tree

- Çift döndürme



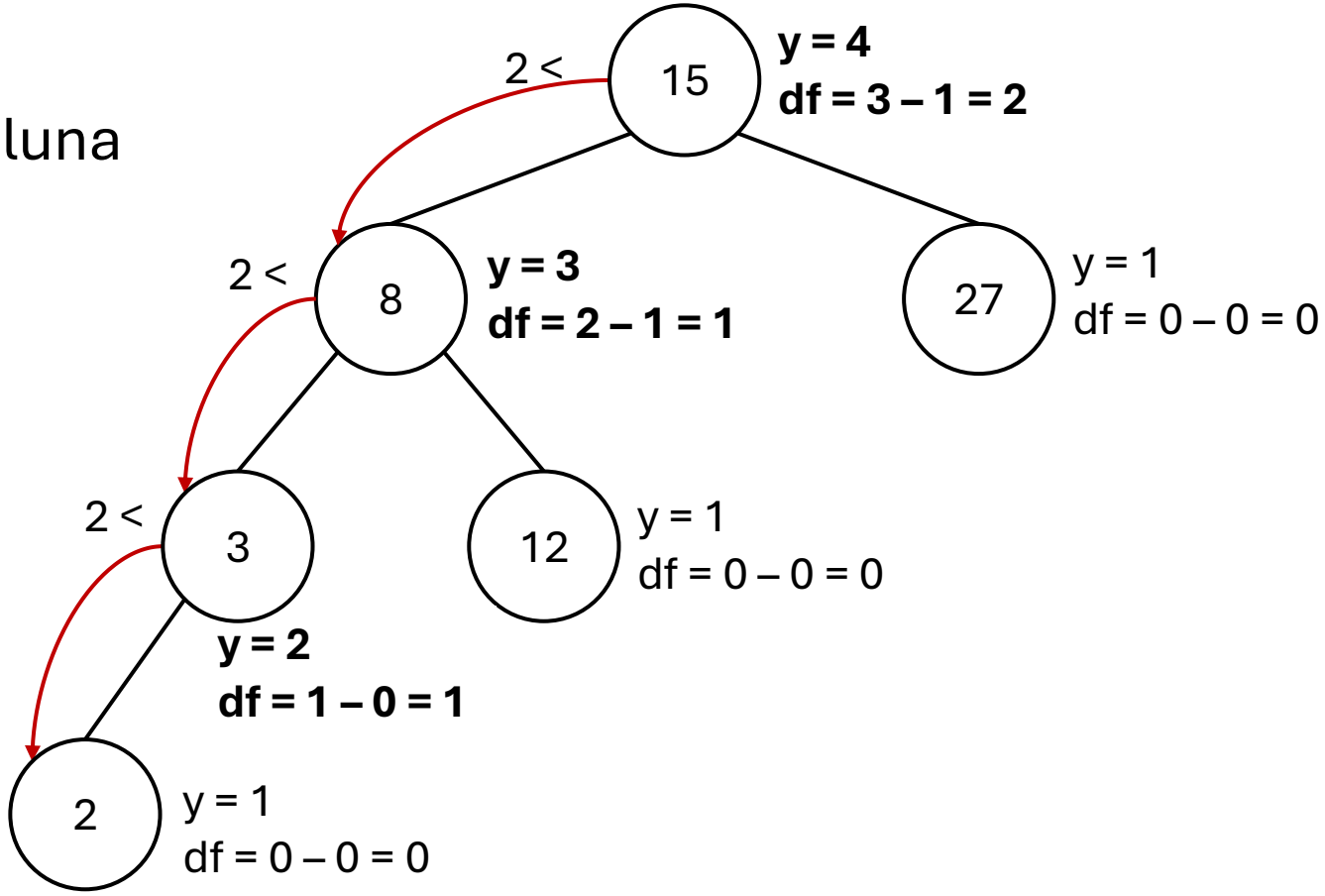
AVL Tree

- LL Dengesizliği
  - Düğümün sol altağacının soluna eklenmesi
  - Ekle: 2



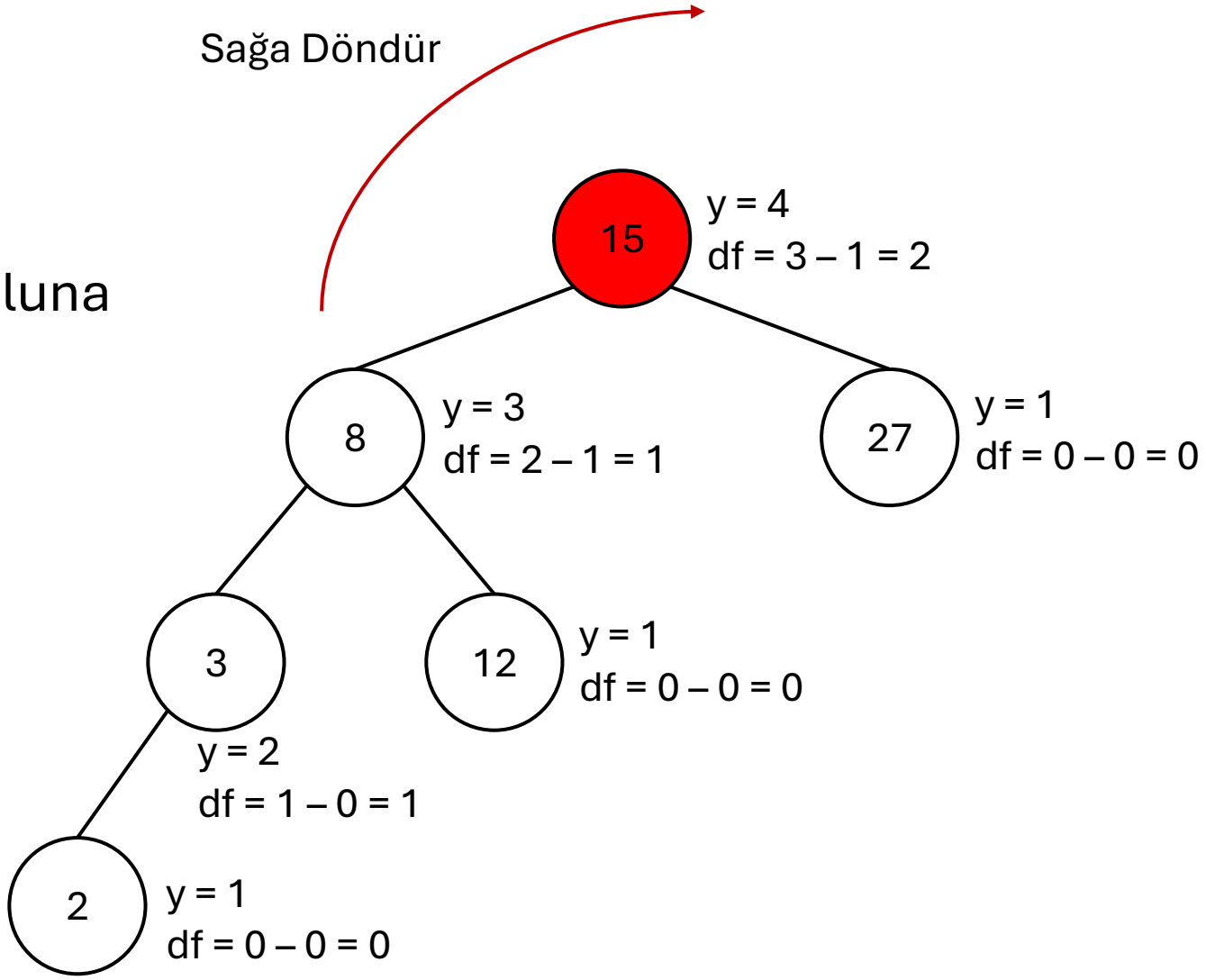
AVL Tree

- LL Dengesizliği
  - Düğümün sol altağacının soluna eklenmesi
  - Ekle: 2



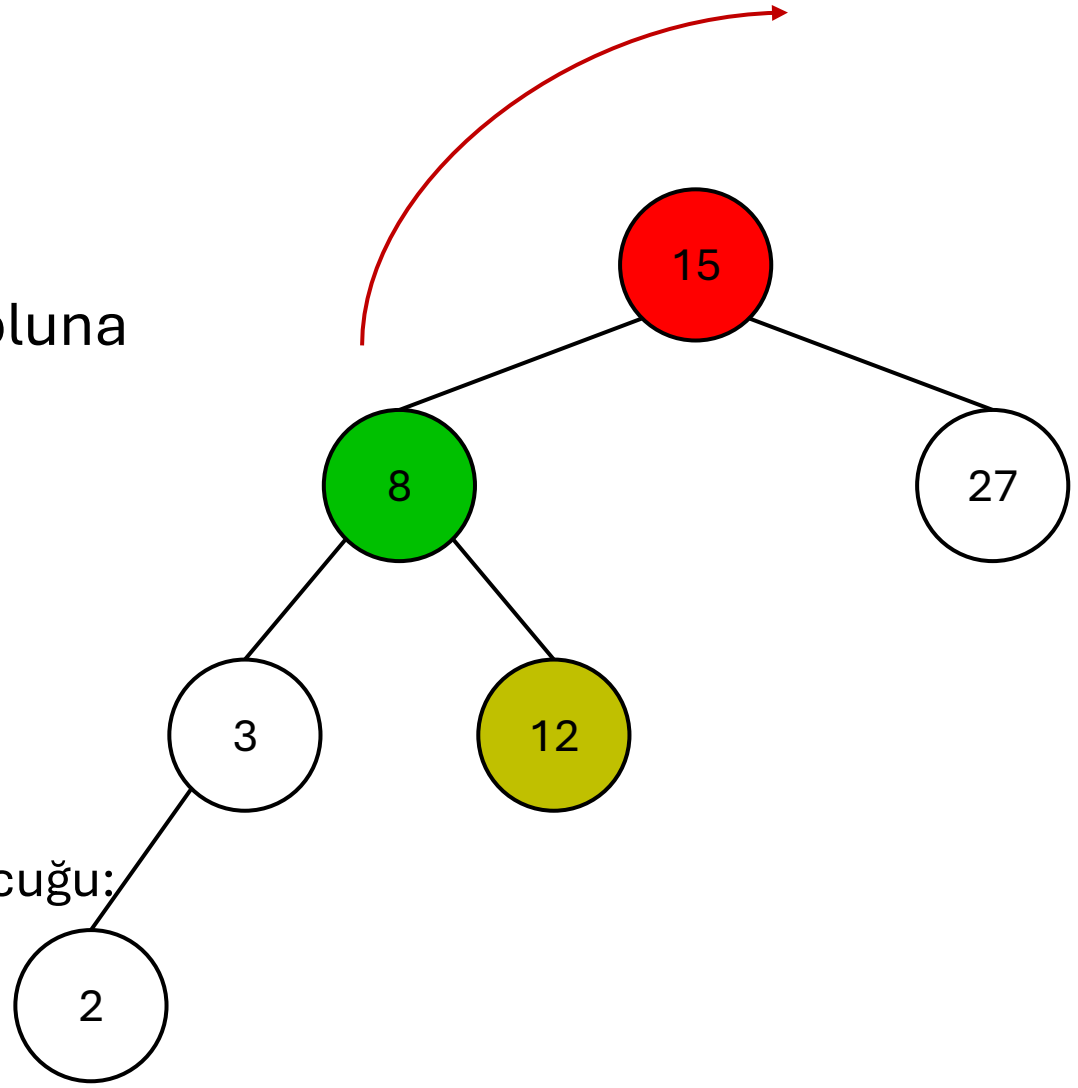
AVL Tree

- LL Dengesizliği
  - Düğümün sol altağacının soluna eklenmesi
  - Ekle: 2
- Çözüm
  - Sağa Döndür



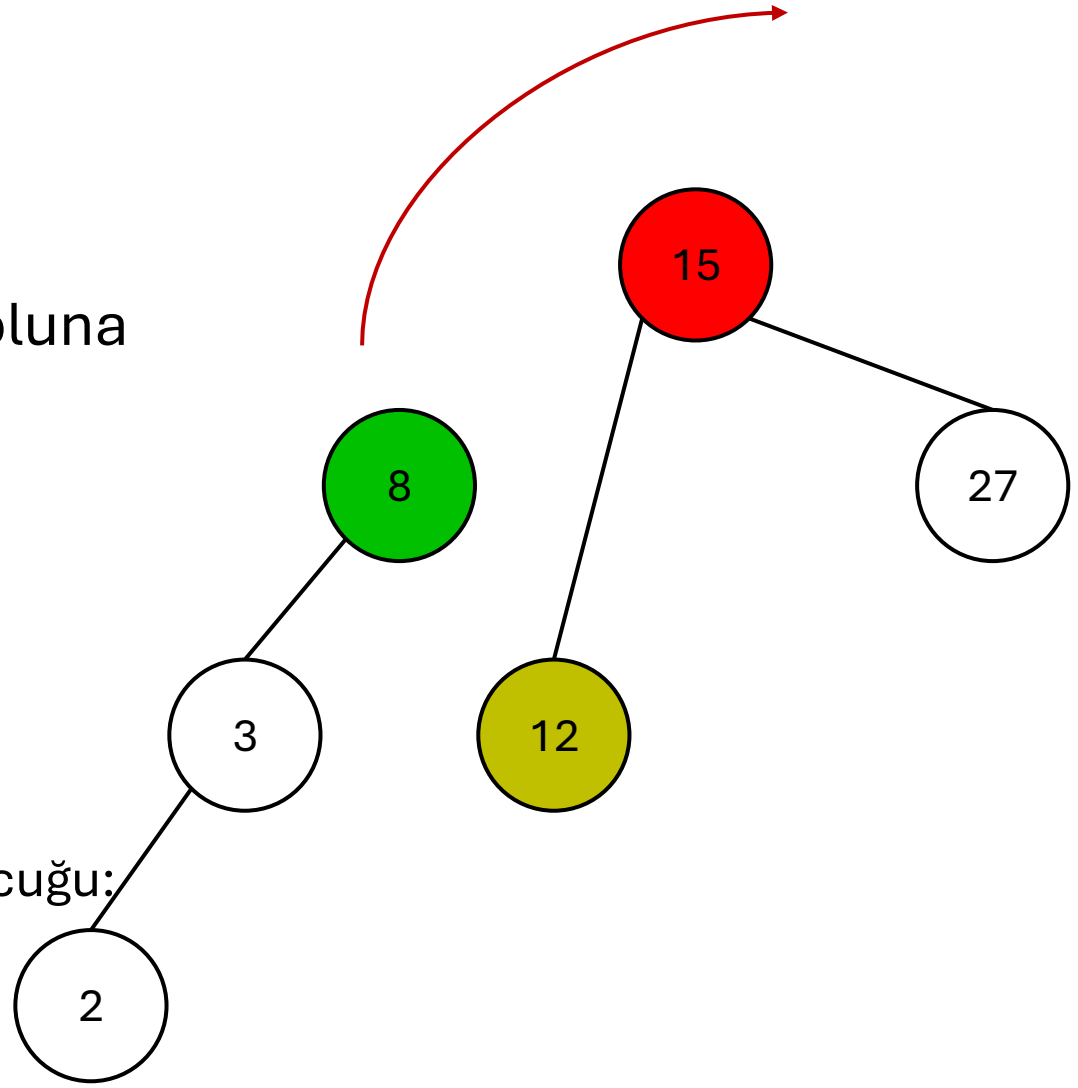
AVL Tree

- LL Dengesizliği
  - Düğümün sol altağacının soluna eklenmesi
  - Ekle: 2
- Çözüm
  - Sağa Döndür
    - Düğümün yeni sol çocuğu:
      - Sol çocuğun sağ çocuğu
    - Eski sol çocuğun yeni sağ çocuğu:
      - Düğümün kendisi



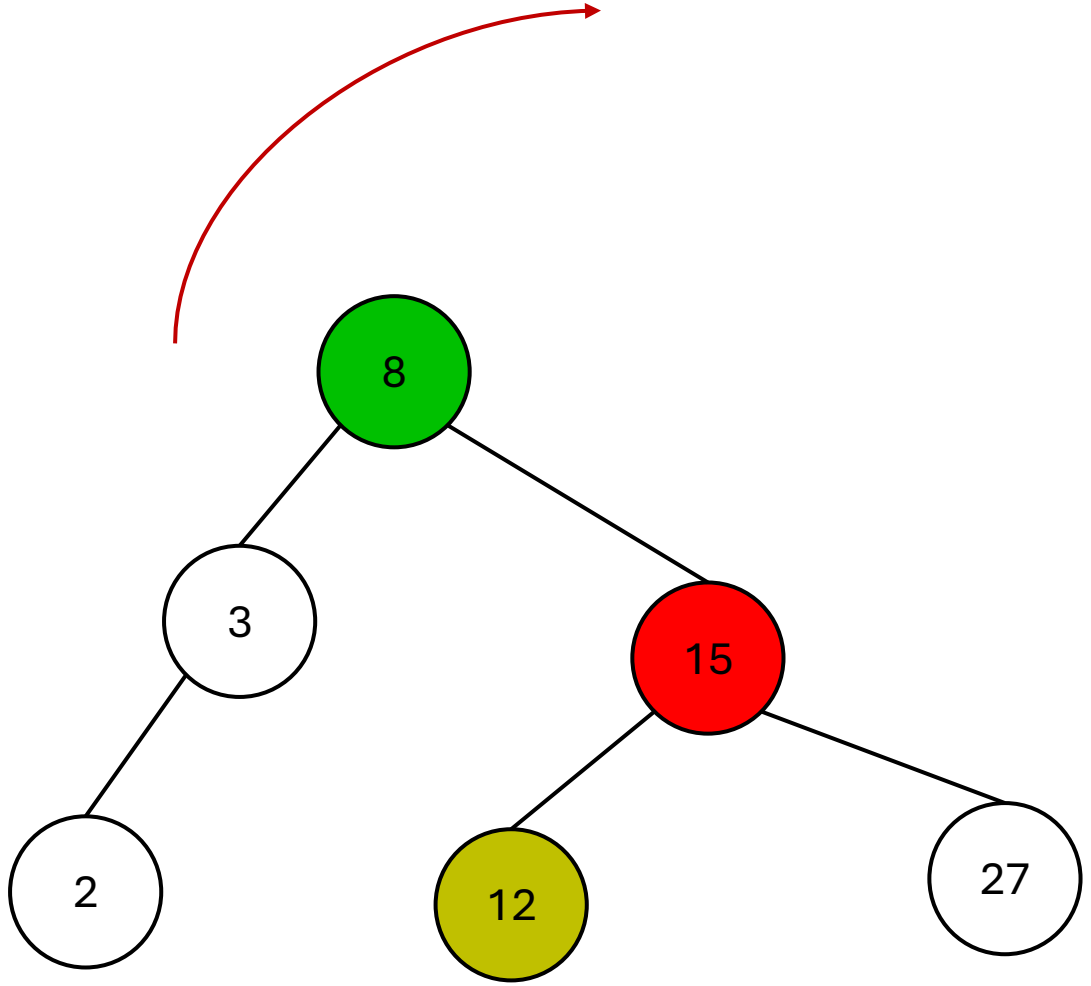
AVL Tree

- LL Dengesizliği
  - Düğümün sol altağacının soluna eklenmesi
  - Ekle: 2
- Çözüm
  - Sağa Döndür
    - **Düğümün yeni sol çocuğu:**
      - **Sol çocuğun sağ çocuğu**
    - Eski sol çocuğun yeni sağ çocuğu:
      - Düğümün kendisi



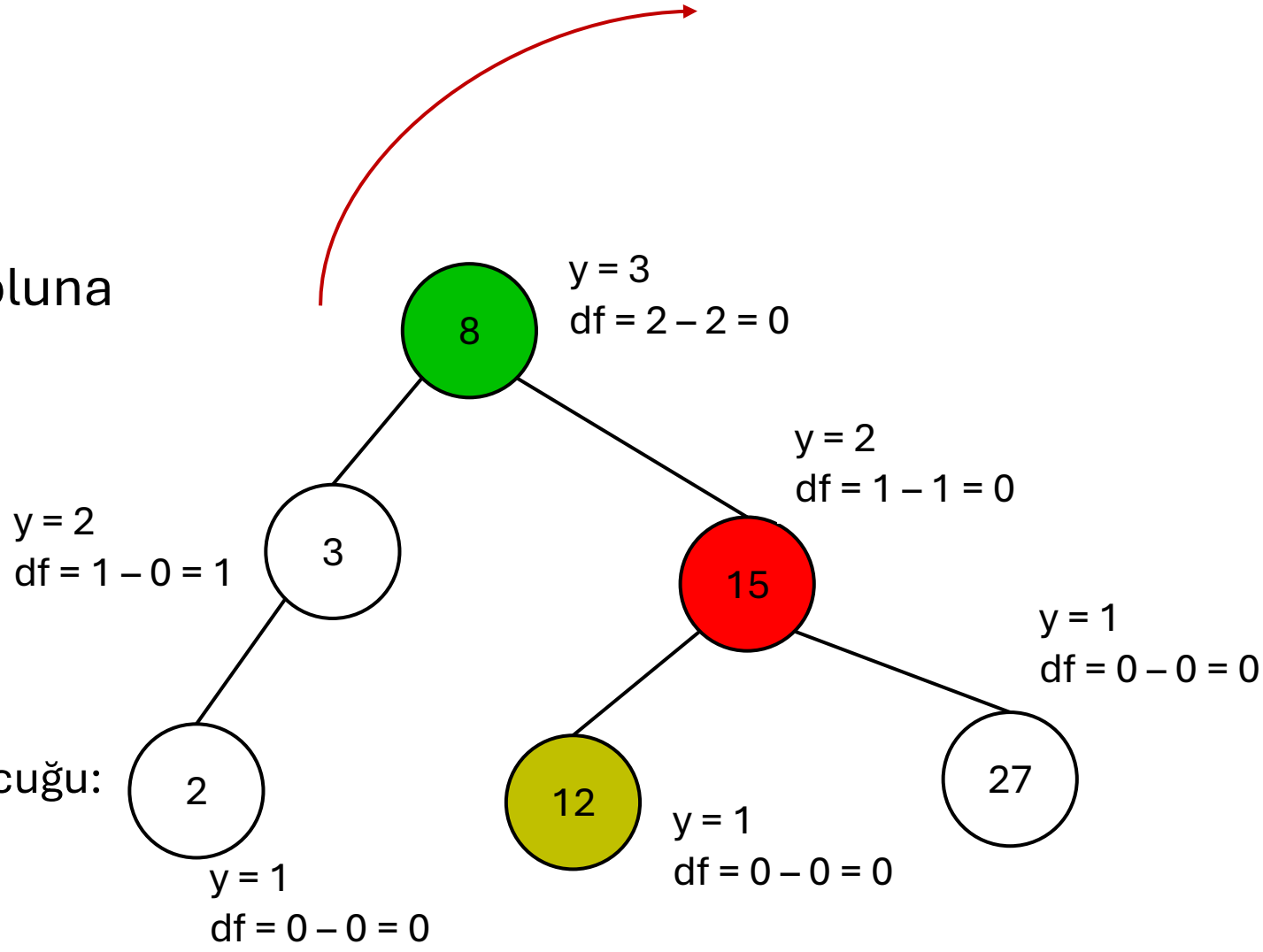
AVL Tree

- LL Dengesizliği
  - Düğümün sol altağacının soluna eklenmesi
  - Ekle: 2
- Çözüm
  - Sağa Döndür
    - Düğümün yeni sol çocuğu:
      - Sol çocuğun sağ çocuğu
    - **Eski sol çocuğun yeni sağ çocuğu:**
      - **Düğümün kendisi**



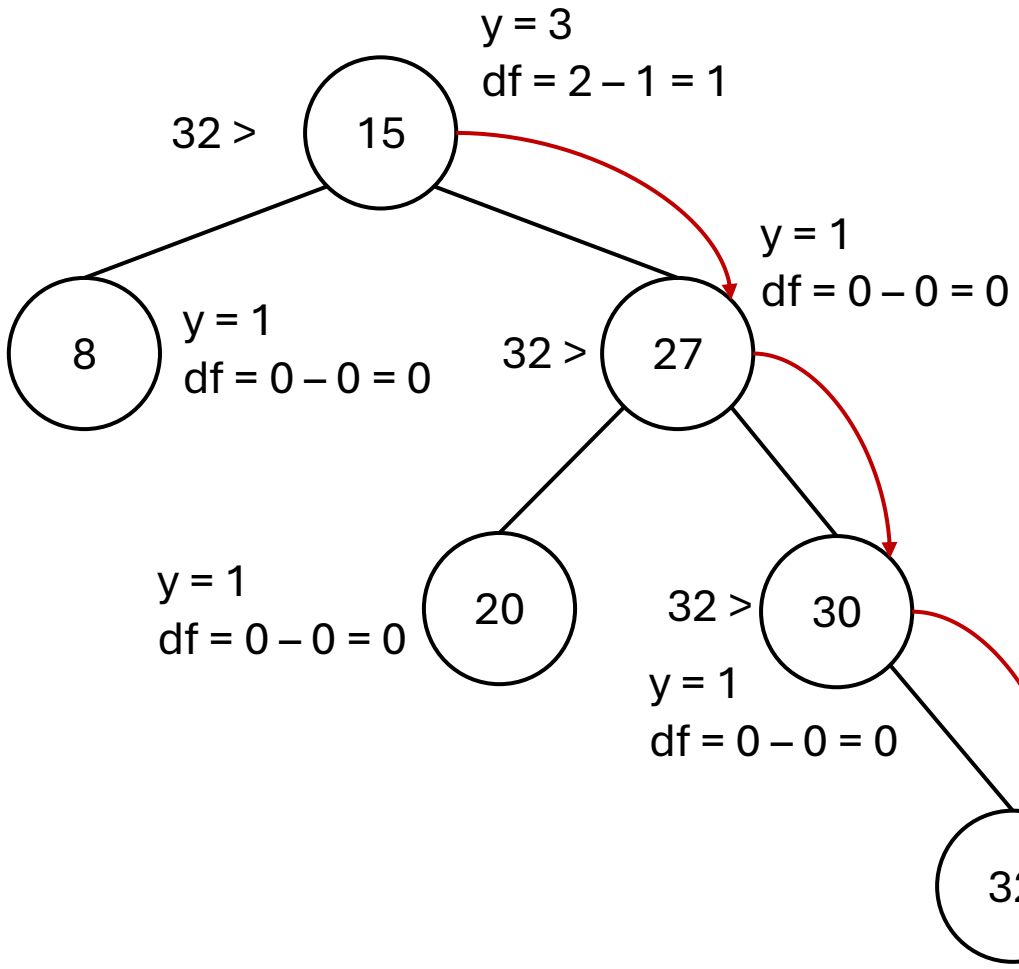
AVL Tree

- LL Dengesizliği
  - Düğümün sol altağacının soluna eklenmesi
  - Ekle: 2
- Çözüm
  - Sağa Döndür
    - Düğümün yeni sol çocuğu:
      - Sol çocuğun sağ çocuğu
    - Eski sol çocuğun yeni sağ çocuğu:
      - Düğümün kendisi



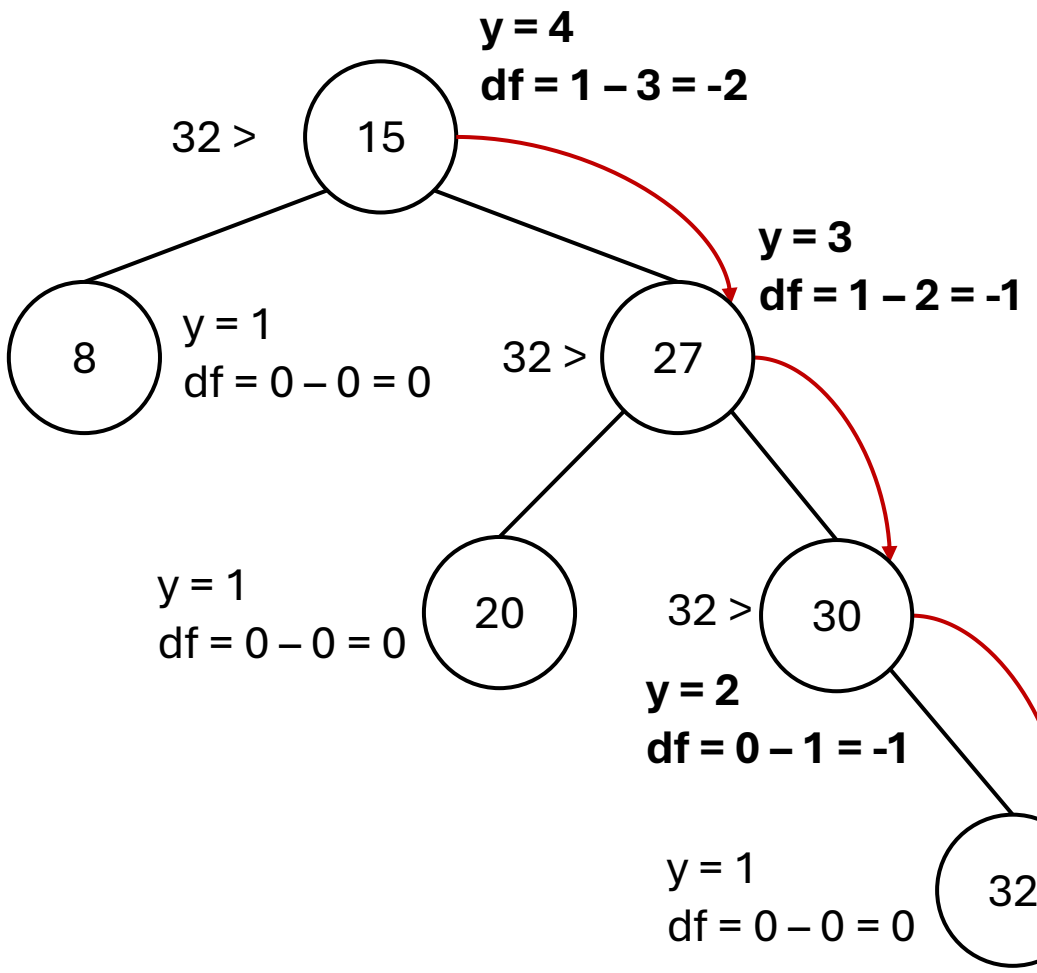
AVL Tree

- RR Dengesizliği
  - Düğümün sağ altağacının sağına eklenmesi
  - Ekle: 32



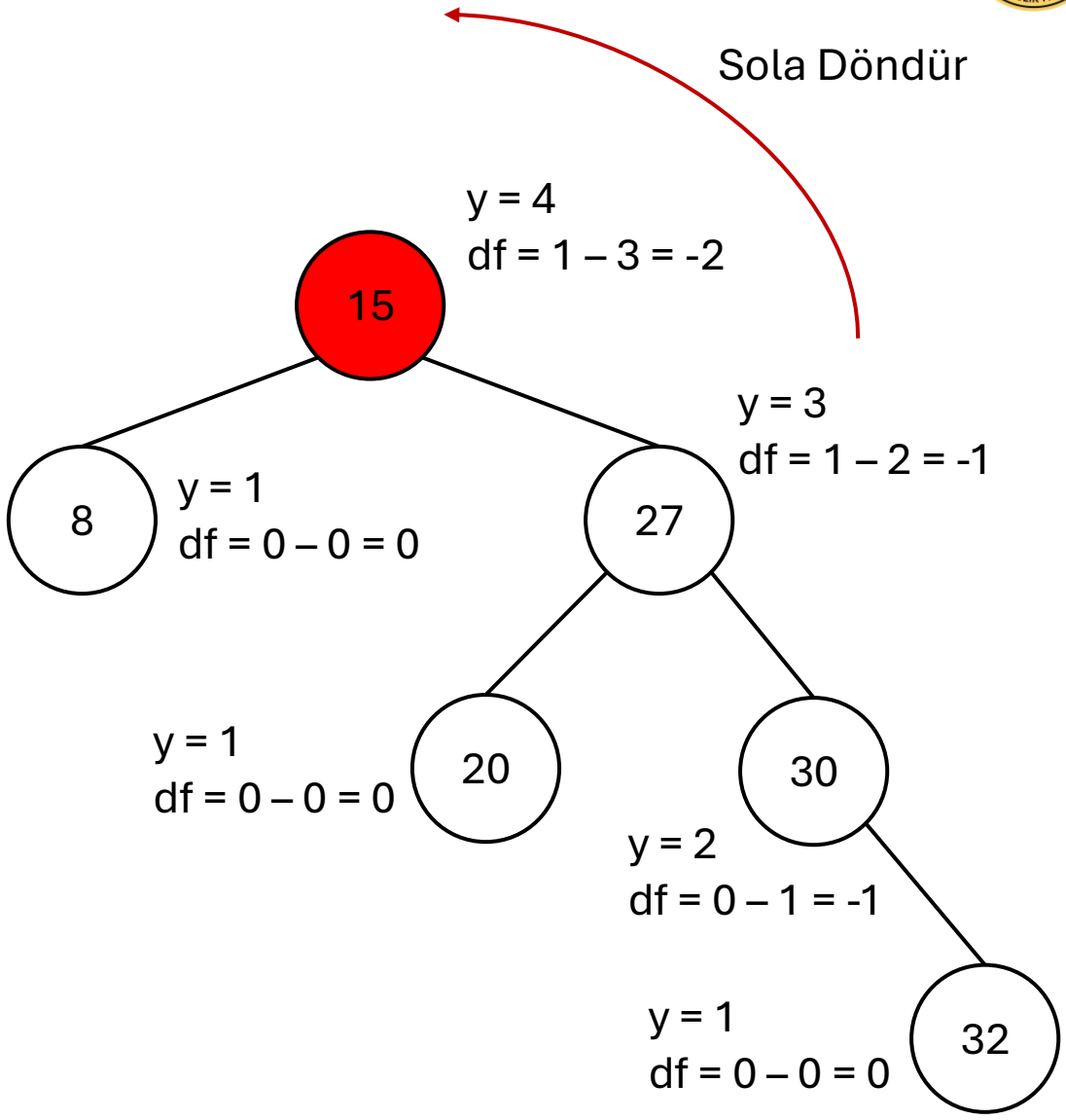
AVL Tree

- RR Dengesizliği
  - Düğümün sağ altağacının sağına eklenmesi
  - Ekle: 32



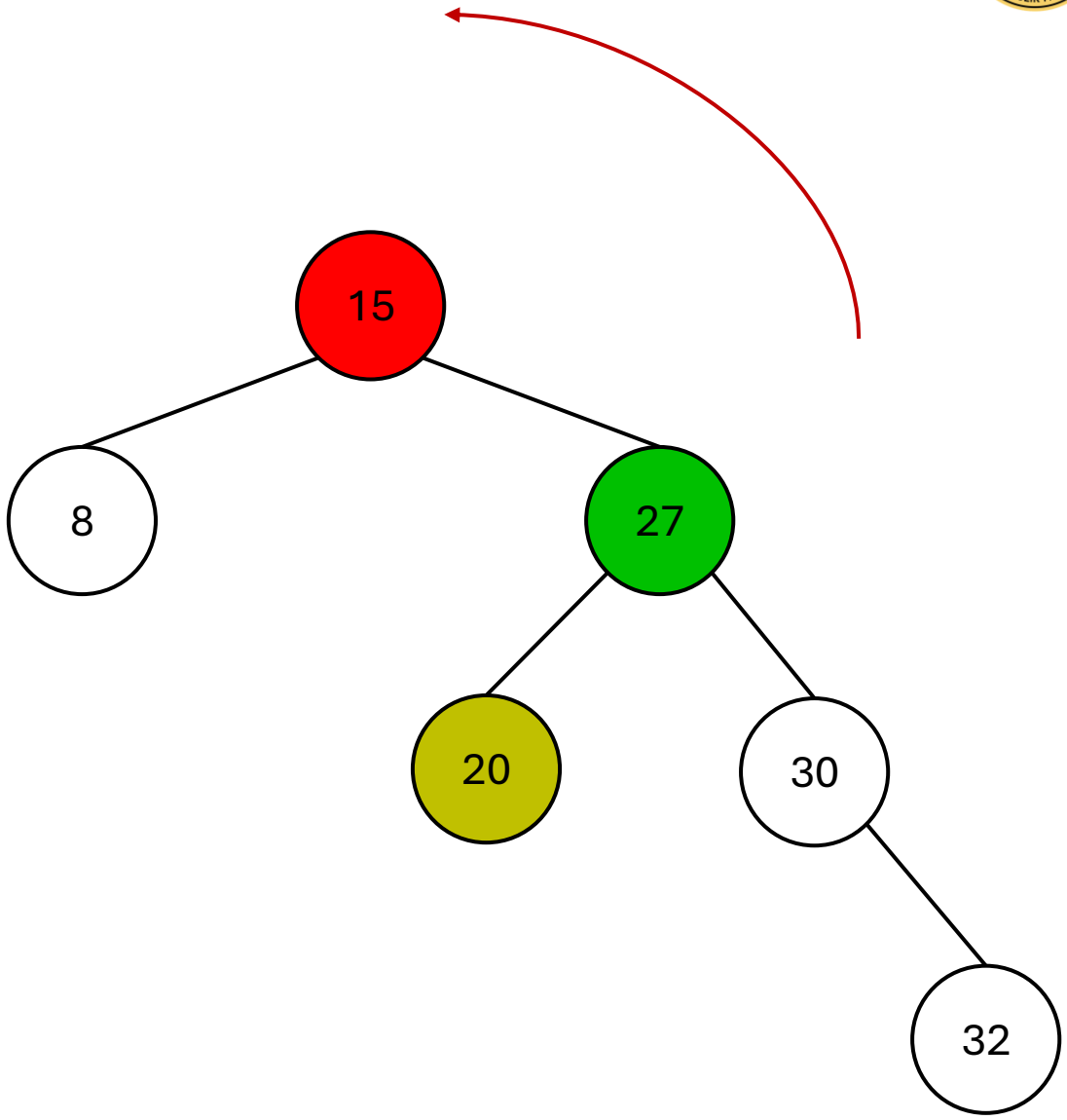
AVL Tree

- RR Dengesizliği
  - Düğümün sağ altağacının sağına eklenmesi
  - Ekle: 32
- Çözüm
  - Sola Döndür



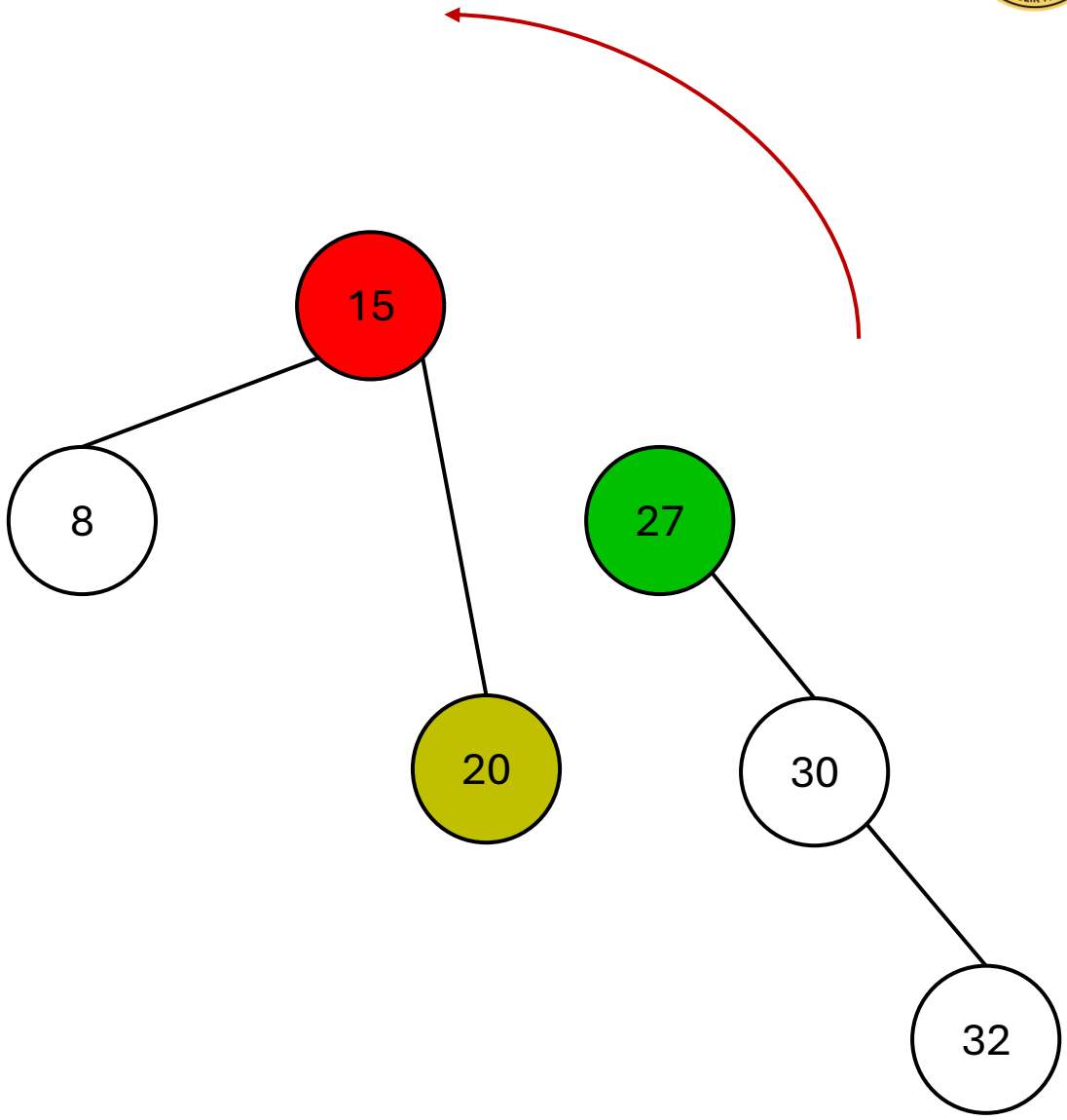
AVL Tree

- RR Dengesizliği
  - Düğümün sağ altağacının sağına eklenmesi
  - Ekle: 32
- Çözüm
  - Sola Döndür
    - Düğümün yeni sağ çocuğu:
      - Sağ çocuğun sol çocuğu
    - Eski sağ çocuğun yeni sol çocuğu:
      - Düğümün kendisi



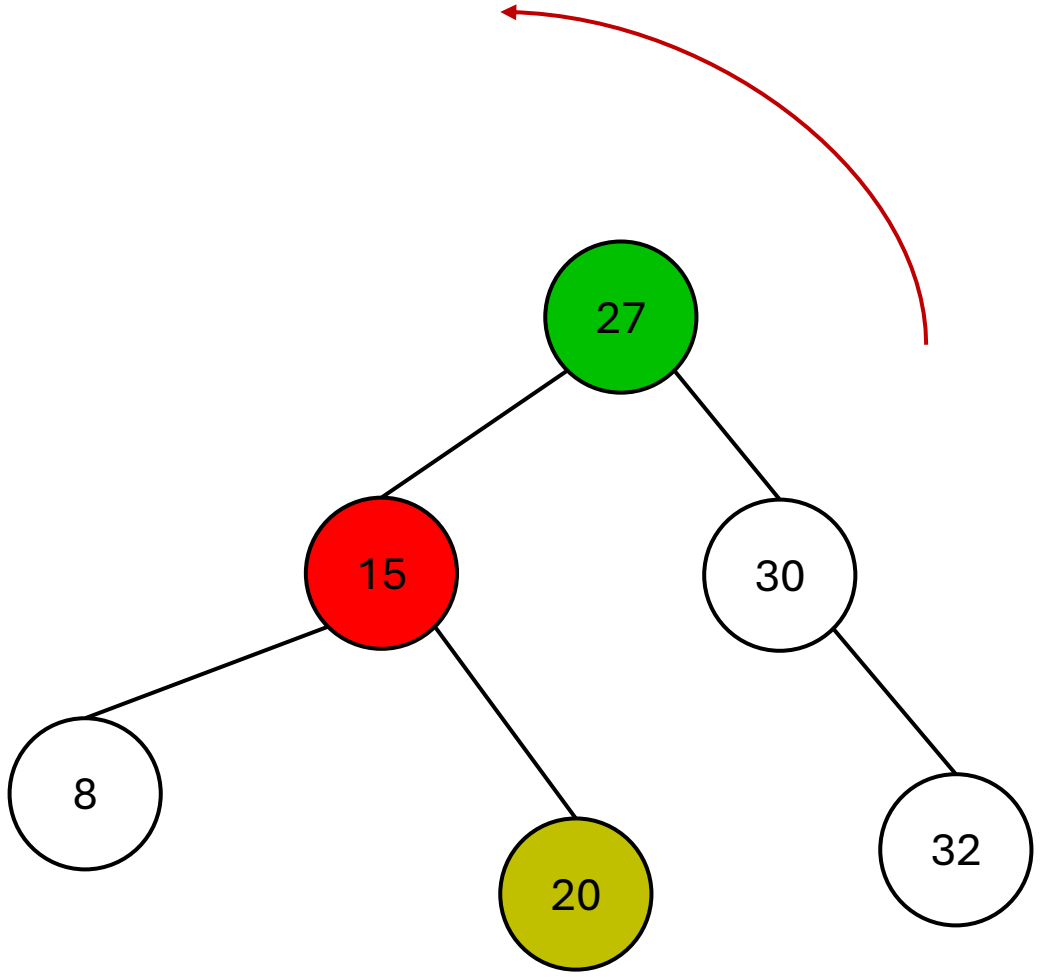
AVL Tree

- RR Dengesizliği
  - Düğümün sağ altağacının sağına eklenmesi
  - Ekle: 32
- Çözüm
  - Sola Döndür
    - **Düğümün yeni sağ çocuğu:**
      - **Sağ çocuğun sol çocuğu**
    - Eski sağ çocuğun yeni sol çocuğu:
      - Düğümün kendisi



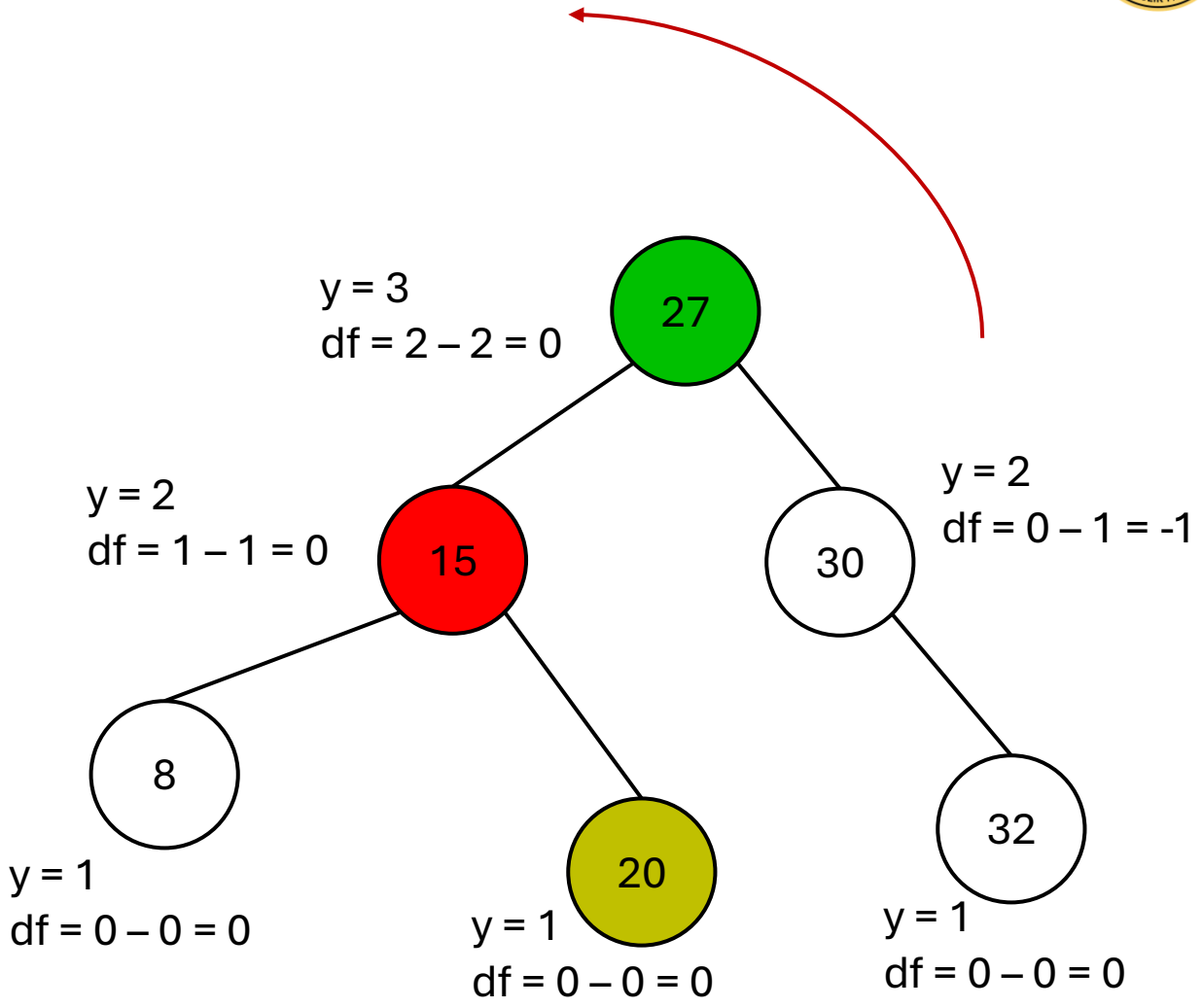
AVL Tree

- RR Dengesizliği
  - Düğümün sağ altağacının sağına eklenmesi
  - Ekle: 32
- Çözüm
  - Sola Döndür
    - Düğümün yeni sağ çocuğu:
      - Sağ çocuğun sol çocuğu
    - **Eski sağ çocuğun yeni sol çocuğu:**
      - **Düğümün kendisi**



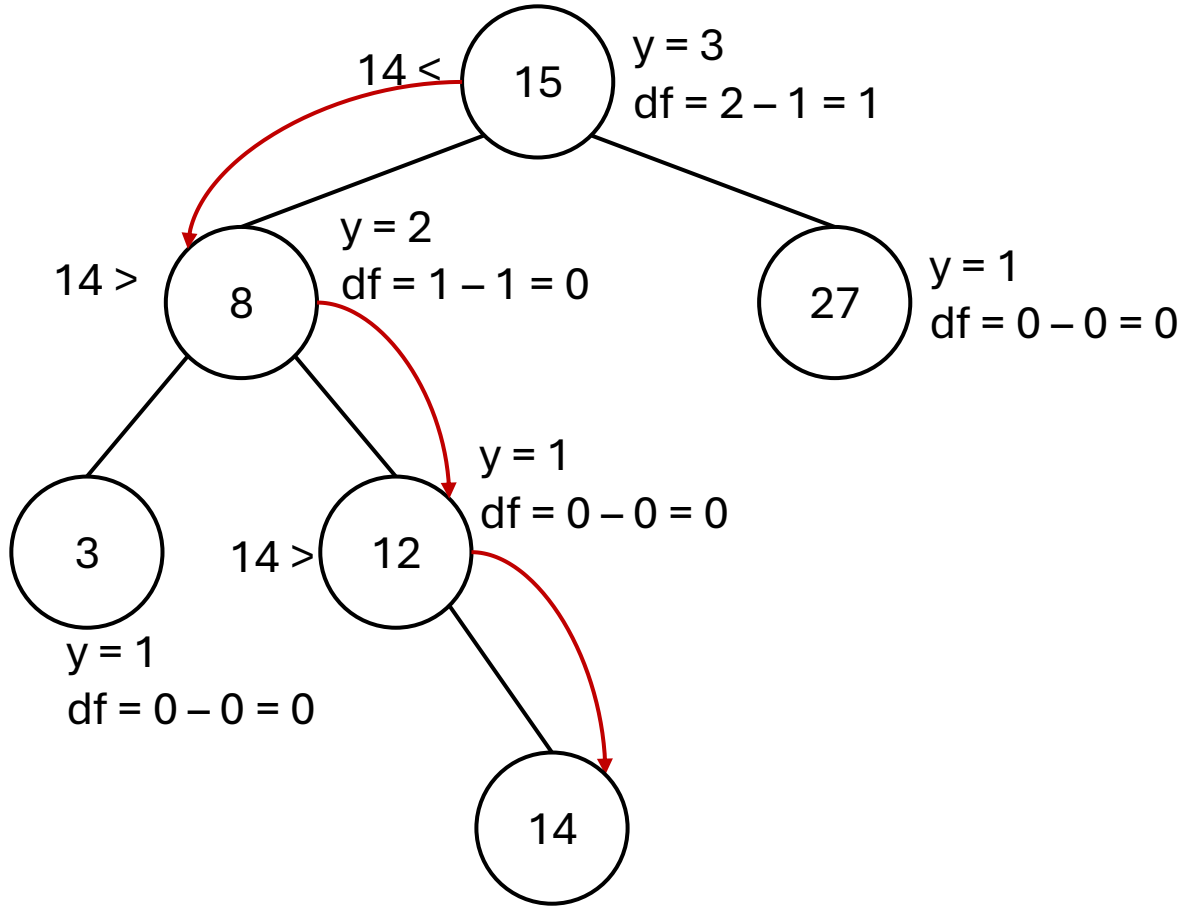
AVL Tree

- RR Dengesizliği
  - Düğümün sağ altağacının sağına eklenmesi
  - Ekle: 32
- Çözüm
  - Sola Döndür
    - Düğümün yeni sağ çocuğu:
      - Sağ çocuğun sol çocuğu
    - Eski sağ çocuğun yeni sol çocuğu:
      - Düğümün kendisi



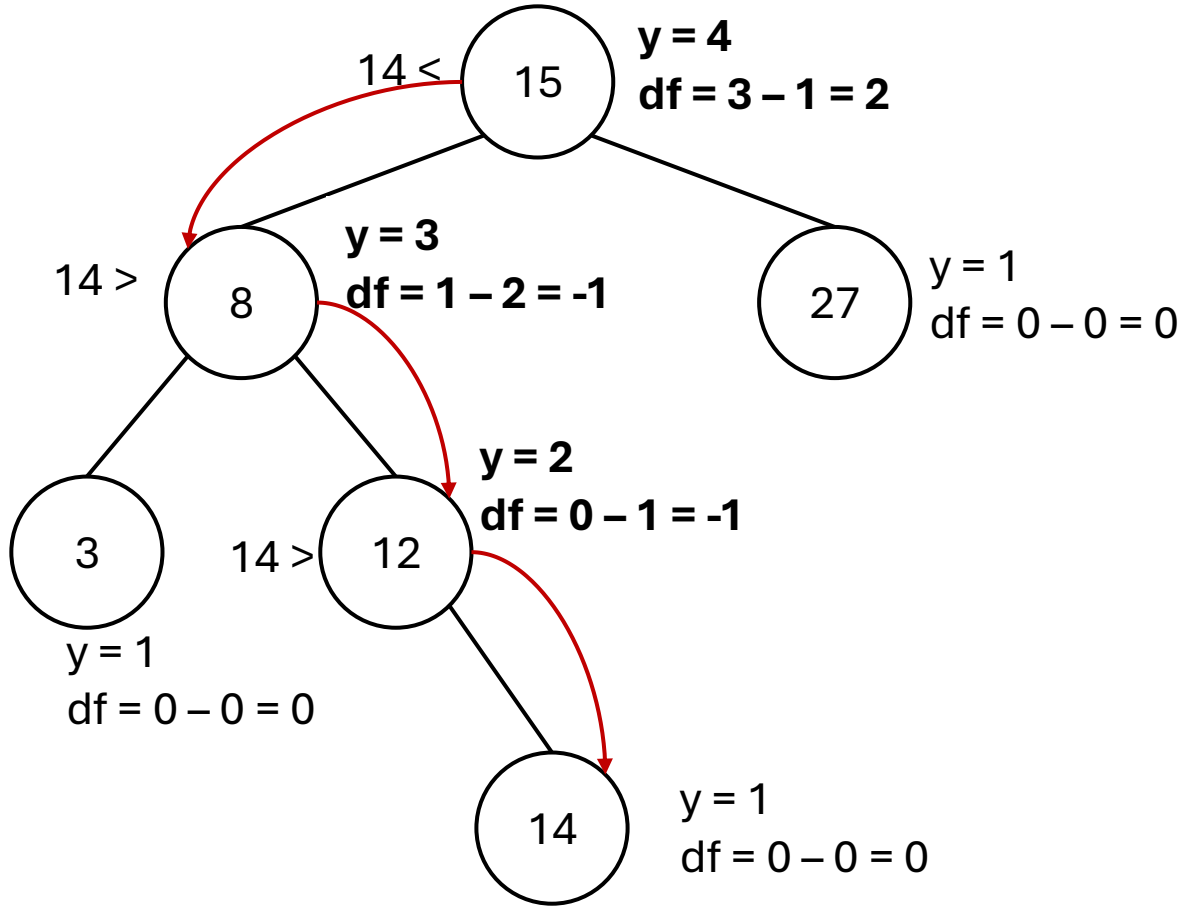
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14



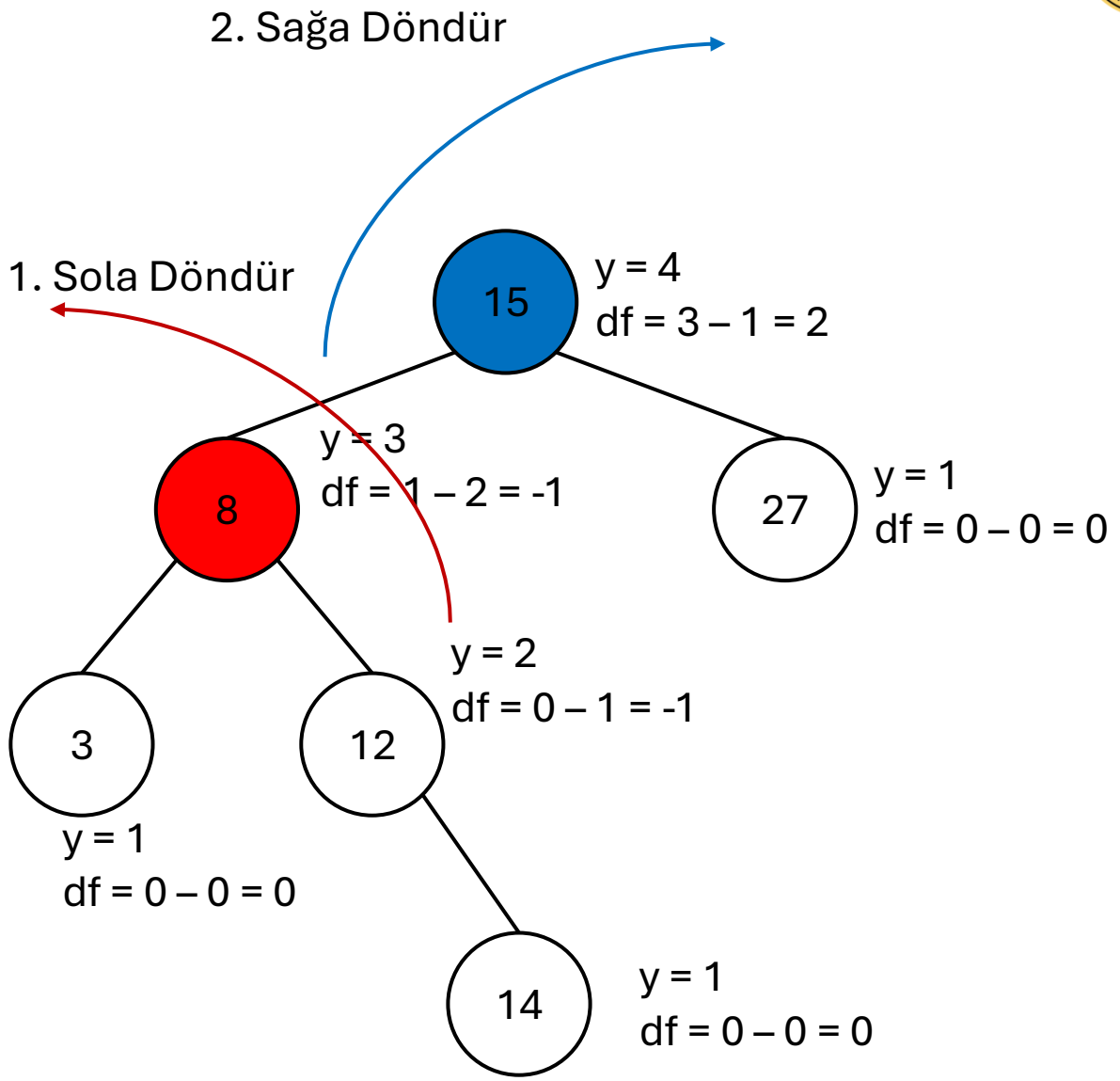
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14



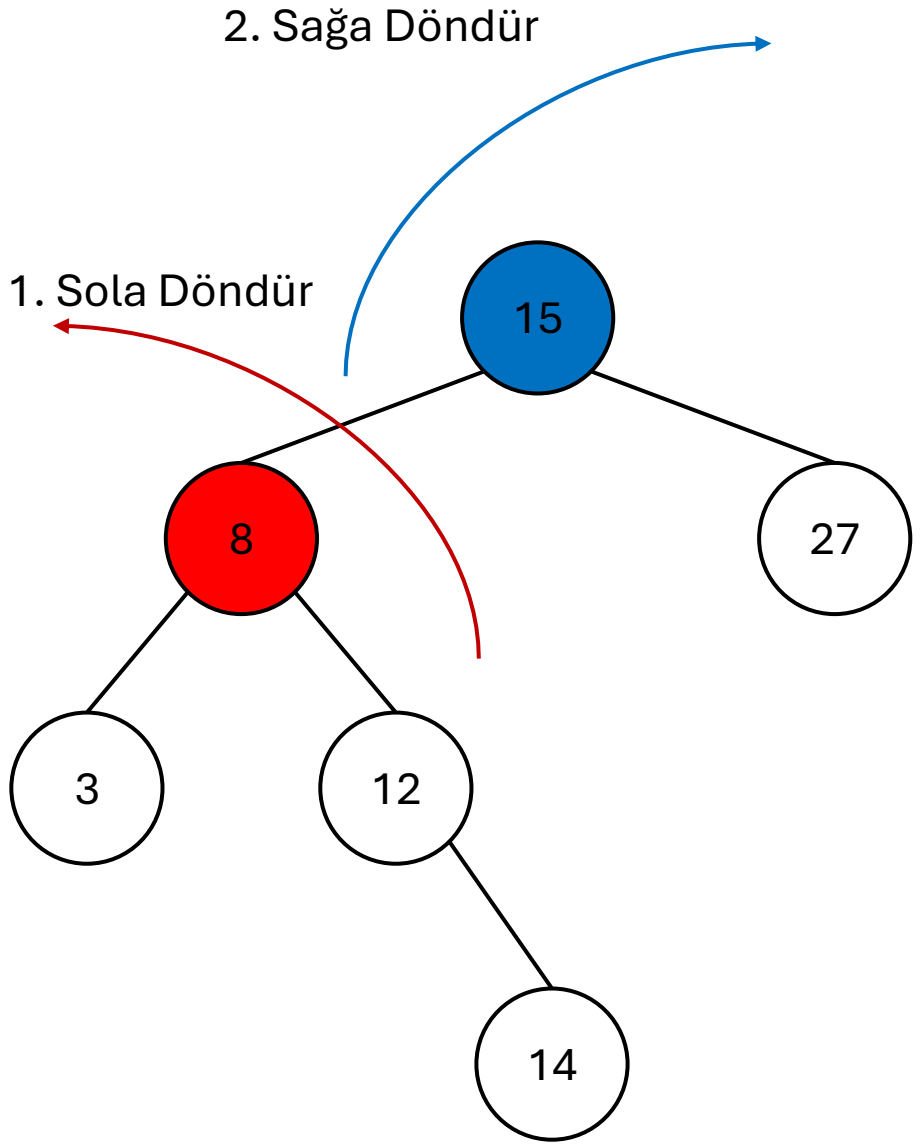
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14
- Çözüm
  - Önce sol çocuğu sola döndür
  - Sonra düğümün kendisini sağa döndür



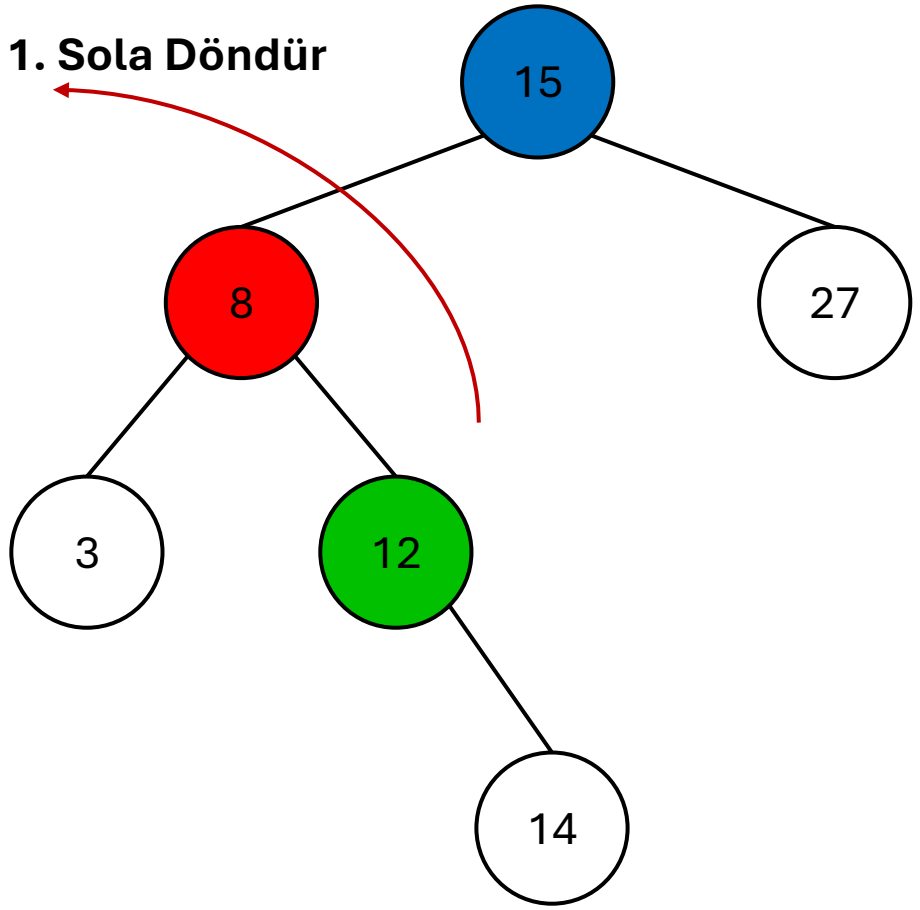
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14
- Çözüm
  - **Önce sol çocuğu sola döndür**
  - Sonra düğümün kendisini sağa döndür



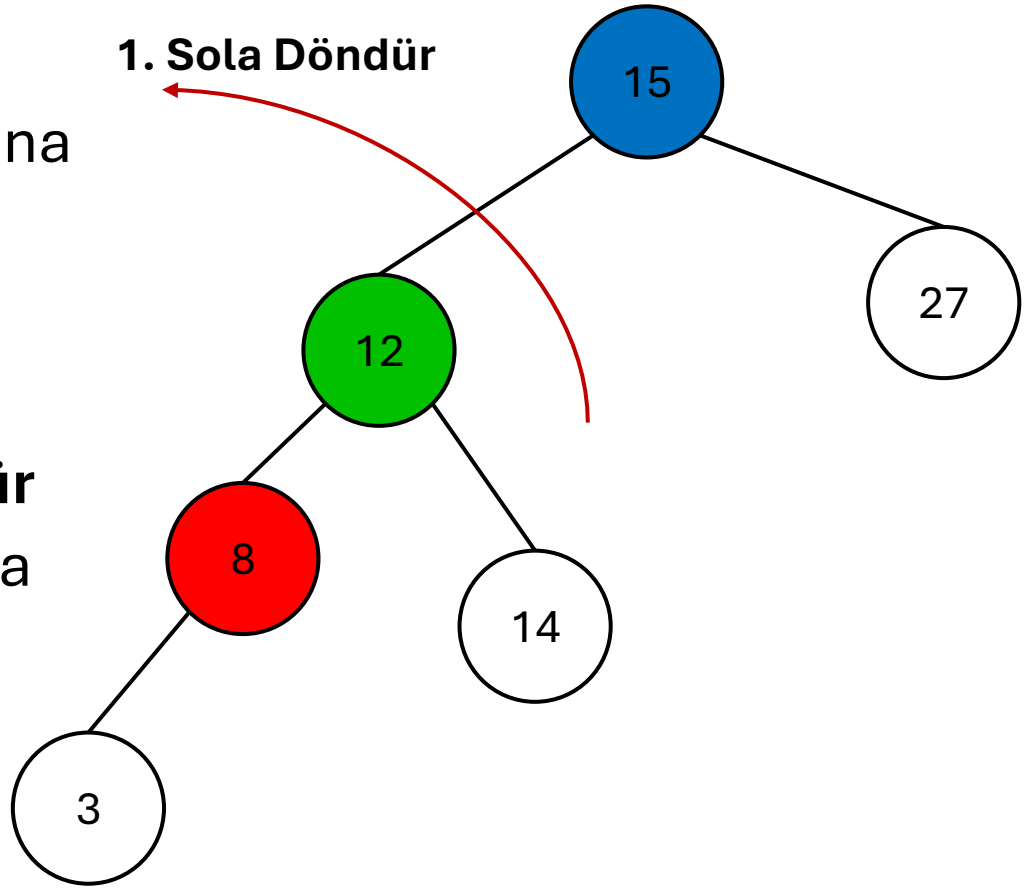
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14
- Çözüm
  - **Önce sol çocuğu sola döndür**
  - Sonra düğümün kendisini sağa döndür



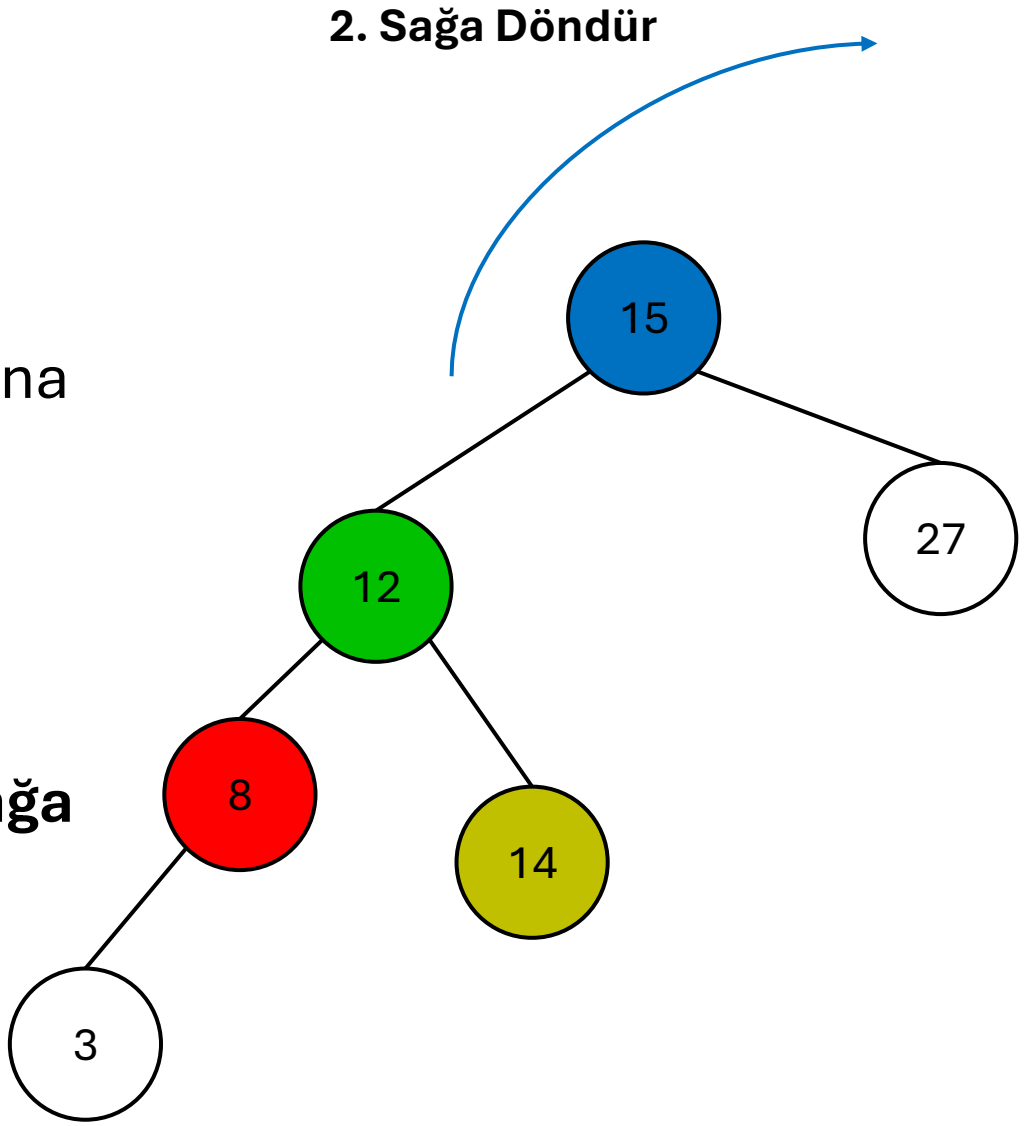
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14
- Çözüm
  - **Önce sol çocuğu sola döndür**
  - Sonra düğümün kendisini sağa döndür



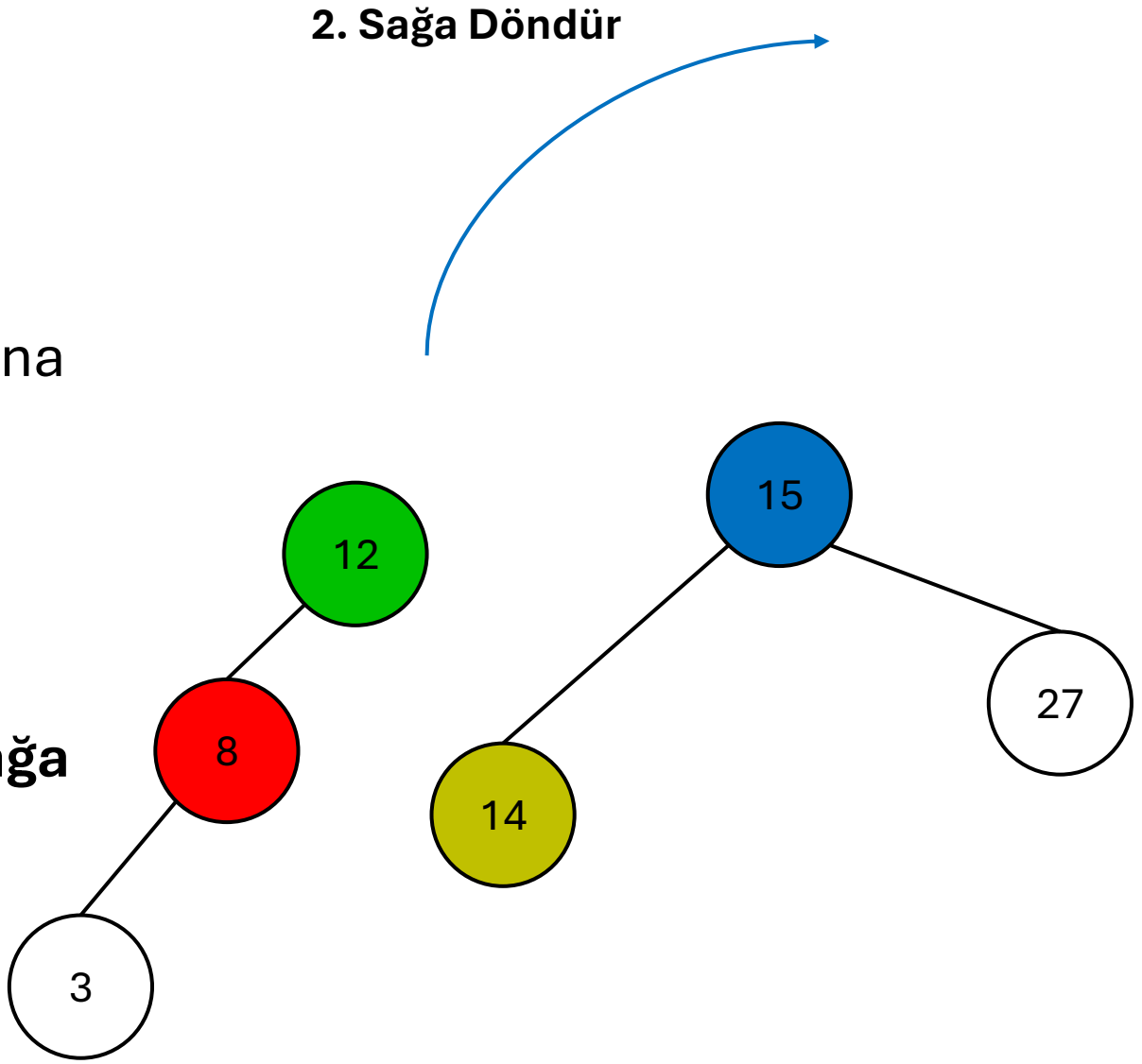
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14
- Çözüm
  - Önce sol çocuğu sola döndür
  - **Sonra düğümün kendisini sağa döndür**



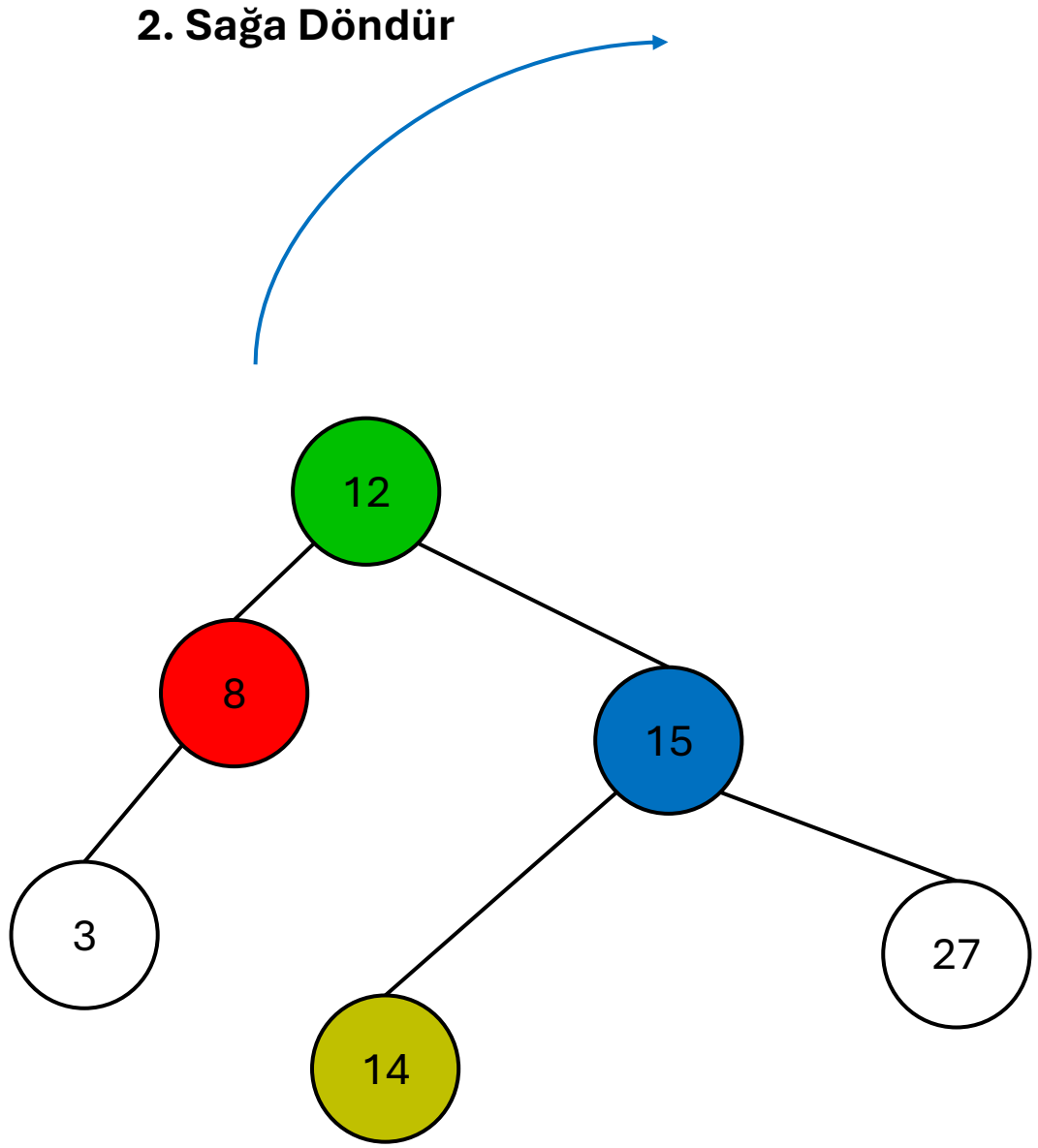
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14
- Çözüm
  - Önce sol çocuğu sola döndür
  - **Sonra düğümün kendisini sağa döndür**



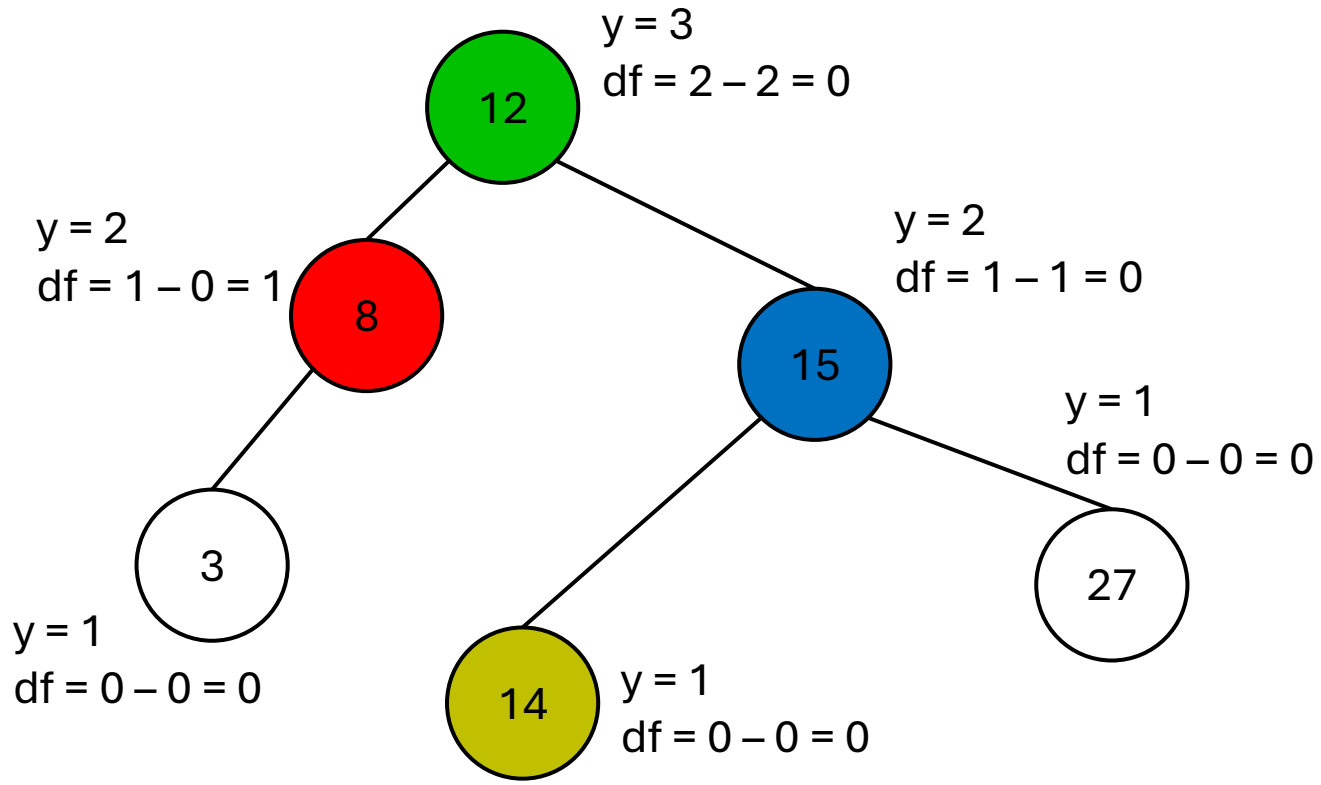
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14
- Çözüm
  - Önce sol çocuğu sola döndür
  - **Sonra düğümün kendisini sağa döndür**



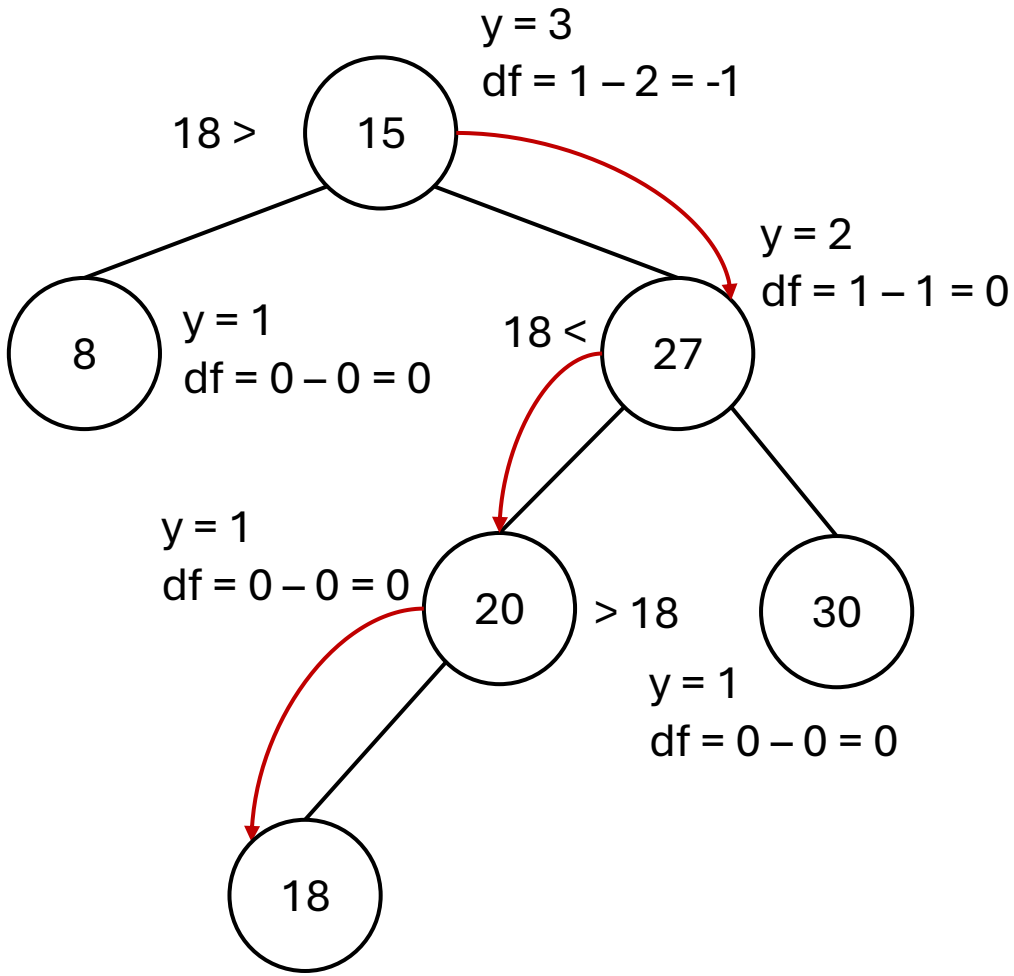
AVL Tree

- LR Dengesizliği
  - Düğümün sol altağacının sağına eklenmesi
  - Ekle: 14
- Çözüm
  - Önce sol çocuğu sola döndür
  - Sonra düğümün kendisini sağa döndür



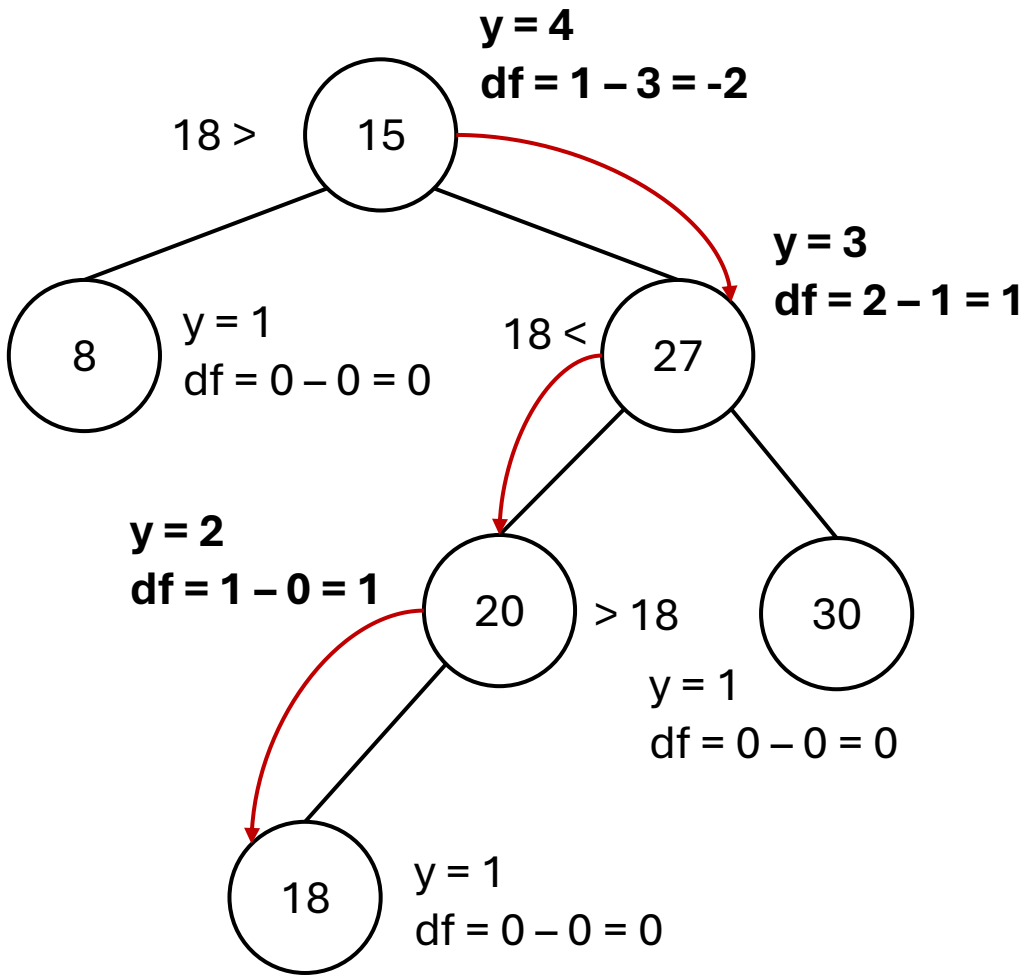
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18



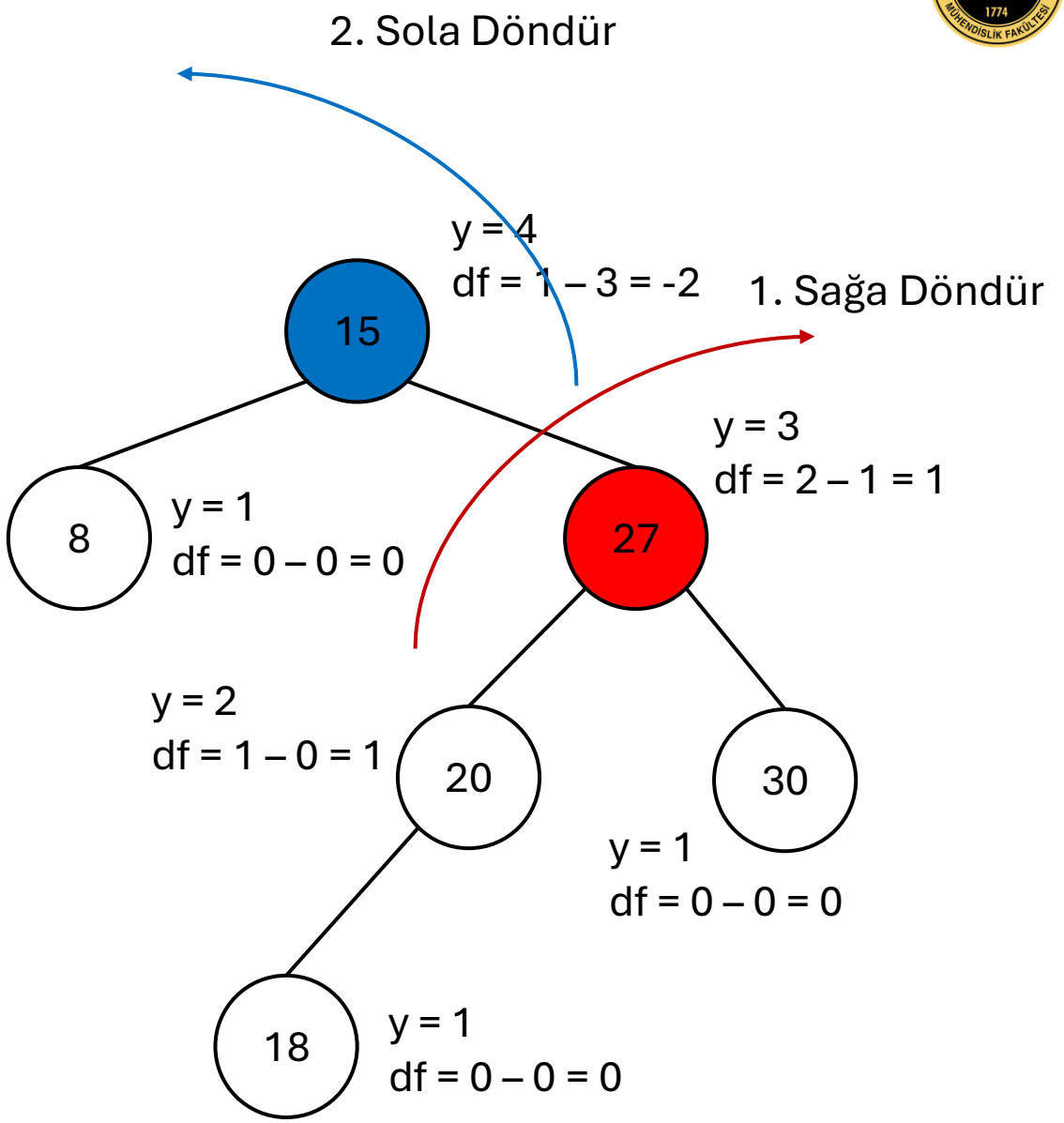
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18



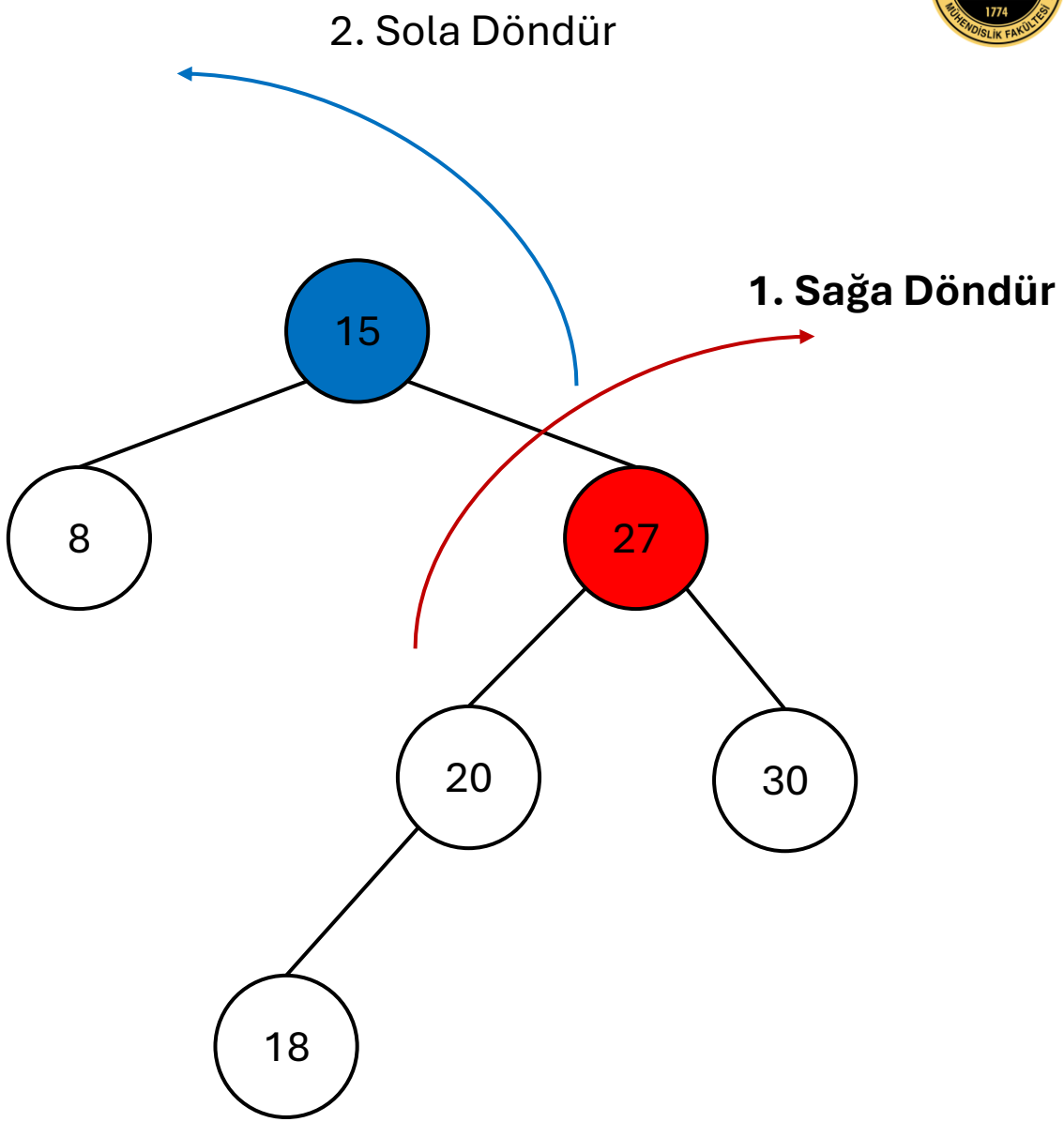
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18
- Çözüm
  - Önce sağ çocuğu sağa döndür
  - Sonra düğümün kendisini sola döndür



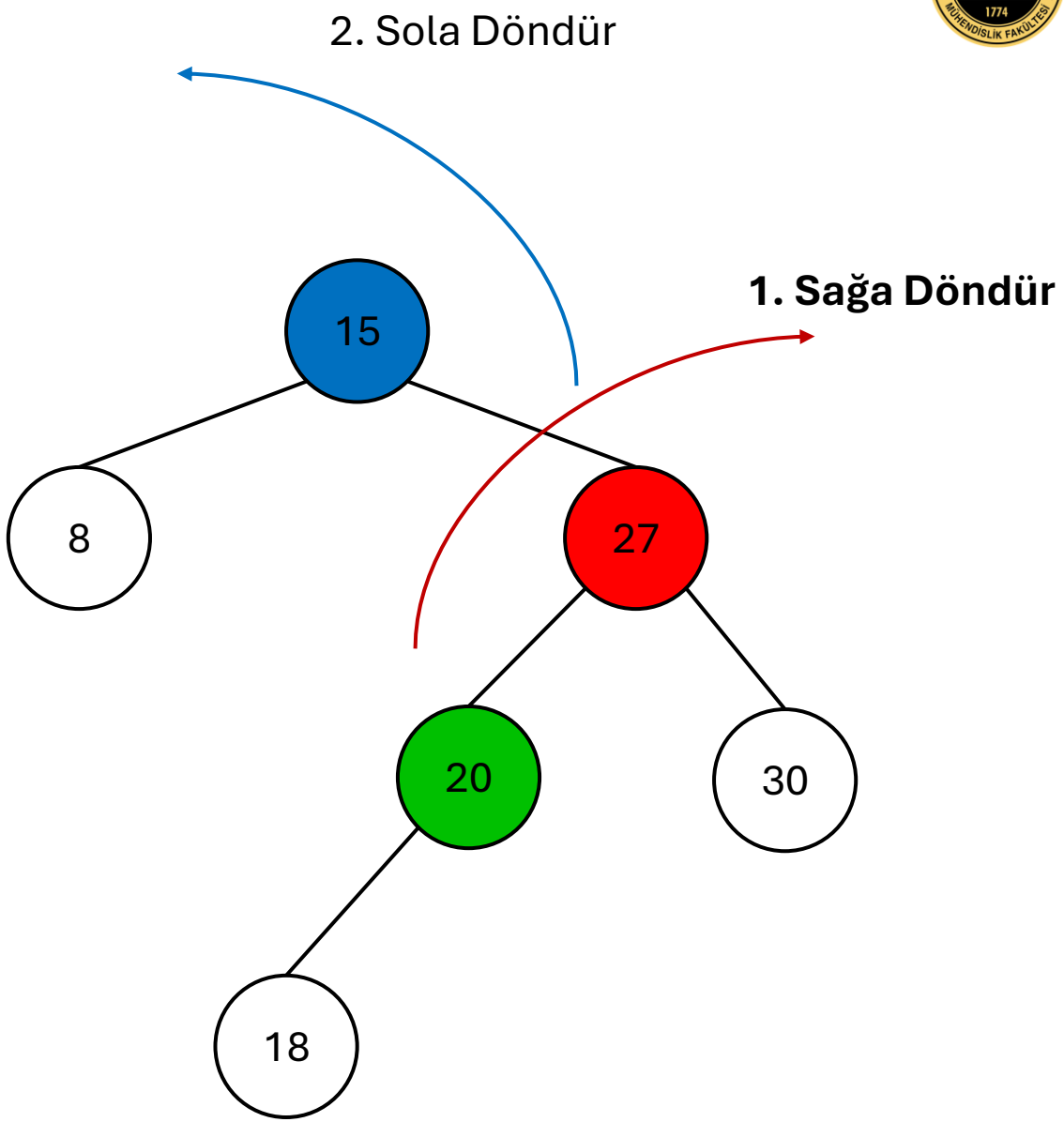
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18
- Çözüm
  - **Önce sağ çocuğu sağa döndür**
  - Sonra düğümün kendisini sola döndür



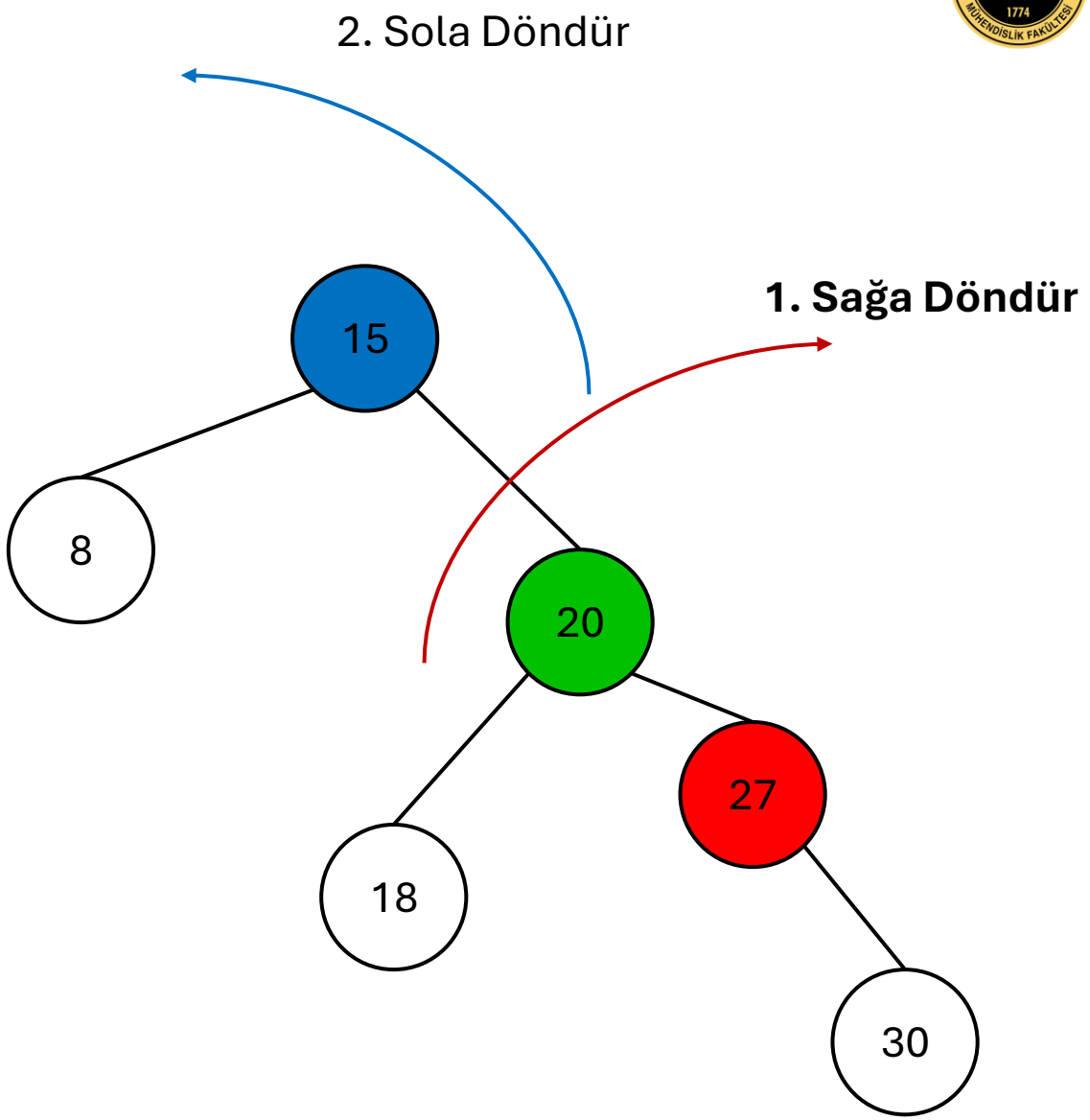
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18
- Çözüm
  - **Önce sağ çocuğu sağa döndür**
  - Sonra düğümün kendisini sola döndür



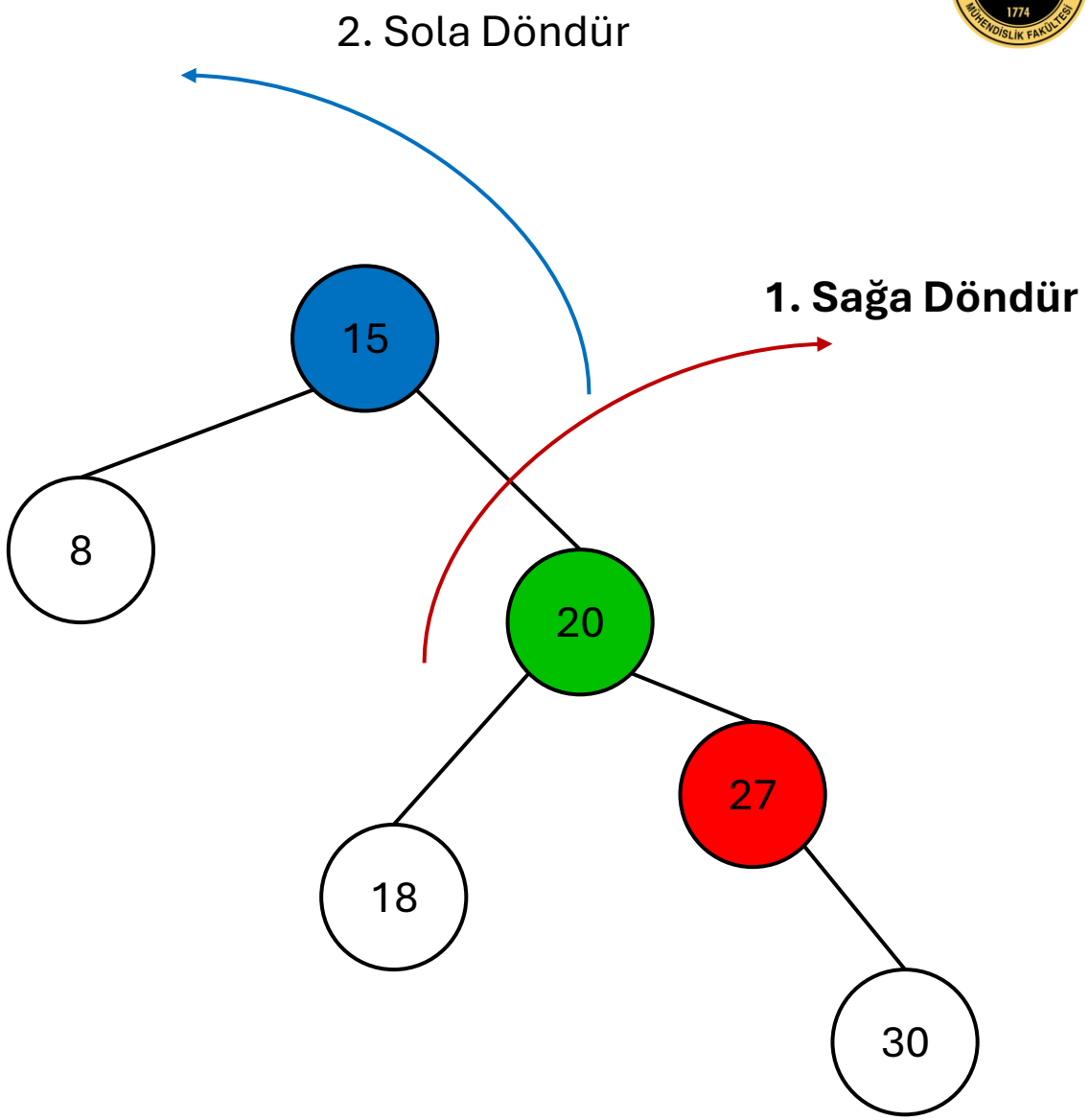
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18
- Çözüm
  - **Önce sağ çocuğu sağa döndür**
  - Sonra düğümün kendisini sola döndür



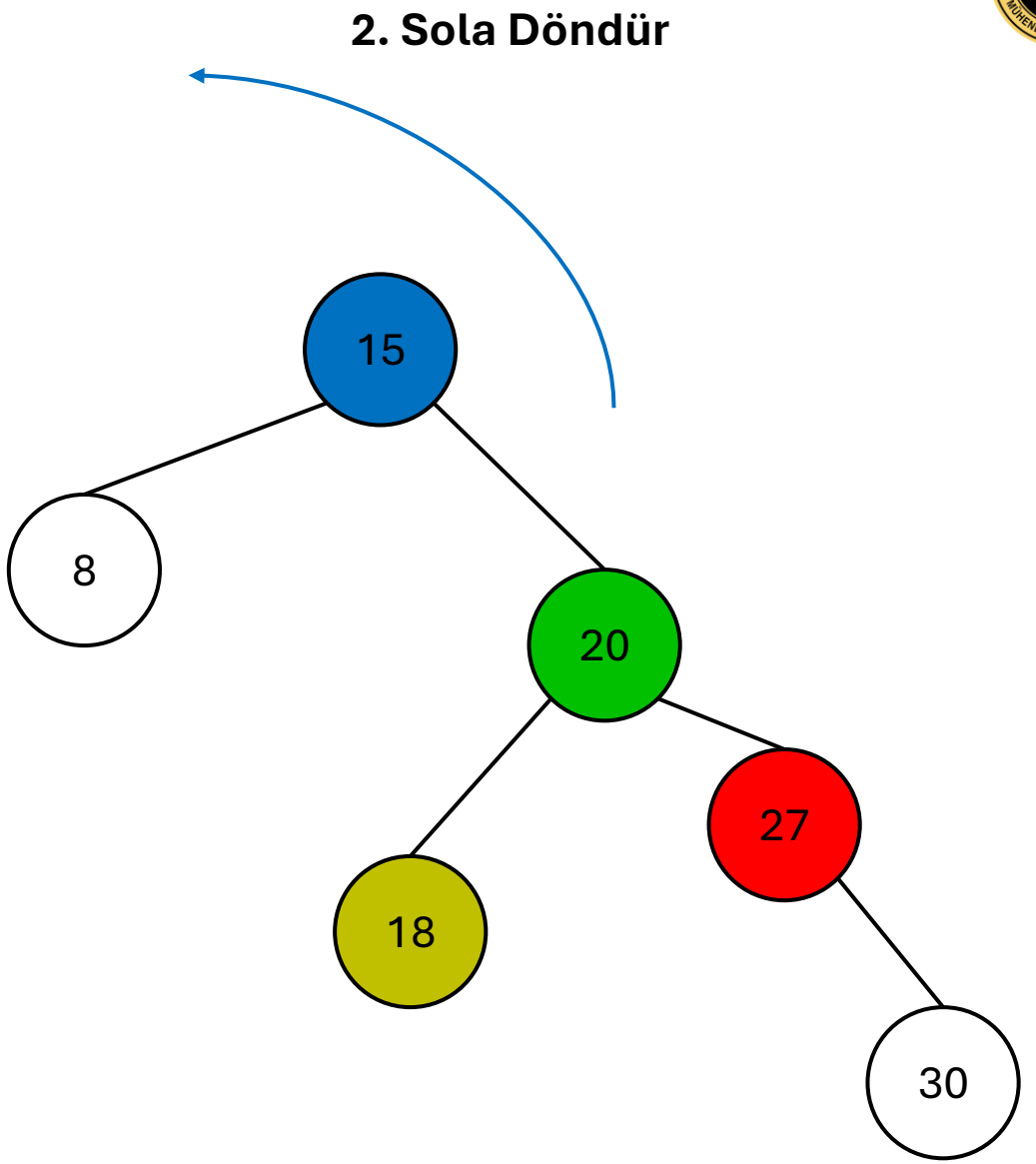
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18
- Çözüm
  - **Önce sağ çocuğu sağa döndür**
  - Sonra düğümün kendisini sola döndür



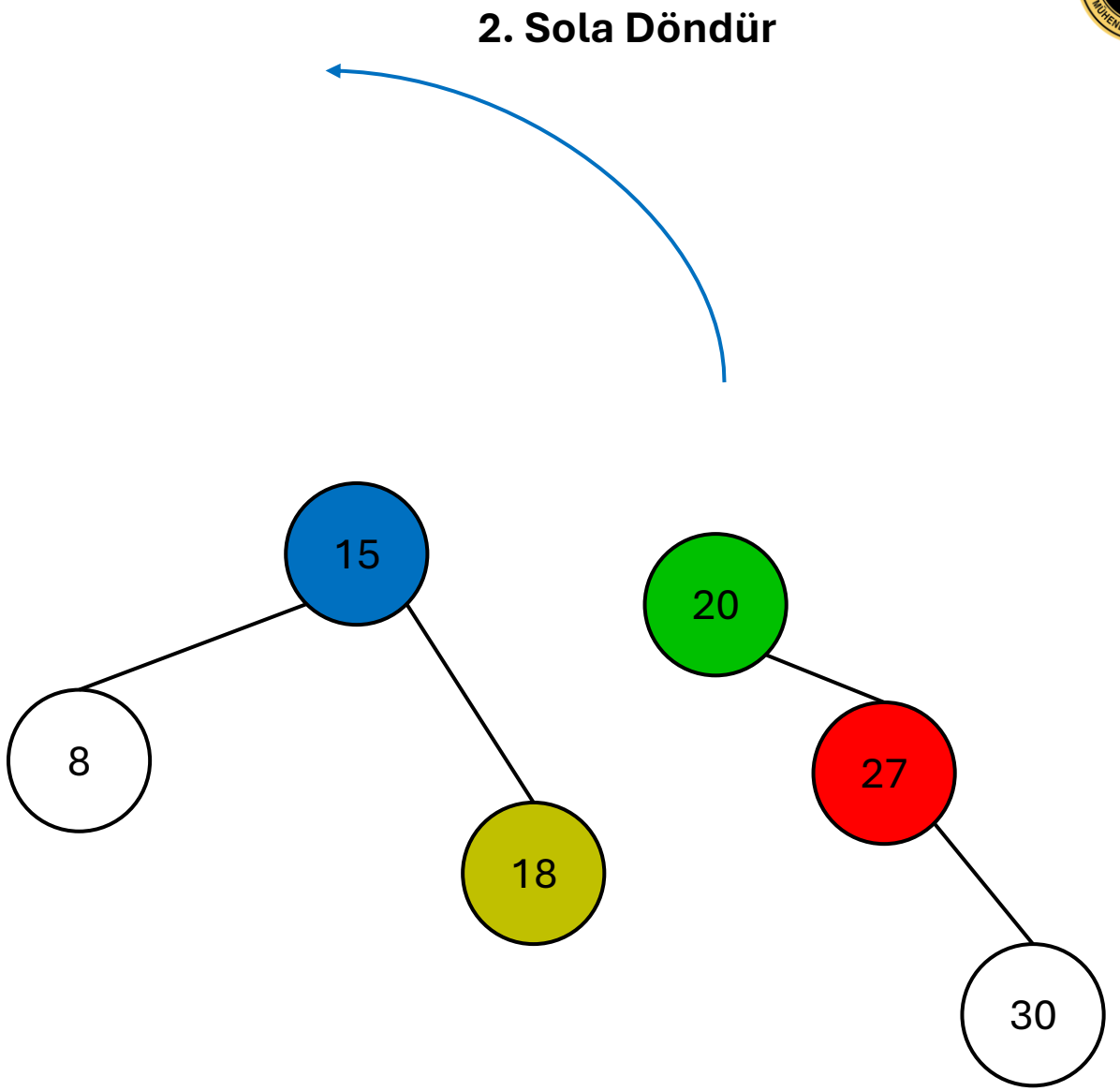
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18
- Çözüm
  - Önce sağ çocuğu sağa döndür
  - **Sonra düğümün kendisini sola döndür**



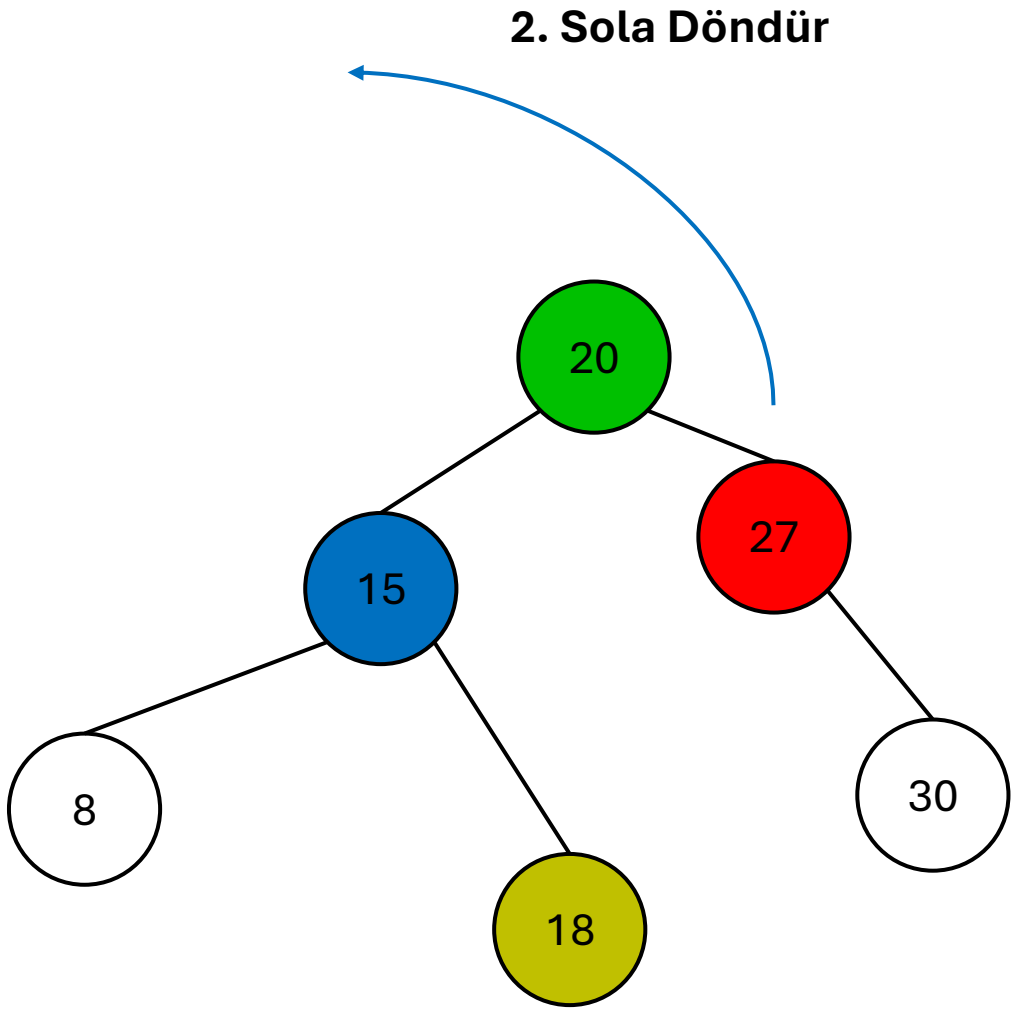
AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18
- Çözüm
  - Önce sağ çocuğu sağa döndür
  - **Sonra düğümün kendisini sola döndür**



AVL Tree

- RL Dengesizliği
  - Düğümün sağ altağacının soluna eklenmesi
  - Ekle: 18
- Çözüm
  - Önce sağ çocuğu sağa döndür
  - **Sonra düğümün kendisini sola döndür**



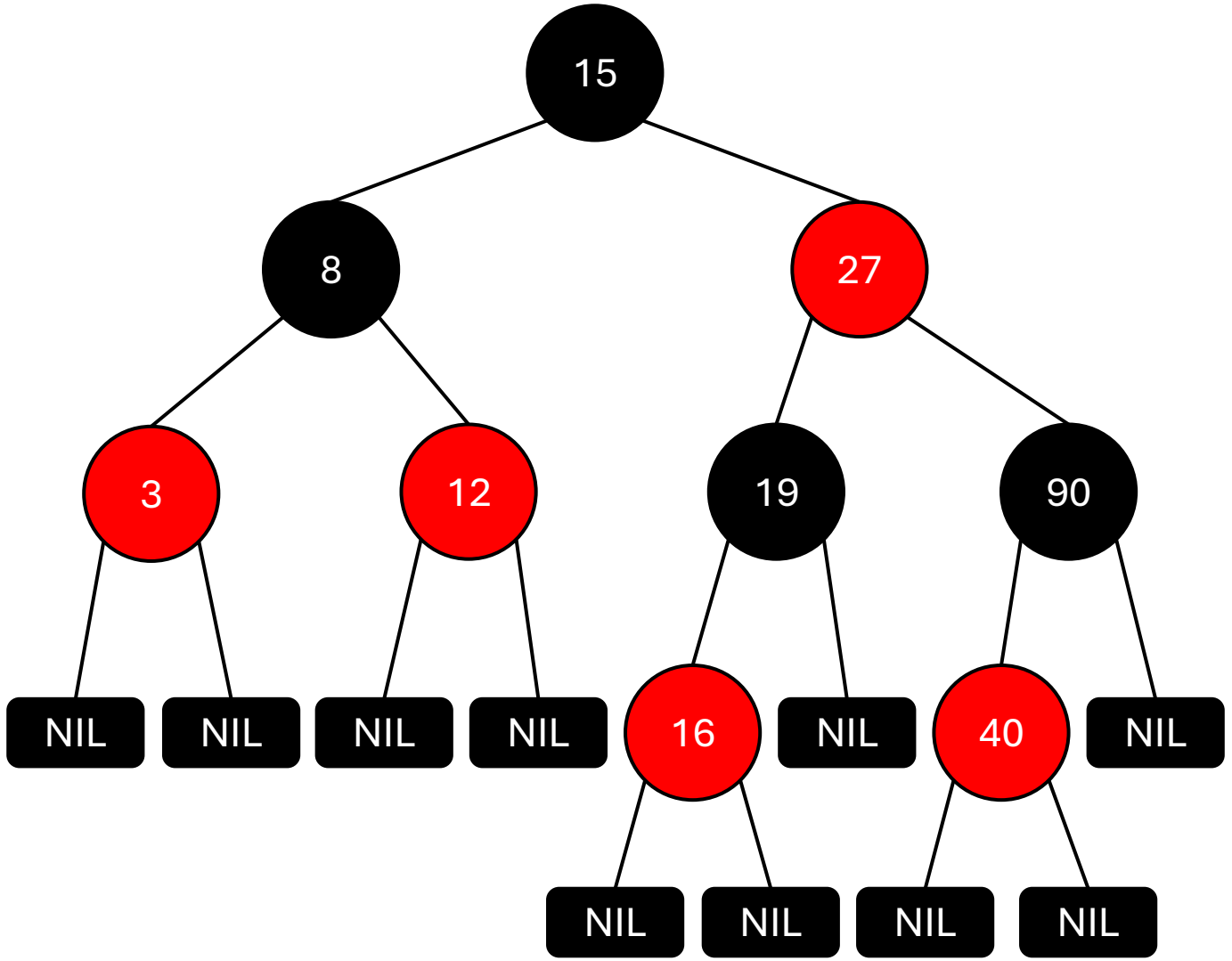
Red Black Tree

- Red-Black ağaçlar kendi denge durumunu koruyan BST ağacıdır.
  - AVL ağaçlarından farklı olarak denge durumu katı bir şekilde gözetilmez
  - Ancak hiçbir yolun diğerinden iki kat daha uzun olmamasını sağlayarak ağacın yaklaşık olarak dengede kalmasını garanti eder.
- Her düğümün ekstra olarak renk özelliği mevcuttur
  - **Kırmızı** ya da **siyah**.
- Renkler, ekleme ve çıkarma sırasında denge durumunu ayarlamak için kullanılır.

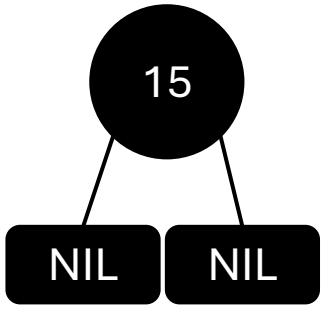
Red Black Tree

- Özellikleri
  - **Düğüm Rengi**: Her düğüm kırmızı ya da siyahtır.
  - **Kök Özelliği**: Kök düğüm her zaman **siyahtır**.
  - **Yaprak Özelliği**: Tüm yapraklar **siyahtır**. Yapraklar değer içermez ve NIL olarak isimlendirilir
  - **Kırmızı Düğüm Kuralı**: **Kırmızı** düğümlerin çocukları her zaman **siyahtır**.
  - **Siyah Düğüm Kuralı**: Herhangi bir düğümden başlamak üzere o düğümün torunu olan yapraklara (NIL düğümlere) giden tüm yollar eşit sayıda **siyah** düğüm içerir.
- Bu özellikler, kökten herhangi bir yaprağa giden en uzun yolun, en kısa yolun iki katından fazla olmamasını sağlar. Böylece ağacın dengesi ve verimli çalışması korunur.

Red Black Tree



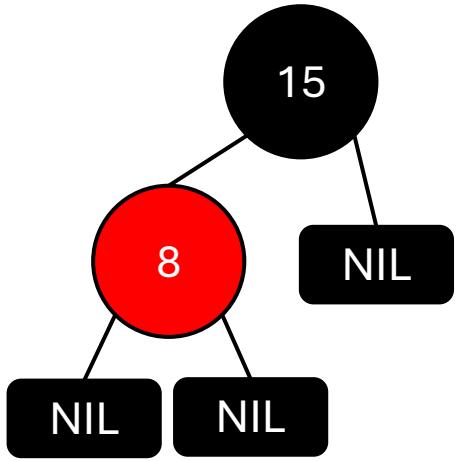
Red Black Tree



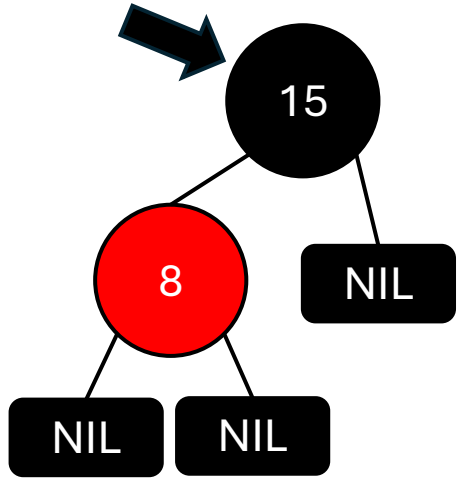
- Ekleme Adımları
  - Yeni node BST kuralına uygun olarak eklenir
    - Eklenen düğüm kök ise rengi siyah olarak ayarlanır.

Red Black Tree

- Ekleme Adımları
  - Yeni node BST kuralına uygun olarak eklenir
    - Eklenen düğüm kök ise rengi siyah olarak ayarlanır.
    - Kök dışındaki tüm düğümler kırmızı olarak eklenir.

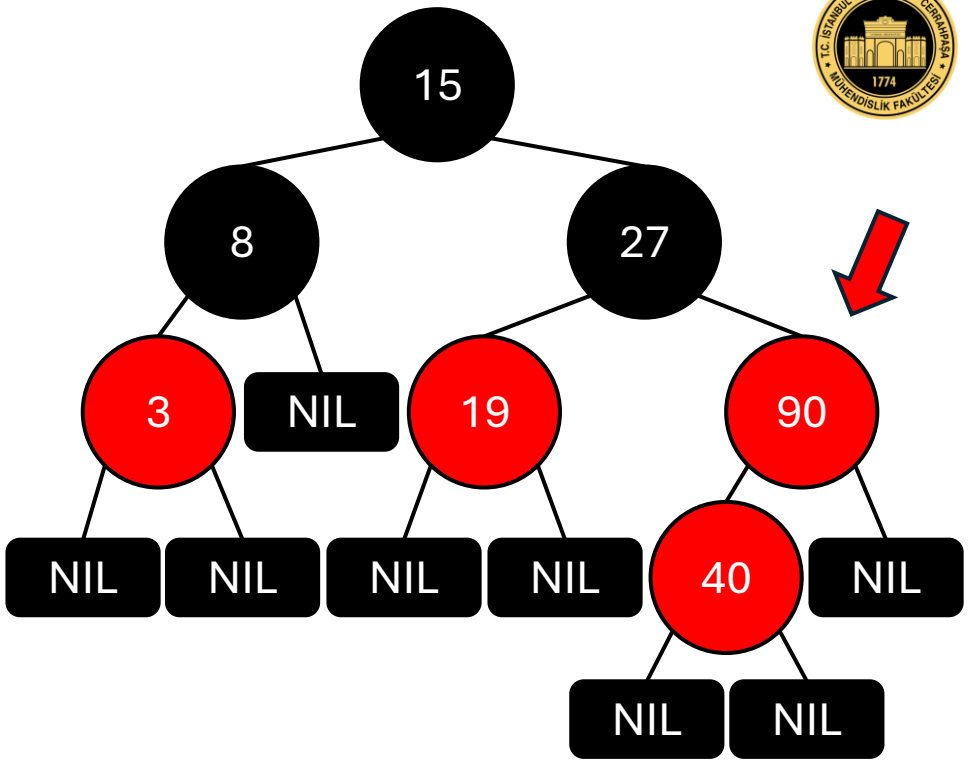


Red Black Tree



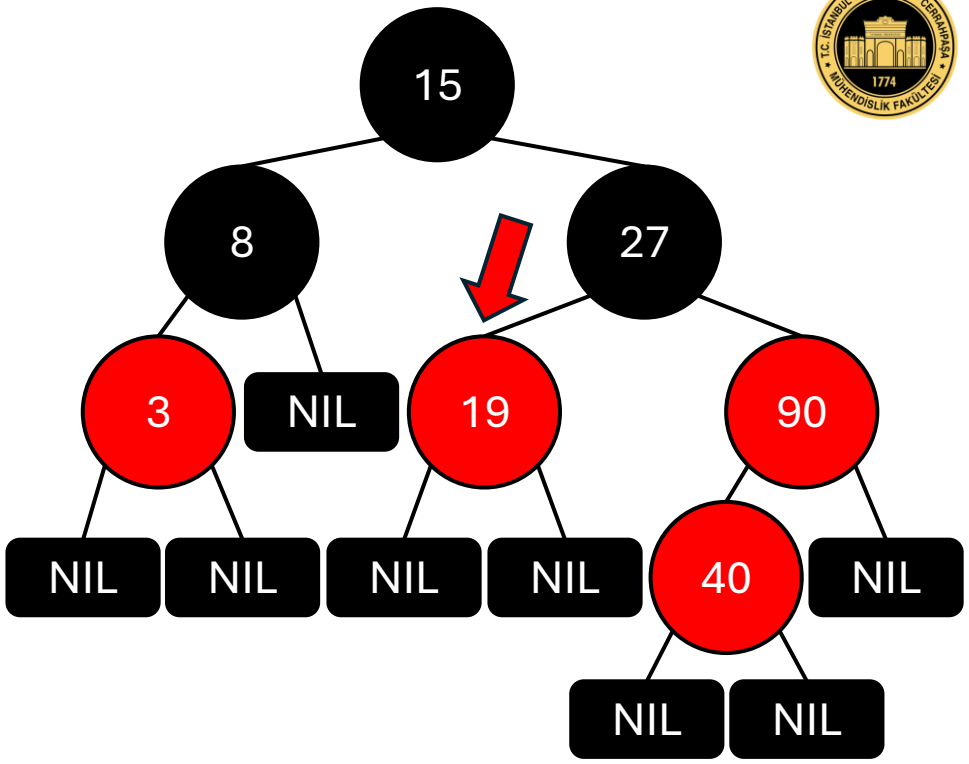
- Ekleme Adımları
  - Yeni node BST kuralına uygun olarak eklenir
    - Eklenen düğüm kök ise rengi siyah olarak ayarlanır.
    - Kök dışındaki tüm düğümler kırmızı olarak eklenir.
  - Kural İhlali Kontrolü
    - Eklenen düğümün (8) parent düğümü (15) siyah ise herhangi bir kural ihlali yok demektir.

Red Black Tree



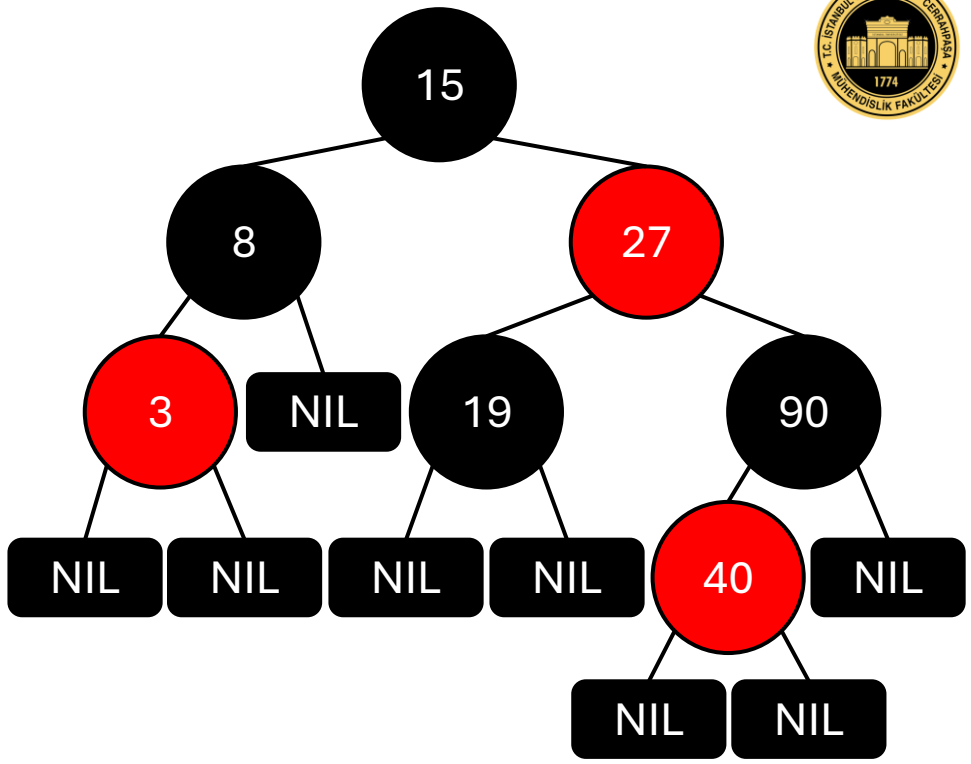
- Ekleme Adımları
  - Yeni node BST kuralına uygun olarak eklenir
    - Eklenen düğüm kök ise rengi siyah olarak ayarlanır.
    - Kök dışındaki tüm düğümler kırmızı olarak eklenir.
  - Kural İhlali Kontrolü
    - Eklenen düğümün parent düğümü siyah ise herhangi bir kural ihlali yok demektir.
    - Eklenen düğümün (40) parent düğümü (90) kırmızı ise **Kırmızı Düğüm Kuralı** ihlal edilmiş demektir. Düzeltme gerekir.

Red Black Tree



- Ekleme Adımları
  - Yeni node BST kuralına uygun olarak eklenir
    - Eklenen düğüm kök ise rengi siyah olarak ayarlanır.
    - Kök dışındaki tüm düğümler kırmızı olarak eklenir.
  - Kural İhlali Kontrolü
    - Eklenen düğümün parent düğümü siyah ise herhangi bir kural ihlali yok demektir.
    - Eklenen düğümün parent düğümü kırmızı ise **Kırmızı Düğüm Kuralı** ihlal edilmiş demektir. Düzeltme gerekir.
      - Parent kardeşi (19) kırmızı ise;

Red Black Tree



- Ekleme Adımları
  - Yeni node BST kuralına uygun olarak eklenir
    - Eklenen düğüm kök ise rengi siyah olarak ayarlanır.
    - Kök dışındaki tüm düğümler kırmızı olarak eklenir.
  - Kural İhlali Kontrolü
    - Eklenen düğümün parent düğümü siyah ise herhangi bir kural ihlali yok demektir.
    - Eklenen düğümün parent düğümü kırmızı ise **Kırmızı Düğüm Kuralı** ihlal edilmiş demektir. Düzeltme gerekir.
      - Parent kardeşi kırmızı ise;
        - Parent (90) ve kardeşinin (19) rengini siyah, grandparent (27) rengini kırmızı olarak değiştir.
        - **Kırmızı Düğüm Kuralı** ihlal kontrolünü grantparent (27) için tekrarla. Köke kadar devam et.

Red Black Tree

- Ekleme Adımları
  - Yeni node BST kuralına uygun olarak eklenir
    - Eklenen düğüm kök ise rengi siyah olarak ayarlanır.
    - Kök dışındaki tüm düğümler kırmızı olarak eklenir.
  - Kural İhlali Kontrolü
    - Eklenen düğümün parent düğümü siyah ise herhangi bir kural ihlali yok demektir.
    - Eklenen düğümün parent düğümü kırmızı ise **Kırmızı Düğüm Kuralı** ihlal edilmiş demektir. Düzeltme gerekir.
      - Parent kardeşi kırmızı ise;
        - Parent (90) ve kardeşinin (19) rengini siyah, grandparent (27) rengini kırmızı olarak değiştir.
        - **Kırmızı Düğüm Kuralı** ihlal kontrolünü grantparent (27) için tekrarla. Köke kadar devam et.
      - Parent kardeşi siyah ise;
        - AVL ağaçlarındaki 4 durumdan biri (LL, RR, LR, RL) oluşmuş demektir.
        - Gerekli döndürmeleri yaptıktan sonra renkleri tekrar düzenle.

Referanslar

- Data Structures and Algorithms in C++, Third Edition, Adam Drozden, 2005
- Introduction to Algorithms, Third Edition, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, The MIT Press, 2009
- Algorithms, 4th Edition by Robert Sedgewick and Kevin Wayne, Addison-Wesley Professional, 2011
- Principles of Data Structures Using C and C++, Vinu V Das, New Age International
- https://www.geeksforgeeks.org/dsa